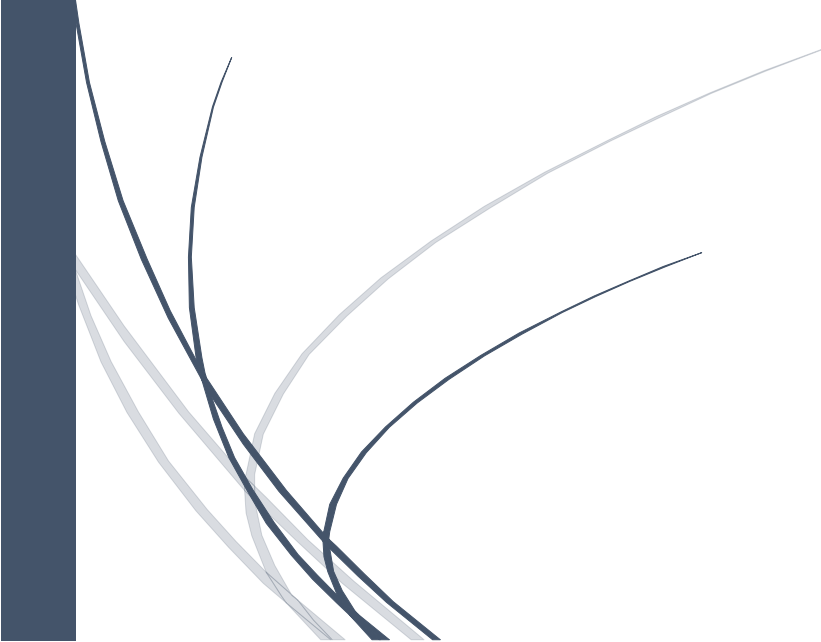




September 2026

Unconstrained Optimization

EE403



Dr Erivelton Nepomuceno
erivelton.nepomuceno@mu.ie

Module Name	Unconstrained Optimisation
Module Code	EE403
Module Co-ordinator	Refer to Excel document <i>Module_Co-ordinators</i>
Department	Electronic Engineering
Credit rating	5 ECTS credits
Pre-requisites	EE112 Engineering Mathematics 2
Co-requisites	None

Aims	• To introduce the field of unconstrained optimisation. • To outline basic line search methods for 1-variable functions • To outline basic direct and gradient-based search methods for multi-variable functions.
Learning Outcomes	At the end of this module a student should be able to: 1. LO1 Carry out basic line searches for functions with one variables. 2. LO2 Apply direct search methods to functions of more than one variable. 3. LO3 Apply gradient-based methods to functions of more than one variable. 4. LO4 Apply hessian-based methods to functions of more than one variable. 5. LO5 Solve the ‘sum of squares’ problems using the Gauss-Newton method.

Time Allowance for Constituent Elements	
Lectures / Tutorials Independent study Assignments (2 x 15 hr)	22 hours 103 hours

Indicative Syllabus

1. Fundamental principles of optimization.
2. Line searches for functions with one variable; Grid, Fibonacci, Golden Section & Quadratic.
3. Direct search methods for functions of more than one variable; Univariate, DSC algorithm, Gram-Schmidt Orthogonalisation & Simplex method.
4. Gradient methods for functions of more than one variable; Steepest Descent, Conjugate Gradient Algorithm – Fletcher & Reeves.
5. Hessian-based Gradient methods for functions of more than one variable; Newton's method, Quasi-Newton's method, DFP & BFGS algorithms.
6. The Sum of Squares Problem; Gauss-Newton and Levenberg-Marquart algorithms.
7. An introduction to constrained optimisation .

Assessment Criteria

Semester Examination	60%
Assignments (2)	20%
Class Tests	20%

- Class tests: questions in each class. Final score will be computed after eliminating the smallest 3 results.

1. Introduction to Optimisation

1.1 What is Optimisation?

- Optimisation deals with the following problem – choose the value x such that some quantity F , which depends on x , is a *minimum*. Here, F is referred to as the *objective function*.
- In mathematical terms, we express this problem as: $\min_x F(x)$
- In seeking the minimum of $F(x)$ we may often be restricted in the choice of x . Typically, there may be other quantities $C_i(x)$ whose value must be maintained within certain limits. These are called *constraints*.
- The variable x may be a scalar or an n -vector (i.e. $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_n]^T$).
- Notice that vectors are represented with bold font, while scalars are not. Also, matrices are generally represented with bold capital font. Alternatively, vectors can be represented using an underline, i.e. $\underline{x}$.
- Hence, the full expression of the problem in standard form is as follows:

$$\min \quad F(\mathbf{x}), \mathbf{x} \in \mathcal{R}^n$$

$$\begin{aligned} \text{subject to } C_i(\mathbf{x}) &= 0; i = 1, 2, \dots, k && \text{(equality constraints)} \\ C_j(\mathbf{x}) &\geq 0; j = k+1, k+2, \dots, m && \text{(inequality constraints)} \end{aligned}$$

where m represents the total number of constraints.

- Note that if $m = 0$, then we are dealing with an ***unconstrained*** optimisation problem, otherwise, if $m \neq 0$, then we have a ***constrained*** optimisation problem.
- Furthermore, note that the ***maximum*** of a function $F(\mathbf{x})$ is the same as the minimum of the negative of the function, i.e. $-F(\mathbf{x})$. In other words:

$$\max F(\mathbf{x}) \equiv \min(-F(\mathbf{x}))$$

$$C_i(\mathbf{x}) \leq 0 \equiv -C_i(\mathbf{x}) \geq 0$$

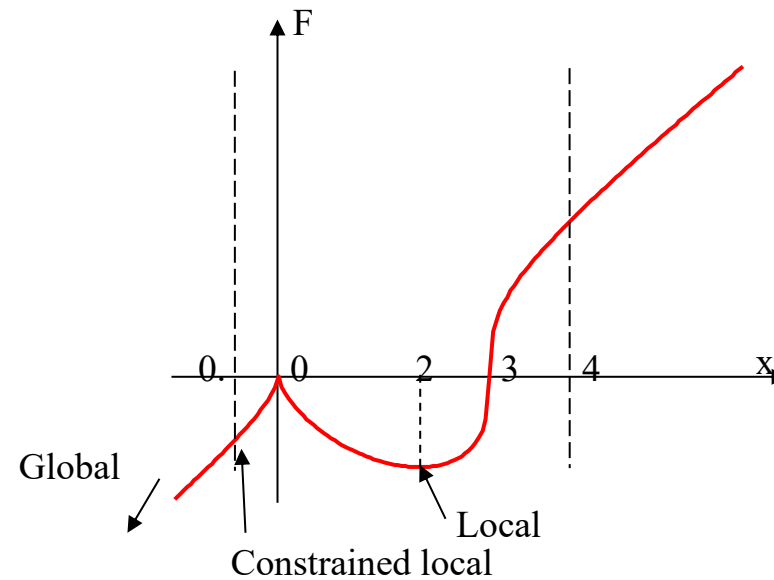
Engineering Examples of Optimisation

- In its broadest sense, optimisation can be applied to solve almost any (if not all) engineering problems.
- Some typical applications that use optimisation include:
 - Design of aircraft and aerospace structures for minimum weight.
 - Design of water resources systems for maximum benefit.
 - Design of pumps, turbines and heat transfer equipment for maximum efficiency.
 - Optimal design of electrical networks.
 - Selection of a site for an industry.
 - Analysis of statistical data and building empirical models from experimental results to obtain the most accurate representation of the physical phenomenon.
 - Allocation of resources or services among several activities to maximise benefit.
 - Controlling the waiting and idle times and queueing in production lines to reduce the costs.
 - Optimum design of control systems.
- These example applications clearly illustrate the wide scope of optimisation!

An Optimisation Example

- Consider the function $F(x) = (x^3 - 3x^2)^{\frac{1}{3}}$ where x is a scalar value (i.e. $n = 1$ in terms of the vector $\mathbf{x}!$).
- By plotting $F(x)$ versus x (see diagram below), it is easily seen that $F(x)$ is minimised at $x = 2$ (local minimum) and $x = -\infty$ (global minimum).
- If we apply constraints so that x must lie in the range -0.5 to 4 , then our optimisation problem can be stated as:

$$\min_{x \in \mathbb{R}^1} = (x^3 - 3x^2)^{\frac{1}{3}} \text{ subject to } (x + 0.5) \geq 0 \text{ and } (-x + 4) \geq 0$$

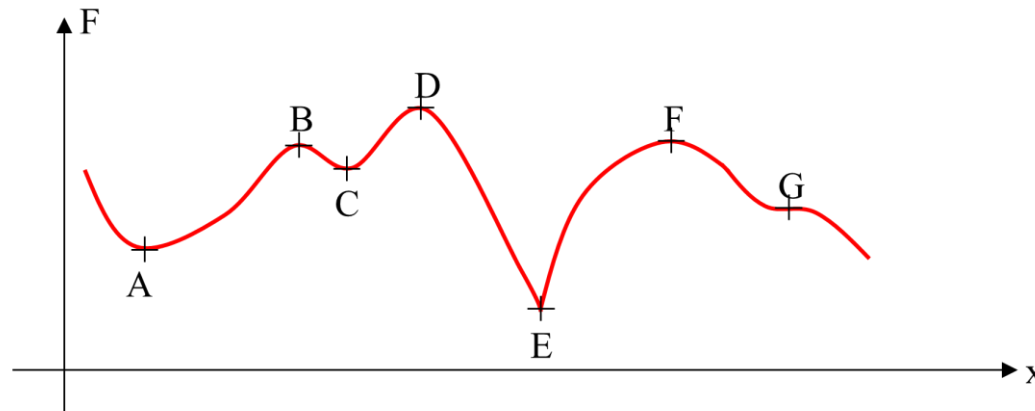


- In this case, F has minima at $x = 2$ and $x = -0.5$.
- This example illustrates some of the usual features of optimisation problems.
- They are nonlinear and constraints may or may not exist.
- It is worth noting that in this example, the equation of $F(\mathbf{x})$ was known.

- In general, it may or may not be possible to express the objective function $F(\mathbf{x})$ and the constraints $C_i(\mathbf{x})$ analytically. However, this is okay, as it is only necessary that these functions can be computed for any given value of $\mathbf{x}$.
- Analytical solutions are hardly ever available and so iterative methods are used.

1.2 Fundamental Principles

- Consider the following arbitrary graph, where all points shown are stationary points, i.e. the differential of the function at these points is zero.



- The points A, C and E are all **local** minima. Each has a lower function value than any other point in its own neighbourhood. In mathematical terms, a local minimum $\mathbf{x}^*$ exists for some small value δ if:

$$F(\mathbf{x}^*) \leq F(\mathbf{x}) \text{ for all } \mathbf{x} \text{ such that } \|\mathbf{x} - \mathbf{x}^*\| < \delta.$$

- The point E is the **global** minimum in the range shown. In mathematical terms, a global minimum $\mathbf{x}^*$ exists if:

$$F(\mathbf{x}^*) \leq F(\mathbf{x}) \text{ for all } \mathbf{x}.$$

- Points B, D and F are all local maxima, with D being the global maximum in the range shown.
- Finally, point G is referred to as a point of inflexion.
- For this course, we make the following assumptions in relation to $F(x)$:
 - (i) continuous
 - (ii) smooth – i.e. its first and second derivatives are continuous (i.e. point E can't occur!)
 - (iii) single valued and dependent for every value of x , i.e. for each value of x there is one and only one corresponding value of $F(x)$.

Basic optimality condition – 1 variable

- Let x^* denote the value of x at a local minimum.
- Consider any value of x near x^* , i.e. $x = x^* + h$, where h is sufficiently small. Using Taylor's Theorem we obtain the following expansion:

$$F(x^* + h) = F(x^*) + h \left. \frac{dF}{dx} \right|_{x^*} + \frac{1}{2} h^2 \left. \frac{d^2 F}{dx^2} \right|_{x^*} + \dots$$

- Therefore:

$$F(x^* + h) - F(x^*) = h \left. \frac{dF}{dx} \right|_{x^*} + \frac{1}{2} h^2 \left. \frac{d^2 F}{dx^2} \right|_{x^*} + \dots$$

- We know that $F(x^* + h) - F(x^*) > 0$ for a local minimum at x^* given that $|h|$, i.e. $|x - x^*|$, is sufficiently small.
- Furthermore, we know that the first derivative at a minimum (or maximum) is zero.
- Therefore, **a sufficient condition for a local minimum** is:

$$\left. \frac{dF}{dx} \right|_{x^*} = 0 \quad \text{and} \quad \left. \frac{d^2F}{dx^2} \right|_{x^*} > 0$$

- Note that a global minimum is also a local minimum but there is no local criterion for verifying it. We would need to compare all local minimum points to determine which one is the global minimum in the range examined!
- **Example** - determine the minimum point for the function $f(x) = 3x^3 + 9x^2 - 4$:

$$f(x) = 3x^3 + 9x^2 - 4$$

$$f'(x) = 9x^2 + 18x = 0 \Rightarrow x = 0 \text{ or } -2$$

$$\text{and } f''(x) = 18x + 18$$

- At $x = 0$, $f''(x) = 18 > 0$ which means that there's a minimum point at $x = 0$.
- At $x = -2$, $f''(x) = -18 < 0$ which means that the point $x = -2$ is not a minimum (it is in fact a maximum).
- Determining $f(x)$ at $x = 0$ gives $f(0) = -4$. Hence the function $f(x)$ has a minimum point at $(0, -4)$.

Basic optimality condition – n variables

- Let $\mathbf{x}^*$ be a local minimum. Then, at a point $\mathbf{x}^* + \mathbf{h}$ we have, by Taylor's Theorem:

$$F(\mathbf{x}^* + \mathbf{h}) = F(\mathbf{x}^*) + \mathbf{h}^T \nabla F(\mathbf{x}^*) + \frac{1}{2} \mathbf{h}^T \nabla^2 F(\mathbf{x}^*) \mathbf{h} + \dots$$

- Here, ∇F (Grad F) and $\nabla^2 F$ (Hessian Matrix of F) are given by:

$$\nabla F = \begin{bmatrix} \partial F / \partial x_1 \\ \partial F / \partial x_2 \\ \vdots \\ \partial F / \partial x_n \end{bmatrix} \quad \nabla^2 F = \begin{bmatrix} \partial^2 F / \partial x_1^2 & \partial^2 F / \partial x_1 \partial x_2 & \dots & \partial^2 F / \partial x_1 \partial x_n \\ \partial^2 F / \partial x_2 \partial x_1 & \partial^2 F / \partial x_2^2 & & \\ \vdots & & \ddots & \vdots \\ \partial^2 F / \partial x_n \partial x_1 & & \dots & \partial^2 F / \partial x_n^2 \end{bmatrix}$$

- Consider the case for a 3-dimensional function, i.e. $n = 3$. Hence $\mathbf{x} = [x_1 \ x_2 \ x_3]^T$ and:

$$F(x^* + h) = F(x^*) + [h_1 \quad h_2 \quad h_3] \begin{bmatrix} \partial F / \partial x_1 \\ \partial F / \partial x_2 \\ \partial F / \partial x_3 \end{bmatrix} \\ + \frac{1}{2} [h_1 \quad h_2 \quad h_3] \begin{bmatrix} \partial^2 F / \partial x_1^2 & \partial^2 F / \partial x_1 \partial x_2 & \partial^2 F / \partial x_1 \partial x_3 \\ \partial^2 F / \partial x_2 \partial x_1 & \partial^2 F / \partial x_2^2 & \partial^2 F / \partial x_2 \partial x_3 \\ \partial^2 F / \partial x_3 \partial x_1 & \partial^2 F / \partial x_3 \partial x_2 & \partial^2 F / \partial x_3^2 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} + \dots$$

- Note that this can be expressed as follows:

$$F(x^* + h) = F(x^*) + \sum_{i=1}^3 h_i \frac{\partial F}{\partial x_i} + \frac{1}{2} \sum_{i=1}^3 \sum_{j=1}^3 h_i h_j \frac{\partial^2 F}{\partial x_i \partial x_j} + \dots$$

- For $\|\mathbf{h}\|$ sufficiently small, $F(\mathbf{x}^* + \mathbf{h}) - F(\mathbf{x}^*)$ cannot be < 0 , by definition of a local minimum.

- Hence, we can deduce that $\nabla F(\mathbf{x}^*) = \mathbf{0}$, i.e. a vector of zeros. If this wasn't the case, then we could choose a value of $\mathbf{h}$ that would make $F(\mathbf{x}^* + \mathbf{h}) - F(\mathbf{x}^*)$ negative!
- Therefore the sign of $F(\mathbf{x}^* + \mathbf{h}) - F(\mathbf{x}^*)$ is determined by:

$$\frac{1}{2} \mathbf{h}^T \nabla^2 F(\mathbf{x}^*) \mathbf{h}$$

- Hence, to ensure a local minimum the term $\frac{1}{2} \mathbf{h}^T \nabla^2 F(\mathbf{x}^*) \mathbf{h}$ must be ≥ 0 for all $\mathbf{h}$ for which $\|\mathbf{h}\|$ is sufficiently small.
- Hence the Hessian matrix at $\mathbf{x}^*$ must be **positive definite**.
- Therefore, a **sufficient condition for a local minimum for an n-variable continuous, differentiable function** is:

$$\nabla F(\mathbf{x}^*) = \mathbf{0} \text{ and } \nabla^2 F(\mathbf{x}^*) \text{ is positive definite.}$$

- Note that $\nabla^2 F$ is an $n \times n$ **symmetric** matrix because:

$$\nabla^2 F_{ij} = \nabla^2 F_{ji}, \text{ i.e.: } \frac{\partial^2 F}{\partial x_i \partial x_j} = \frac{\partial^2 F}{\partial x_j \partial x_i}.$$

A note on positive definite matrices

- A positive definite matrix $\mathbf{P}$ has the property:

$\mathbf{z}^T \mathbf{P} \mathbf{z} > 0$ if $\mathbf{P}$ is an $n \times n$ matrix and $\mathbf{z}$ is any non-null n -vector.

- For example, consider the matrix $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$, hence $n = 2$ and:

$$\begin{bmatrix} z_1 & z_2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = z_1^2 + 2z_2^2 > 0, z \neq 0$$

- Therefore $\mathbf{A}$ is positive definite.
- Now consider $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, hence $n = 2$ and:

$$\begin{aligned} \begin{bmatrix} z_1 & z_2 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} &= \begin{bmatrix} z_1 + 3z_2 & 2z_1 + 4z_2 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \\ &= (z_1^2 + 3z_1z_2) + (2z_1z_2 + 4z_2^2) \end{aligned}$$

$$= z_1^2 + 5z_1z_2 + 4z_2^2$$

- If we take $z_1 = -2$ and $z_2 = 1$, this gives a value of -2 which is < 0 . Hence B is not a positive definite matrix.

- **Example** - determine the minimum point(s) for the following function:

$$f(x) = x_1^2 + x_2^2 + x_3^2 - 4x_1 - 8x_2 - 12x_3 + 100$$

- Grad of $f(x)$ is given by:

$$\nabla f = \begin{bmatrix} \partial f / \partial x_1 \\ \partial f / \partial x_2 \\ \partial f / \partial x_3 \end{bmatrix} = \begin{bmatrix} 2x_1 - 4 \\ 2x_2 - 8 \\ 2x_3 - 12 \end{bmatrix}$$

- Solving $\nabla F = \mathbf{0}$ gives:

$$x_1 = 2, \quad x_2 = 4, \quad x_3 = 6$$

- The Hessian matrix is given by:

$$\nabla^2 f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_1 \partial x_3} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \frac{\partial^2 f}{\partial x_2 \partial x_3} \\ \frac{\partial^2 f}{\partial x_3 \partial x_1} & \frac{\partial^2 f}{\partial x_3 \partial x_2} & \frac{\partial^2 f}{\partial x_3^2} \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

- We can easily show that this matrix is positive definite. Hence the point (2, 4, 6) is a minimum of the function $f(\mathbf{x})$ and the value at this point is 44.

2. Line searches & functions of one variable

- When it comes to minimising a function of several variables, we tend to use repeated minimisation of a function along search directions. In other words, minimising a multi-variable function involves repeated minimisation of the function for a single variable. We will look at this in more detail in due course.
- Nevertheless, the fact remains that the procedure of finding the minimum of functions of one variable is an important element in any optimisation method.
- We are now going to deal with such procedures in this section of the notes. In particular we are going to concentrate on two types of search methods – one based on fixed interval reductions (Grid, Fibonacci and Golden Section), the other based on curve fitting (Quadratic).
- Note that these search methods are iterative approaches which continue until the minimum is found to be within the required degree of accuracy. ***As such, these methods are better suited for computer simulation!***
- This approach is useful in cases where:

(1) a mathematical expression for $F(x)$ is unknown

(2) calculating the solution of $\nabla F(x) = 0$ proves exceedingly difficult!

- In the case of (1) above, an example might be an industrial process that we do not have a mathematical expression for. However, we can still measure the output for given inputs by physically conducting the actual process itself.
- This example highlights one of the key features of a good optimisation algorithm. Running the industrial process is timely and expensive and therefore each time we want to see the effect of an input on the industrial process means extra cost for the company involved.
- Clearly, therefore, ***a good optimisation is one that can converge on the minimum in the least possible number of function evaluations.***
- Finally, note that all these search methods assume that we can locate a range $[a, b]$ in which a single local minimum exists. In practice, how do we find such an interval?

Finding interval containing minimum

- The first step is to calculate $F(x)$ at some point $x = k$.

- We then need to find a point $b = k + h$, where h is a positive step.
- If $F(k+h) > F(k)$, then we are moving away from the minimum and therefore, we now need to move in the opposite direction from k , i.e. we need to consider $F(k-h)$.
- If $F(k-h) > F(k)$, then we have found a bracketing interval $[k-h, k+h]$.
- If not then we move further in the same direction, subtracting $2h$ each time we fail to establish an end for the bracketing interval, i.e. we consider $F(k-3h)$, $F(k-7h)$, etc.
- If $F(k+h) < F(k)$, however, then we continue moving in the same direction and consider $F(k+3h)$, $F(k+7h)$, etc... until we find a function value greater than $F(k+h)$.
- So, let us say that we find $F(k+3h) > F(k+h)$, then the bracketing interval is $[k, k+3h]$.

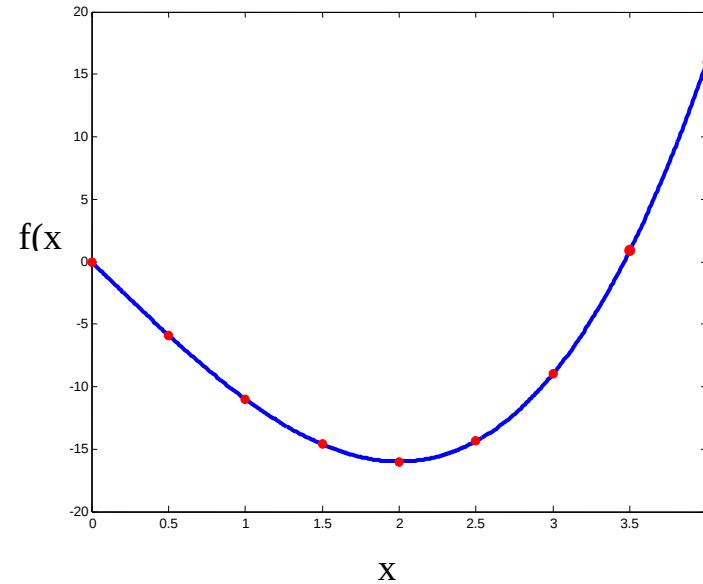
2.1 The Grid method

- This is the simplest method of determining the minimum of $F(x)$.

- We split the range $[a, b]$ into a number of subintervals of equal size. We then identify the subinterval which contains the minimum and repeat the process of subdividing.
- This process continues until we obtain a subinterval of suitable accuracy.

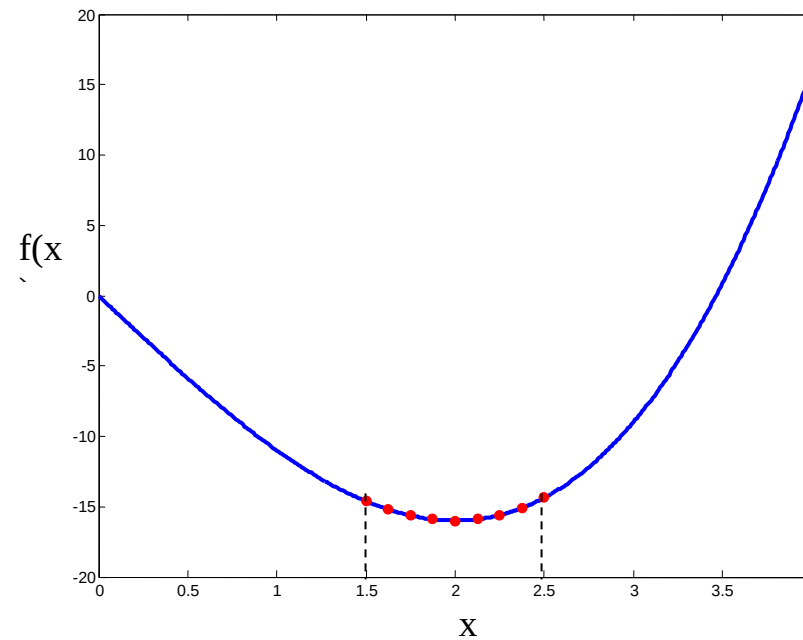
Illustrative Example

- Consider the following function: $f(x) = x^3 - 12x$
- Given that a local minimum exists in the range $[0, 4]$, determine the value of this minimum with an error in this location of less than 0.001.
- Let us take a grid with 8 divisions (arbitrarily chosen). The result of the first iteration is shown below:



- Notice that the function has been divided into 8 subintervals. The function output is calculated at each one and the lowest one is determined.
- In this case the function output at $x = 2$, i.e. $f(2) = -16$, is the lowest obtained. Our new interval needs to bracket this value. Hence, we obtain the interval $[1.5 \ 2.5]$. We know that this interval contains the local minimum.

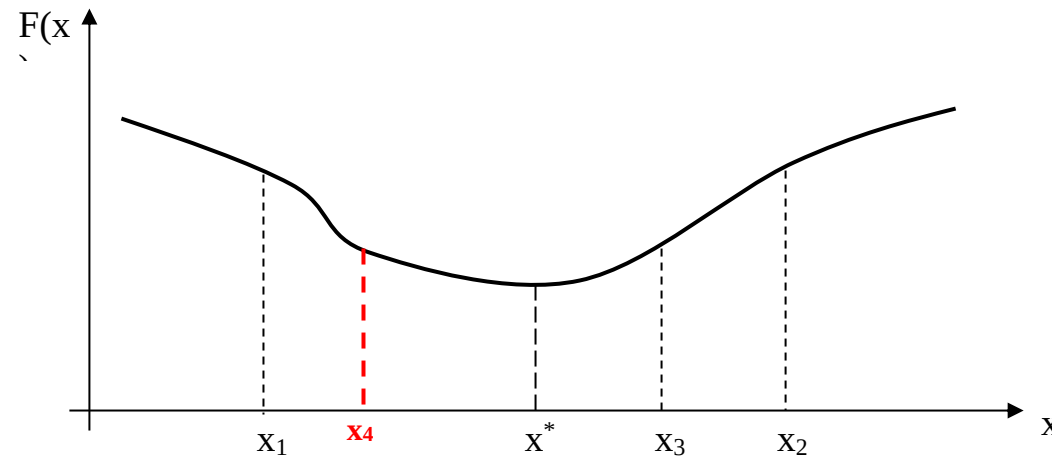
- The size of this interval is 1 which is clearly greater than 0.001, the acceptable error value, and therefore we need to continue.
- The second iteration gives:



- Now we see that the subinterval from the first iteration has been further subdivided. This leads to the new, even smaller, interval $[1.875 \ 2.125]$, which still gives a bound with an error greater than 0.001.
- After 6 iterations we obtain the acceptable interval of $[1.9995 \ 2.0005]$. The actual minimum of 2 is clearly within this interval.
- This method guarantees that we reach the minimum (within an acceptable error limit). However, its downside is that it requires a lot of function evaluations in order to achieve the minimum.
- In this example, we required 9 function evaluations on the first iteration, followed by 7 more per each iteration after this. Note, we do not need 9, as the outside two for each interval has already been evaluated on the previous iteration. Hence, in this case, we need 44 function evaluations in total.
- In order to improve on the efficiency of the grid method, we need to remove the restriction of dividing the original interval into equal size subintervals. This provides the motivation behind the Fibonacci and the Golden Section search methods.

2.2 The Fibonacci search

- Consider the case where we want to obtain the local minimum, i.e. an interval of suitable accuracy, and we have only n evaluations to do so.
- Clearly it is unwise to use up all n evaluations on the first iteration. Instead, we should use the function evaluations we obtain on one iteration to determine the position of our next function evaluation. In other words, we learn information from one iteration and use this to improve our chances of locating the minimum on the next iteration.
- Now consider the following diagram:

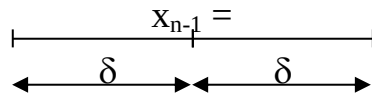


- Let us say that we know the minimum is located between the values x_1 and x_2 and that we know the value of $F(x_3)$ where x_3 lies in the range $[x_1, x_2]$.
- If we were allowed to carry out one more function evaluation, where should we locate the point x_4 in order to obtain the smallest possible interval in which the minimum is certainly located?
- For convenience let $x_3 - x_1 = L$ and $x_2 - x_4 = R$.
- Now consider the following two cases:
 - (a) if $F(x_4) < F(x_3)$ then the new interval for consideration will be $[x_1, x_3]$ of length $x_3 - x_1 = L$
 - (b) if $F(x_4) > F(x_3)$ then the new interval for consideration will be $[x_4, x_2]$ of length $x_2 - x_4 = R$
- Since we will not know in advance which of these two cases will occur, we choose the position of x_4 so as to minimise the larger of L and R .

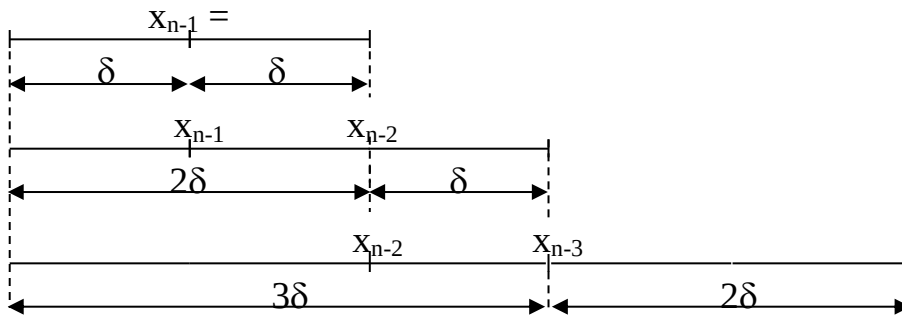
- We achieve this by simply making $L = R$, i.e.:

$$x_3 - x_1 = x_2 - x_4 \text{ or } x_4 = x_1 + (x_2 - x_3).$$

- Hence x_4 is placed symmetrically in the interval with respect to x_3 .
- Any other position of x_4 will result in either $L < R$ or $L > R$ and therefore can result in a new interval that is bigger than that obtained for $L = R$.
- We can now repeat this process for each new interval until we obtain an interval that bounds the minimum within a certain acceptable limit.
- So how do we determine the starting location for our first iteration? In other words how far from x_1 do we place x_3 (or, by symmetry, how far from x_2 do we place x_4)?
- In order to determine this we need to consider the end point and work backwards.
- Given that we want to obtain our local minimum with an error tolerance of less than ε , we need to obtain its estimate at the centre of the interval 2δ of size 2ε or less. Hence, our last useful interval can be represented as follows:



- Working backwards gives us the following pattern:



- This gives us the sequence $\delta, \delta, 2\delta, 3\delta, 5\delta, 8\delta$, etc. Here the interval δ is obtained from half the last useful interval.

- The reason why 2δ is the last useful interval is because $x_n = x_{n-1}$ and therefore no new information regarding the minimum is obtained. In other words, which half of this interval is the minimum if $F(x_n) = F(x_{n-1})$?
- This sequence is the same as the Fibonacci sequence (hence the name of the search method) which is defined by:

$$F_i = F_{i-1} + F_{i-2}, \text{ with } F_0 = F_1 = 1$$

(do not confuse the function notation $F_i(\cdot)$ with the Fibonacci number F_i)

- This gives the numbers: 1, 1, 2, 3, 5, 8, 13, 21, etc...
- Therefore, given an interval $[a, b]$ and a final one of size δ , we can build the sequence:

$$\delta, \delta, 2\delta, 3\delta, 5\delta, 8\delta, \dots, F_i\delta, \dots$$

- Letting N be the number of iterations it takes to reduce the interval $[a, b]$ to the required interval δ , we can make:

$$F_N\delta = b - a$$

- Hence: $\delta = \frac{b-a}{F_N}$

- Thus, given a , b and ε , we finally need to determine the smallest Fibonacci number that satisfies:

$$\delta \leq \varepsilon$$

- The internal points are then obtained using the following relationships:

$$x_l = b - \delta F_{N-1} \quad \text{and} \quad x_r = a + \delta F_{N-1}$$

where a and b are the bounds of the current interval and x_l and x_r are the new internal subdivision points.

Illustrative Example

- The best way to understand the Fibonacci Search method is by carrying out an example.
- Once again consider the function: $f(x) = x^3 - 12x$
- Given that a local minimum exists in the range $[0, 4]$, use the Fibonacci search method to determine the value of this minimum with an error in this location of less than 0.001.

- Firstly, we need to determine the smallest Fibonacci number that satisfies $\delta \leq \varepsilon$:

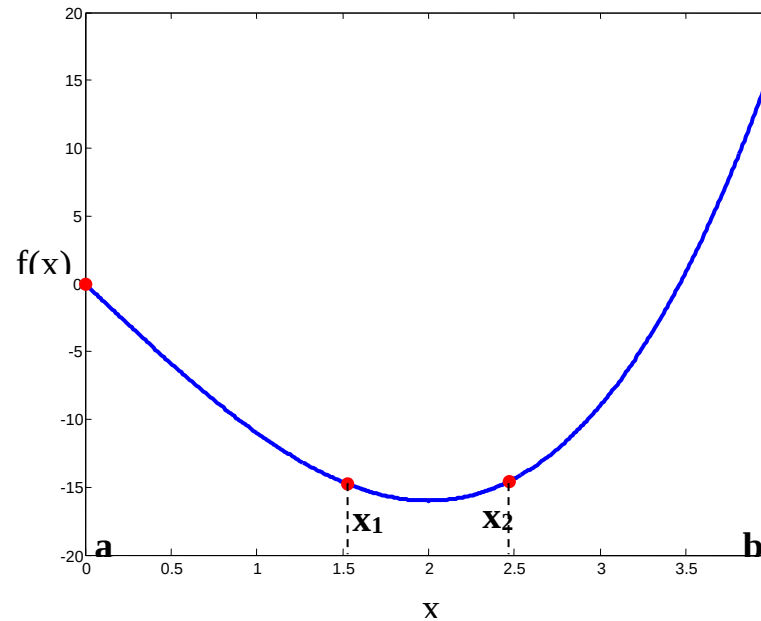
$$\delta = \frac{b-a}{F_N} = \frac{4}{F_N} \leq 0.001 \quad \therefore F_N \geq \frac{4}{0.001} = 4000$$

- The smallest Fibonacci to satisfy this is $F_{18} = 4181$ which gives a value of $\delta = 0.0009567$.
- Noting that $F_{N-1} = F_{17} = 2584$, for our first iteration we locate:

$$x_1 = b - \delta F_{N-1} \quad \text{and} \quad x_2 = a + \delta F_{N-1}$$

$$\therefore x_1 = b - 2584\delta \quad \text{and} \quad x_2 = a + 2584\delta$$

- This gives: $x_1 = 1.528$ and $x_2 = 2.472$
- These values are shown in the first iteration below:



- Calculating the function at these new positions gives:

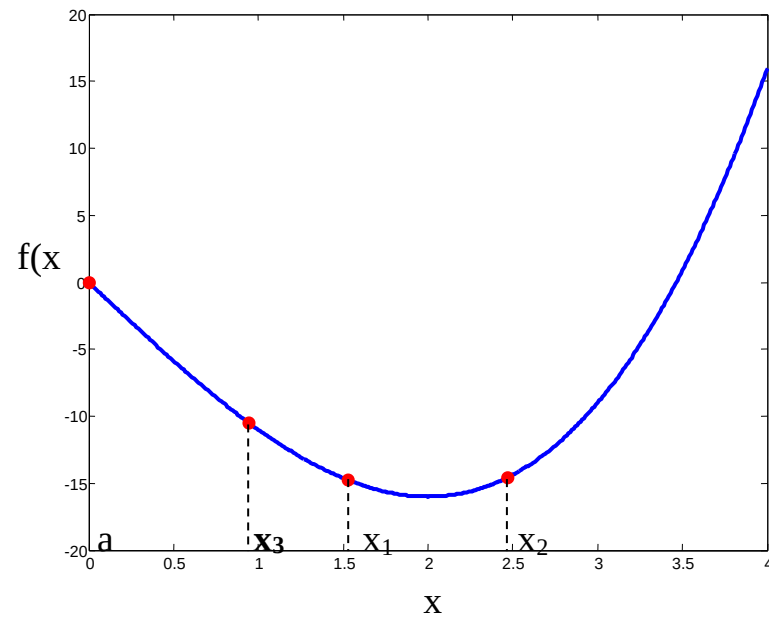
$$f(x_1) = -14.768 \quad \text{and} \quad f(x_2) = -14.557$$

- Since $f(x_1) < f(x_2)$, our new interval is given by $[a \ x_2]$ and contains the point x_1 .

- Due to symmetry, the next point x_3 must satisfy, in this case, the following equation:

$$x_1 - a = x_2 - x_3$$

- Therefore $x_3 = a + (x_2 - x_1)$ giving $x_3 = 0.944$, as illustrated in the second iteration:



- Calculating the function at x_3 gives $f(x_3) = -10.489$. Clearly, $f(x_3) > f(x_1)$ and therefore our next interval is $[x_3 \ x_2]$, containing x_1 .
- Remember, from one iteration to the next, we always make sure to bracket the minimum.
- Continuing in this manner, we eventually obtain the final interval (after 16 iterations) as:

$$[1.9986 \ 2.0005]$$

- Now, this interval is the last useful interval we obtain, and is equivalent to 2δ .
- We take our estimate of the local minimum as the midpoint of the interval, i.e. 1.9995. Hence this has a maximum error value of $\delta = 0.0009567$ (which is less than 0.001 as required).
- Note that this value is very close to the actual minimum of 2.

- The number of function evaluations required for this example is 4 for the first iteration plus 1 per each of the next 16 iterations, i.e. a total of 20 function evaluations ($4 + 16$).
- The Fibonacci search method uses a fixed number of function evaluations to determine the position of the minimum within a given maximum error.
- However, there are situations where the maximum error is not useful for obtaining the required precision – for example, we may require obtaining the position of the minimum correct to a given number of significant figures.
- Therefore, it's not always possible to obtain in advance the number of function evaluations required. Unfortunately, this value is needed by the Fibonacci search method in order to determine the position of x_1 and x_2 , that lead to the first subinterval.
- The Golden Section search method is a simple efficient alternative that provides the required property, i.e. it does not need to know the number of function evaluations in advance.

2.3 The Golden Section search

- The Golden Section search is based on the same principles as the Fibonacci search. However it differs in the way in which the location of the intervals is chosen.

- In this case, the intervals are reduced at each iteration by the same factor, τ . In other words:

$$\tau = \frac{F_{i-1}\delta}{F_i\delta}$$

where $F_i \delta$ is the length of one interval and $F_{i-1} \delta$ is the length of the subsequent reduced interval.

- Since $F_i = F_{i-1} + F_{i-2}$ (the Fibonacci sequence), we can state that:

$$F_N = F_{N-1} + F_{N-2}$$

- Dividing across by F_{N-1} gives:

$$\frac{F_N}{F_{N-1}} = 1 + \frac{F_{N-2}}{F_{N-1}} \quad \Rightarrow \quad \frac{1}{\tau} = 1 + \tau$$

- Multiplying across by τ gives:

$$1 = \tau + \tau^2 \quad \text{or} \quad \tau^2 + \tau - 1 = 0$$

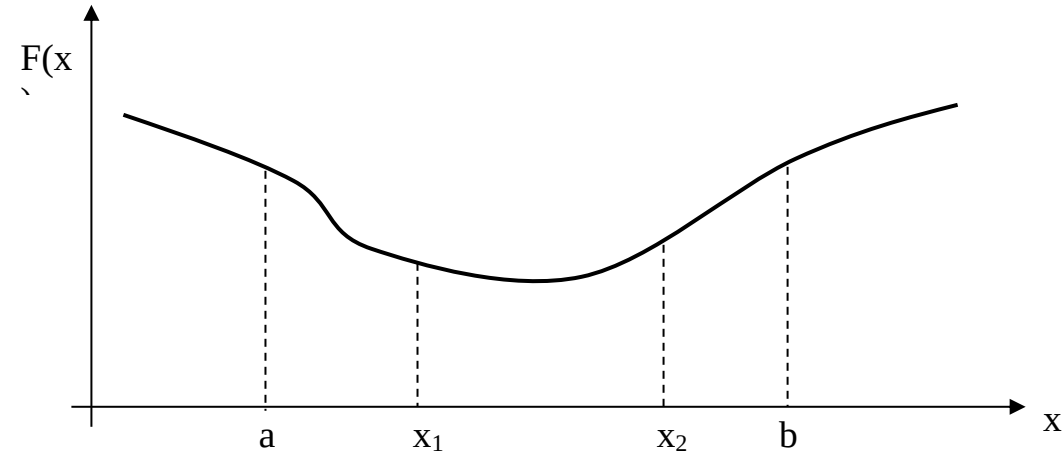
- Solving this equation (and considering only the positive root) we obtain:

$$\tau = \frac{(-1+\sqrt{5})}{2} \approx 0.6180339887$$

- This is known as the golden ratio, hence the name of the search method.
- So how do we determine the new positions x_1 and x_2 in the first interval $[a, b]$?
- Refer to the following diagram. From symmetry we know that the reduced intervals equal each other, i.e.:

$$x_2 - a = b - x_1$$

- We now know that each of these distances have to be $\tau(b - a)$, i.e. the golden ratio times the original interval length.



- Hence: $x_2 - a = \tau(b - a)$ and $b - x_1 = \tau(b - a)$
- This gives: $x_2 = a + \tau(b - a)$ and $x_1 = b - \tau(b - a)$
- These expressions are used for all subsequent intervals (with the appropriate substitutions – i.e. a and b are replaced with the new interval boundaries, etc...).

Illustrative Example

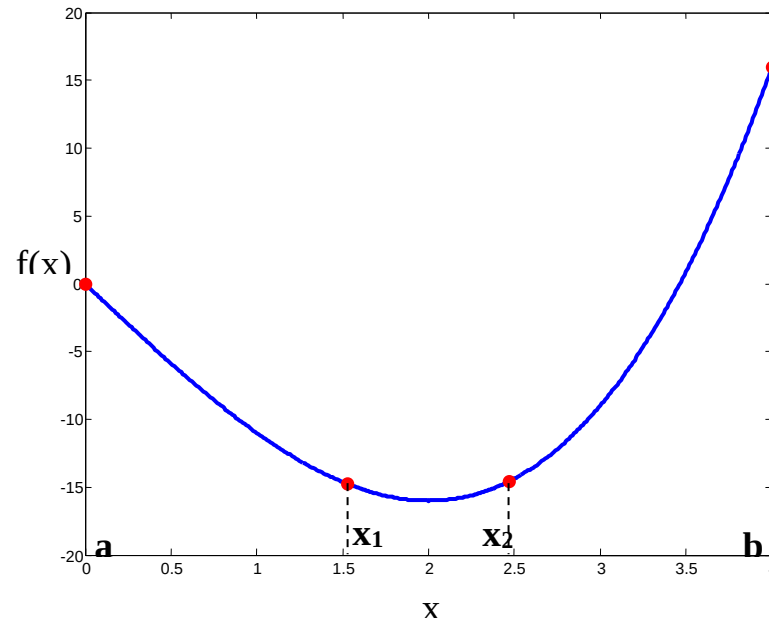
- Once again, consider the function: $f(x) = x^3 - 12x$
- Given that a local minimum exists in the range $[0, 4]$, use the Golden Section search method to determine the value of this minimum with an error in this location of less than 0.001.
- Using the above equations for x_1 and x_2 we get:

$$x_1 = b - \tau (b - a) = 4 - 0.618 (4) = 1.528$$

and

$$x_2 = a + \tau (b - a) = 0 + 0.618 (4) = 2.472$$

- Thus, the first iteration looks like:



- Calculating the function at these new positions gives:

$$f(x_1) = -14.768 \quad \text{and} \quad f(x_2) = -14.557$$

- Since $f(x_1) < f(x_2)$, our new interval is given by $[a, x_2]$ and contains the point x_1 .

- The next point x_3 has to satisfy, in this case, the following equation:

$$x_3 = x_2 - \tau (x_2 - a)$$

- This gives $x_3 = 0.944$. We could also determine this value from symmetry, i.e.:

$$x_1 - a = x_2 - x_3$$

- Therefore $x_3 = a + (x_2 - x_1)$ giving $x_3 = 0.944$.
- At this stage, it's worth pointing out that these are very similar to the results we got using the Fibonacci search method.
- This is because the two methods are very similar in the first place. Consider the ratio obtained by the Fibonacci method for large values of N .
- It can be shown that as N becomes large, the ratio of the Fibonacci numbers satisfies:

$$\lim_{N \rightarrow \infty} \frac{F_{N-1}}{F_N} = \tau = 0.6180339887$$

- For example:

$$\frac{F_{10}}{F_{11}} = \frac{89}{144} = 0.6180555 \quad \text{or} \quad \frac{F_{17}}{F_{18}} = \frac{2584}{4181} = 0.61803396$$

- Clearly these values are not too different from the golden ratio! The largest reduction of interval in the Fibonacci search is:

$$\frac{F_2}{F_3} = \frac{2}{3} = 0.6666666$$

- Hence, the Fibonacci is only slightly more efficient than the golden section search method for a few iterations. The latter has the advantage of simplicity!
- Completing our example, we eventually obtain the final, slightly different interval (after 16 iterations) of:

$$[1.9985 \ 2.0003] - x^* \text{ is located at the centre, i.e. } x^* = 1.9994$$

- The actual minimum of 2 is clearly within this interval.

- The three fixed interval reduction methods (Grid, Fibonacci and Golden section) all converge to the minimum, independently of the type of function to be optimised, so long as there is only one local minimum in the range considered.
- However, they all require a large number of function evaluations to achieve this. The next method adopts a different approach and aims to reduce the number of function evaluations required by increasing the convergence rate.

2.4 The Quadratic search (curve fitting approach) – 1 variable

- This is usually done by evaluating $F(x)$ for a number of values of x and fitting a quadratic function approximation, which can be minimised analytically.
- The approximate minimum that results is used to improve the approximating function and a new minimum is found.
- This process is repeated until a suitable minimum value is obtained.
- Let us consider that a function $F(x)$ is known at 3 points α , β and γ and has the values F_α , F_β and F_γ at these points respectively.

- We approximate $F(x)$ by the following quadratic polynomial:

$$Q(x) = Ax^2 + Bx + C$$

- Thus: $F_\alpha = A\alpha^2 + B\alpha + C$
 $F_\beta = A\beta^2 + B\beta + C$
 $F_\gamma = A\gamma^2 + B\gamma + C$

- Solving these equations for A, B and C gives:

$$A = \frac{(\gamma-\beta)F_\alpha + (\alpha-\gamma)F_\beta + (\beta-\alpha)F_\gamma}{(\alpha-\beta)(\beta-\gamma)(\gamma-\alpha)}$$

$$B = \frac{(\beta^2-\gamma^2)F_\alpha + (\gamma^2-\alpha^2)F_\beta + (\alpha^2-\beta^2)F_\gamma}{(\alpha-\beta)(\beta-\gamma)(\gamma-\alpha)}$$

$$C = \frac{\beta\gamma(\gamma-\beta)F_\alpha + \gamma\alpha(\alpha-\gamma)F_\beta + \alpha\beta(\beta-\alpha)F_\gamma}{(\alpha-\beta)(\beta-\gamma)(\gamma-\alpha)}$$

- Since differentiating $Q(x)$ gives $2Ax + B$ and differentiating again gives A , we can state that if $A > 0$, then $Q(x)$ has a minimum at the point δ , where:

$$\delta = -\frac{B}{2A}$$

- Substituting for A and B gives:

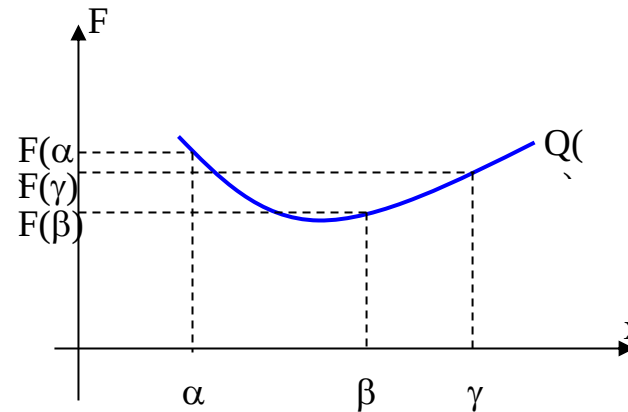
$$\delta = \frac{1}{2} \left[\frac{(\beta^2 - \gamma^2)F_\alpha + (\gamma^2 - \alpha^2)F_\beta + (\alpha^2 - \beta^2)F_\gamma}{(\gamma - \beta)F_\alpha + (\alpha - \gamma)F_\beta + (\beta - \alpha)F_\gamma} \right]$$

- An equivalent, but better conditioned formula is:

$$\delta = \frac{1}{2}(\alpha + \beta) + \frac{\frac{1}{2}(F_\alpha - F_\beta)(\beta - \gamma)(\gamma - \alpha)}{(\beta - \gamma)F_\alpha + (\gamma - \alpha)F_\beta + (\alpha - \beta)F_\gamma}$$

- $F(\delta)$ can now be calculated and the current approximation of $F(x)$, i.e. $Q(x)$ can be approved upon. This process is repeated until δ reaches an acceptable value. At the end of this process we would like $\delta \approx x^*$.

- To clarify, δ represents the minimum of $Q(x)$. Since $Q(x)$ is approximating the actual function $F(x)$ then δ represents an estimate of the actual local minimum point (i.e. x^*).
- So how do we improve the current quadratic approximation of $F(x)$? One approach is to select α , β and γ so as to bracket a minimum of $F(x)$ – i.e. so that a minimum happens in the range $[\alpha, \gamma]$.
- This can be done by ensuring that $F(\beta) < F(\alpha)$ and $F(\beta) < F(\gamma)$, as illustrated below:



- This ensures $A > 0$ and $Q(x)$ has a minimum. Then one of these points is replaced by δ in such a way that this property is preserved.

- In this way A is always positive and $Q(x)$ always has a minimum. The sequence of δ s will hopefully converge to x^* , a local minimum of $F(x)$.
- **Detailed illustrative example** – Use the quadratic search method to find the local minimum of the following one variable function:

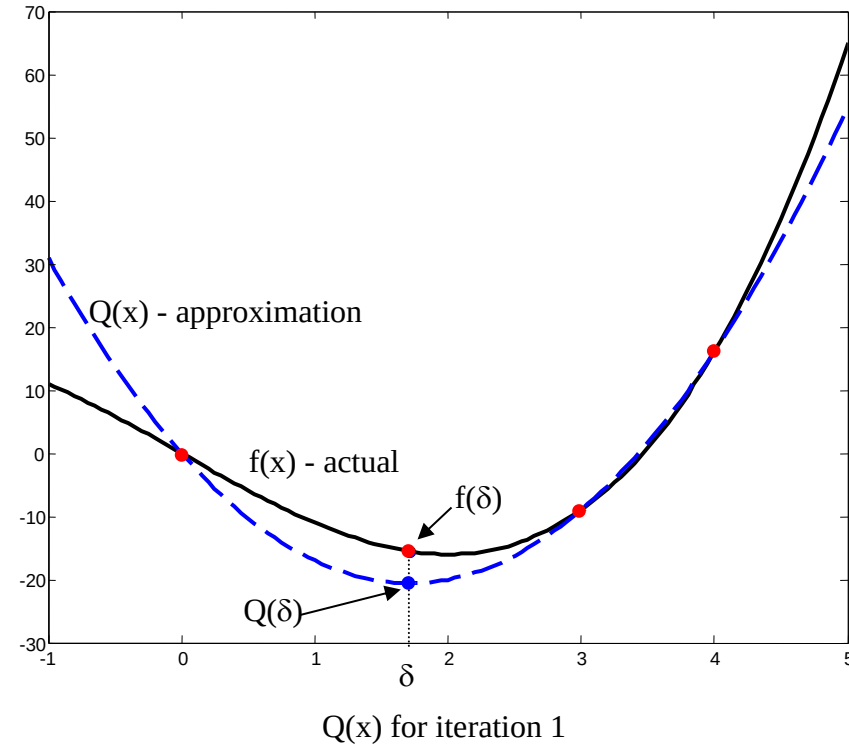
$$f(x) = x^3 - 12x$$

- The first step is to obtain three values (α, β, γ) for this function that satisfies the condition $f(\beta) < f(\alpha)$ and $f(\beta) < f(\gamma)$.
- A quick sketch of the function gives the values:

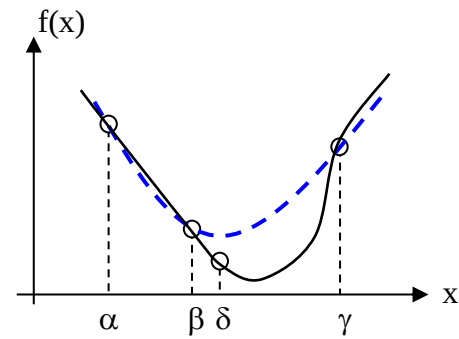
$$\alpha = 0, \beta = 3, \gamma = 4 \quad \text{and} \quad f(\alpha) = 0, f(\beta) = -9, f(\gamma) = 16$$

- Of course, there are numerous other values that we could have chosen instead!
- We can now use these values to determine the values of A , B and C and hence obtain a quadratic function approximation $Q(x)$.

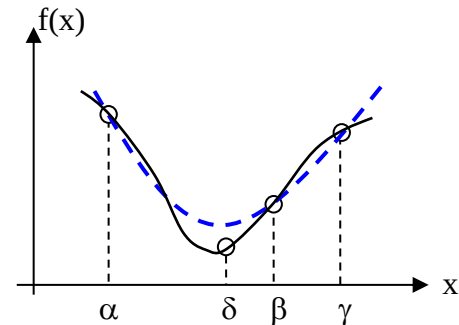
- We can then calculate the initial value of δ as 1.7143.
- The actual value of the function $f(x)$ at this point is $f(\delta) = -15.5335$.
- The following graph shows the results of this first iteration:



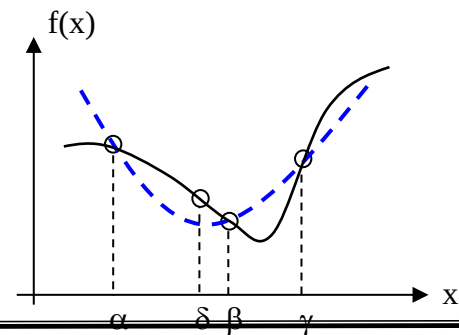
- Now what? Do we need to continue? We compare the two lowest values, namely $f(\delta) = -15.5335$ and $f(\beta) = -9$ and since the difference is not within an acceptable limit of say 0.001, we need to continue with the next iteration.
- We now have four points, α , β , γ and δ but we only need three.
- Considering the points α , β and γ , we typically discard the one with the highest function value.
- However if retaining this value puts a definite bound on the location of the **minimum** (this is not necessarily δ), then we keep this value and discard the one that gives the next highest function value.
- This concept is illustrated below:



In this case, we discard point α .
By taking the points β , δ and γ , we place a definite bound on the minimum value.



In this case, we discard point γ .
By taking the points α , δ and β , we place a definite bound on the minimum value.



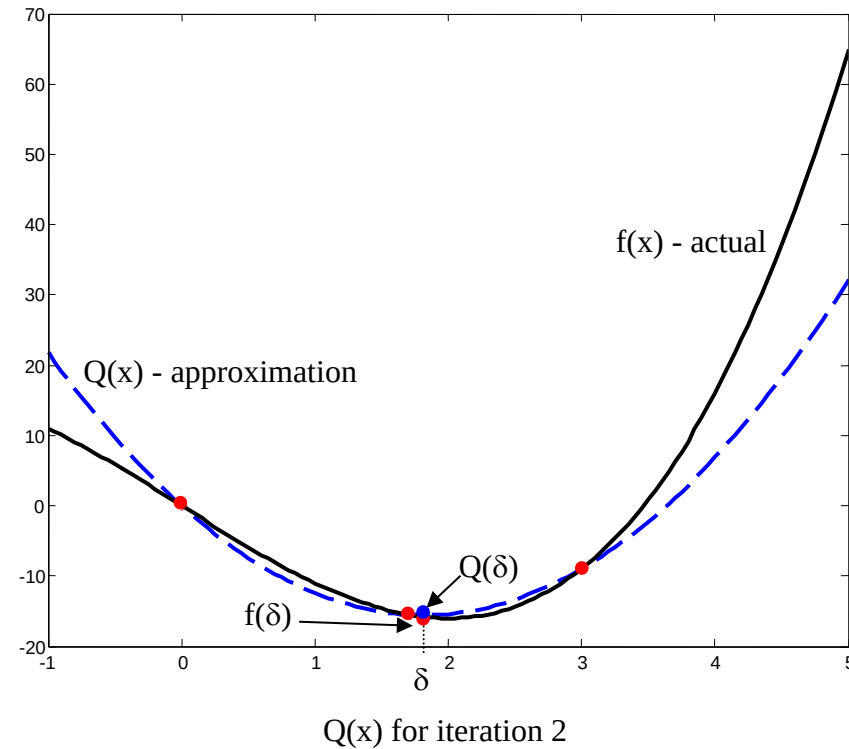
In this case, we discard point α .
By taking the points δ , β and γ , we place a definite bound on the minimum value.

- In our example, therefore, we discard the point γ as the points α and β place a definite bound on the minimum value.
- Hence the points α , β and δ become our new points α , β and γ for the next iteration and the process is repeated.
- Iteration 2: Recall that we need to ensure that $f(\beta) < f(\alpha)$ and $f(\beta) < f(\gamma)$. Therefore, we now take the values as follows:

$$\alpha = 0, \beta = 1.7143, \gamma = 3$$

$$\text{and } f(\alpha) = 0, f(\beta) = -15.5335, f(\gamma) = -9$$

- We can now use these values to determine the new values of A, B and C and hence obtain an improved quadratic function approximation $Q(x)$.
- We calculate the next value of δ as 1.8182. The actual value of the function $f(x)$ at this point is $f(\delta) = -15.8077$.
- The following graph shows the results of this second iteration:

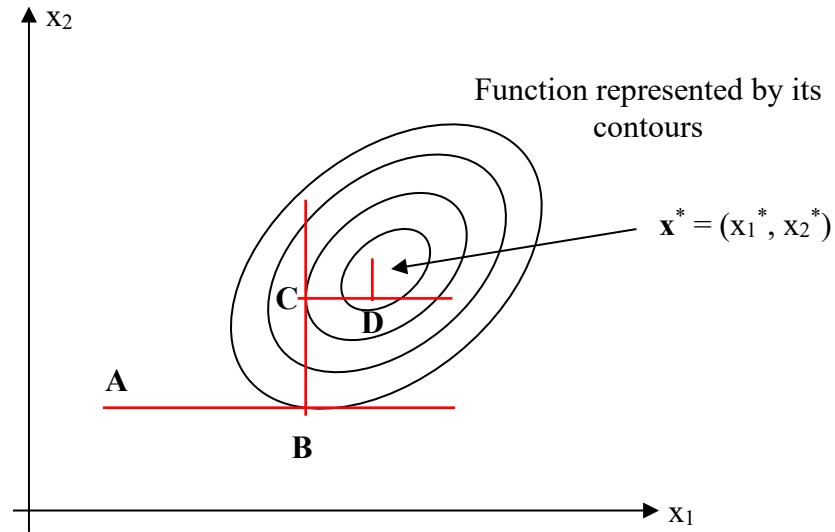


- Here, it's clear to see that $Q(x)$ is now a better approximation to $f(x)$ around the local minimum. Furthermore, the approximate minimum value δ is also closer to the actual local minimum.
- Comparing the new $f(\delta) = -15.8077$ with the old $f(\delta) = -15.5335$, we can see that the difference is still greater than 0.001 and therefore the process needs to be repeated.

- Continuing in this manner, we will eventually obtain our solution.
- In Matlab, it takes 6 iterations in order to obtain $\delta = 1.9997$ and $f(\delta) = -16.000$. This is sufficiently close to the actual local minimum of 2, which gives a value of exactly -16.
- The number of function evaluations required is 4 for the first iteration followed by one per each of the next 5 iterations, giving a total of 9.
- The quadratic search is a compromise between computational efficiency and mathematical certainty.
- The number of function evaluations is clearly less than each of the previous line search methods. However, unlike the other methods, the quadratic search may not converge to a minimum of F .
- It assumes that the environment is ‘friendly’ – i.e. the function F rises sharply in region of local minimum.
- Note that the quadratic search method is based on the principle that when we are near a minimum point, any function behaves like a quadratic!
- In most cases this is justified. If it is not, then we must use a search such as grid search, Fibonacci or Golden section which will always converge.

3. Direct search methods for functions of more than one variable

- Two main types of algorithms (often referred to as hill-climbing algorithms) exist for solving unconstrained functions of more than one variable.
- These are Direct Search and Gradient methods. We are going to consider the former approach in this section. Gradient techniques will be studied in section 4.
- A direct search method is one that uses function values only, i.e. no derivatives are used.
- The basic philosophy behind the direct search method can be summarised as follows.
- Consider a function of two variables, whose contour lines are shown below:



- The minimum of this function is located at (x_1^*, x_2^*) .
- The crudest and most obvious approach for obtaining the minimum is to iteratively minimise with respect to alternative variables.

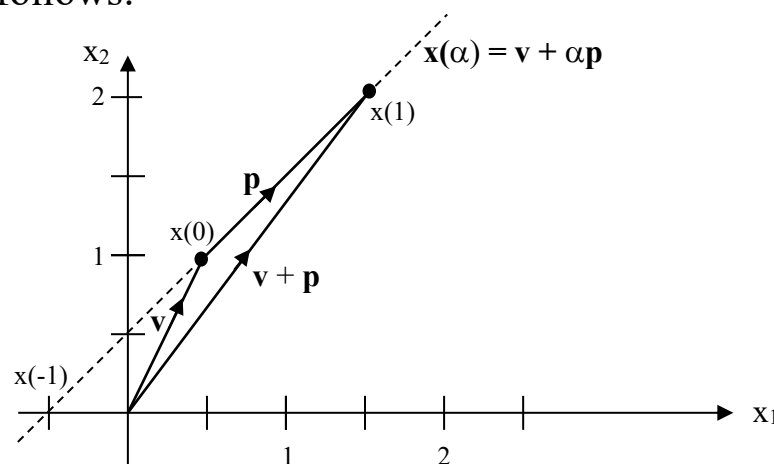
- For example, referring to the diagram, we start at some point A and search in the direction of the x_1 -axis for the minimum in this direction, giving us point B.
- From B we then search in the direction of the x_2 -axis and obtain the minimum along this direction, i.e. point C.
- From C we search along the direction of the x_1 -axis once more and so on to the actual minimum point.
- Clearly, any of the line searches, in the previous section of the notes, could be used for each search.
- Also, this idea extends to functions of n variables.
- Before exploring different direct search algorithms in more detail, we need to examine and understand the basic mathematical notation that is used to represent line searches in n -dimensional space.

Moving about in n -dimensional space

- Consider a given n -vector $\mathbf{v}$. This can be regarded as defining a point in n -space.
- Let $\mathbf{p}$ be another given n -vector. Now consider the n -vector $\mathbf{x}$ defined as follows:

$\mathbf{x}(\alpha) = \mathbf{v} + \alpha \mathbf{p}$, where α is a scalar.

- In other words, $\mathbf{x}(\alpha)$ represents points on a straight line through $\mathbf{v}$ in the direction of $\mathbf{p}$ (positive or negative).
- Consider the example: $n = 2$, $\mathbf{v} = \begin{bmatrix} \frac{1}{2} \\ 2 \end{bmatrix}$, $\mathbf{p} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Hence: $\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ 2 \end{bmatrix} + \alpha \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.
- Graphically, we can represent this as follows:



- The dashed line represents the line defined by the vector $\mathbf{x}$. Hence, any point on a straight line passing through a point $\mathbf{v}$ in the direction $\mathbf{p}$ can be specified by a suitable choice of the variable α in the expression $\mathbf{x}(\alpha) = \mathbf{v} + \alpha \mathbf{p}$.
- We can easily determine the familiar form of the equation of a straight line as follows. The above equations give:

$$x_1 = \frac{1}{2} + \alpha \text{ and } x_2 = 1 + \alpha$$

- Substituting for α gives: $x_2 = x_1 + \frac{1}{2} \quad (y = mx + c)$
- Now we know how to represent a straight line in n-dimensional space. We can easily search along this line to obtain a minimum if we so wish.

Illustrative Example using line search and n-dimensional quadratic functions

- Quadratic functions are the simplest form of n-dimensional functions which can have a local minimum. They take the general form:

$$F(\mathbf{x}) = a + \mathbf{b}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{G} \mathbf{x}$$

where a is a given scalar, $\mathbf{b}$ is a given n -vector and $\mathbf{G}$ is a given $n \times n$ *symmetric* matrix.

- Differentiation gives:

$$\nabla F = \mathbf{b} + \mathbf{G}\mathbf{x} \quad \text{and} \quad \nabla^2 F = \mathbf{G}$$

- To ensure that the quadratic function has a unique minimum, we must choose $\mathbf{G}$ to be positive definite.
- The actual position of the local minimum is given by $\nabla F = \mathbf{0}$, i.e.:

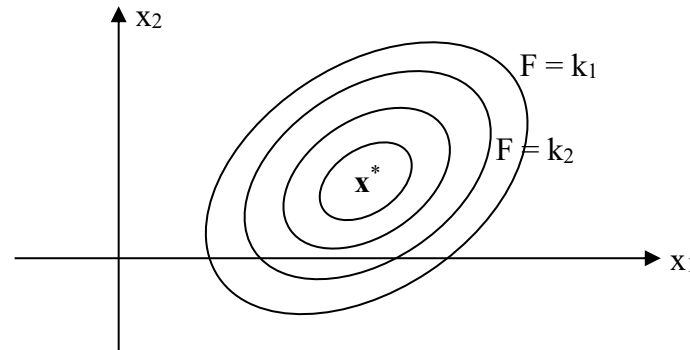
$$\mathbf{G}\mathbf{x}^* = -\mathbf{b}, \quad \text{which can be solved for } \mathbf{x}^*$$

- Note that although we can write $\mathbf{x}^* = -\mathbf{G}^{-1}\mathbf{b}$, in practise, we tend to avoid computing $\mathbf{x}^*$ by inverting $\mathbf{G}$, as this is computationally intensive!

- For a 2-dimensional function (i.e. $n = 2$), we can represent $F(\mathbf{x})$ graphically by means of a **contour plot**. For example, for a positive definite quadratic function, choose any value of F , say k_1 , and plot all values of the vector $\mathbf{x}$ for which:

$$\mathbf{b}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{G} \mathbf{x} = k_1$$

- The contours consist of a nest of ellipses centred on the local minimum $\mathbf{x}^*$:



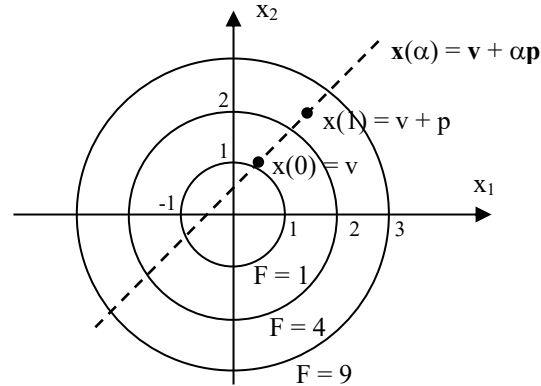
- Hence the position of $\mathbf{x}^*$ is determined by the values of $\mathbf{b}$ and $\mathbf{G}$. The orientation of the axes of the ellipses is determined by $\mathbf{G}$.
- Now, let's consider the 2-dimensional function: $F(\mathbf{x}) = x_1^2 + x_2^2$.
- Hence: $F(\mathbf{x}) = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \mathbf{x}^T \mathbf{I} \mathbf{x}$
- In other words, $F(\mathbf{x})$ is a quadratic function where $\mathbf{G} = 2\mathbf{I}$.
- Contours are obtained by setting F to a series of constant values:

$$F(\mathbf{x}) = 1 = x_1^2 + x_2^2 \quad \text{- unit circle}$$

$$F(\mathbf{x}) = 2 = x_1^2 + x_2^2 \quad \text{- circle, radius } \sqrt{2}$$

etc...

- This gives the contour plot:



- Now consider the search line given in the previous example, i.e.:

$$\text{Starting point } \mathbf{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \text{ and search direction } \mathbf{p} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- Hence, we have the search line, as shown by the dashed line above:

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix} + \alpha \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- To obtain the equation of the section of $F(\mathbf{x})$ cut by the straight line, we substitute $\mathbf{x}(\alpha)$ into $F(\mathbf{x})$:

$$\begin{aligned} F(\alpha) &= \mathbf{x}^T \mathbf{x} = (\mathbf{v} + \alpha \mathbf{p})^T (\mathbf{v} + \alpha \mathbf{p}) \\ &= \mathbf{v}^T \mathbf{v} + 2\alpha \mathbf{p}^T \mathbf{v} + \alpha^2 \mathbf{p}^T \mathbf{p} \end{aligned}$$

- Evaluating gives:

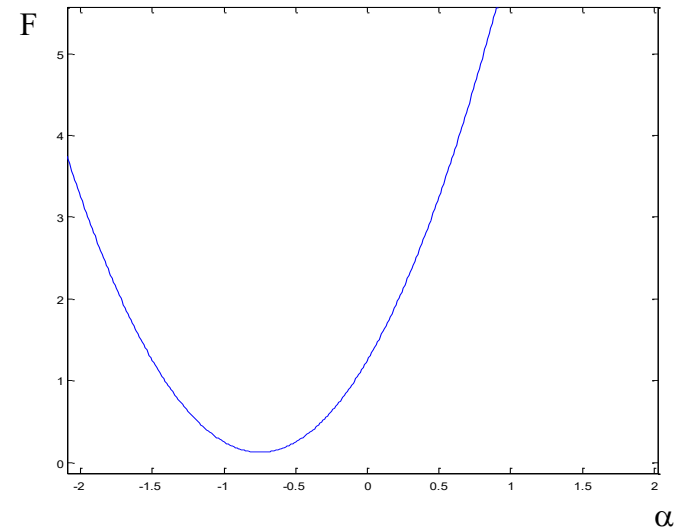
$$\mathbf{v}^T \mathbf{v} = \begin{bmatrix} 0.5 & 1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} = 0.25 + 1 = 1.25$$

$$\text{Similarly, } \mathbf{p}^T \mathbf{v} = 1.5, \mathbf{p}^T \mathbf{p} = 2$$

- Hence:

$$F(\alpha) = 1.25 + 3\alpha + 2\alpha^2$$

- Therefore the cross-section of $F(x)$ along the link $x(\alpha)$ looks like:



- To find the minimum along this line, we could use one of our line searches if necessary, but in this case it's simple to calculate the answer:

$$\frac{dF}{d\alpha} = 3 + 4\alpha = 0 \Rightarrow \alpha = -0.75$$

$$\frac{d^2F}{d\alpha^2} = 4 > 0 \Rightarrow \text{a minimum point.}$$

- Hence:

$$\mathbf{x}_{\min} = \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix} - \frac{3}{4} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \Rightarrow x_1 = -0.25 \text{ and } x_2 = 0.25$$
$$\Rightarrow F(\mathbf{x}_{\min}) = 0.125$$

- So now we can use this $\mathbf{x}_{\min}$ as our estimate of the local minimum (which is in fact equal to 0) or we can use it as a new starting point for the next search direction.
- Clearly, we can see how this direct search approach works.
- Such algorithms can be described in the following way:

-
- (i) Given an estimate $\mathbf{x}^{(k)}$ of the position of a local minimum of $F(\mathbf{x})$ at $\mathbf{x}^*$, generate a search direction $\mathbf{p}^{(k)}$ by some means.
- (ii) Find a minimum of the 1-dimensional function $F(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)})$. Let this minimum be at $\alpha^{(k)}$.
- (iii) Set $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}$. Set $k = k+1$ and repeat.
- Here the value k represents the k^{th} iteration.
 - The question now becomes one of choosing the appropriate lines to search. In other words, for a given point, what is the best direction in which to search for a minimum?

3.1 The Univariate Search Method

- The univariate search adjusts the value of each function variable in turn. This corresponds to **using the co-ordinate axes as search directions**.

- In this case, the search direction vector is given by $\mathbf{e}_i$ where the i^{th} value contains a 1 to represent the axis of interest.
- Note that the vector $\mathbf{e}_i$ is a unit vector in the direction of the i^{th} axis. The size of the vector corresponds to the number of variables in the given function (and therefore the number of different axes!).
- For example, $\mathbf{e}_1 = [1 \ 0 \ 0 \ \dots]^T$ represents a unit vector in the direction of the x_1 -axis.
- Hence, in the 2-D case, if we want to search in the direction of increasing (or decreasing) x_1 , i.e. in a direction parallel to the x_1 -axis, then the search direction $\mathbf{p}$ is given by:

$$\mathbf{p} = \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

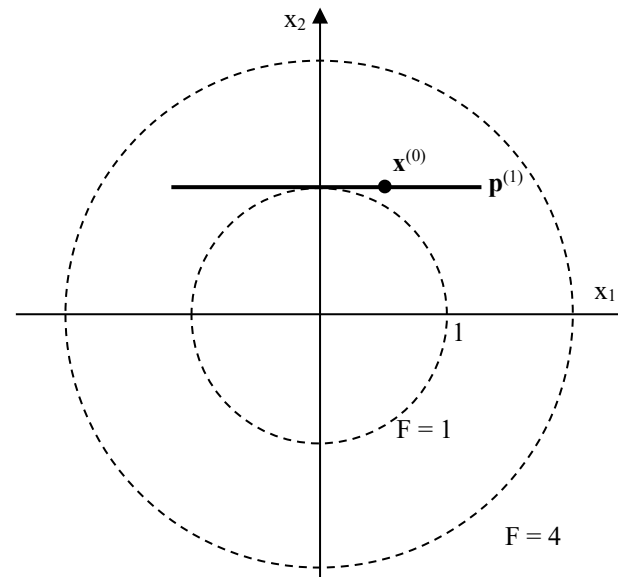
- In general, $\mathbf{p}$ will be an n -dimensional vector and if we are searching in the direction of increasing (of decreasing) x_i then the i^{th} element will be 1, as follows:

$$\mathbf{p} = \mathbf{e}_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Illustrative Example

- Once again, consider the example: $F(\mathbf{x}) = x_1^2 + x_2^2 = \mathbf{x}^T \mathbf{x}$
- And consider the starting point: $\mathbf{x}^{(0)} = \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix}$.
- Note that the superscript (0) represents the starting point (i.e. prior to the first iteration which will be denoted by superscript (1)).

- The graphical representation is as follows, where $\mathbf{x}^{(0)}$ is the starting point and $\mathbf{p}^{(1)}$ is the search direction of interest through this point:



- The value of the function at this point is:

$$F(\mathbf{x}^{(0)}) = \begin{bmatrix} \frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix} = 1.25$$

- Taking the first search direction in the direction parallel to the x_1 -axis, we have:

$$\mathbf{p}^{(1)} = \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

- We want to obtain $\mathbf{x}^{(1)}$, where $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(1)}$. Hence, we need to find the value of α that minimises $F(\alpha)$: *(note that $\mathbf{x}^{(1)}$ is a function of the parameter α , i.e. $\mathbf{x}(\alpha)$)*

$$\begin{aligned} F(\alpha) &= \mathbf{x}(\alpha)^T \mathbf{x}(\alpha) \\ &= (\mathbf{x}^{(0)} + \alpha \mathbf{p}^{(1)})^T (\mathbf{x}^{(0)} + \alpha \mathbf{p}^{(1)}) \\ &= (\mathbf{x}^{(0)})^T \mathbf{x}^{(0)} + 2\alpha (\mathbf{p}^{(1)})^T \mathbf{x}^{(0)} + \alpha^2 (\mathbf{p}^{(1)})^T \mathbf{p}^{(1)} \end{aligned}$$

- Evaluating each of these terms gives:

$$(\mathbf{x}^{(0)})^T \mathbf{x}^{(0)} = 1.25, \quad (\mathbf{p}^{(1)})^T \mathbf{x}^{(0)} = 0.5, \quad (\mathbf{p}^{(1)})^T \mathbf{p}^{(1)} = 1$$

- Therefore:

$$F(\alpha) = 1.25 + \alpha + \alpha^2$$

- And:

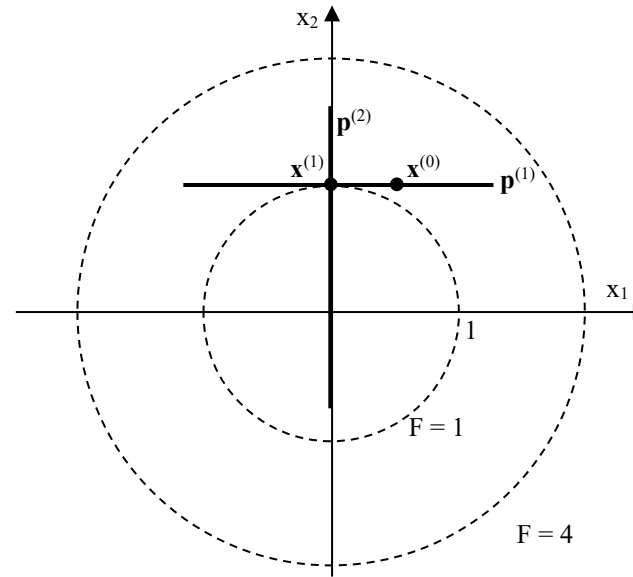
$$\frac{dF}{d\alpha} = 1 + 2\alpha = 0 \Rightarrow \alpha = -0.5$$

$$\frac{d^2F}{d\alpha^2} = 2 > 0 \Rightarrow \text{Minimum at } \alpha = -0.5$$

- Now: $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(1)} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} + \left(-\frac{1}{2}\right) \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$

- Also: $F(\mathbf{x}^{(1)}) = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1$

- These results are illustrated below:



- Now take the next search direction, parallel to the x_2 -axis, i.e.: $\mathbf{p}^{(2)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.
- From the diagram, we can see that this iteration should bring us straight to the actual minimum ... let's see!
- We now want to obtain $\mathbf{x}^{(2)}$, where $\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \alpha \mathbf{p}^{(2)}$. Again, we need to find the value of α that minimises $F(\alpha)$.

- Evaluating $F(\alpha)$ with the new updates:

$$(\mathbf{x}^{(1)})^T \mathbf{x}^{(1)} = 1, \quad (\mathbf{p}^{(2)})^T \mathbf{x}^{(1)} = 1, \quad (\mathbf{p}^{(2)})^T \mathbf{p}^{(2)} = 1$$

$$\therefore F(\alpha) = 1 + 2\alpha + \alpha^2$$

$$\frac{dF}{d\alpha} = 2 + 2\alpha = 0 \Rightarrow \alpha = -1$$

$$\frac{d^2F}{d\alpha^2} = 2 > 0 \Rightarrow \text{Minimum at } \alpha = -1$$

- Now: $\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \alpha \mathbf{p}^{(2)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} + (-1) \begin{bmatrix} 0 \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

- $F(\mathbf{x}^{(2)}) = \begin{bmatrix} 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0$

- This must be a minimum since $F(\mathbf{x}) \geq 0$

- Check:

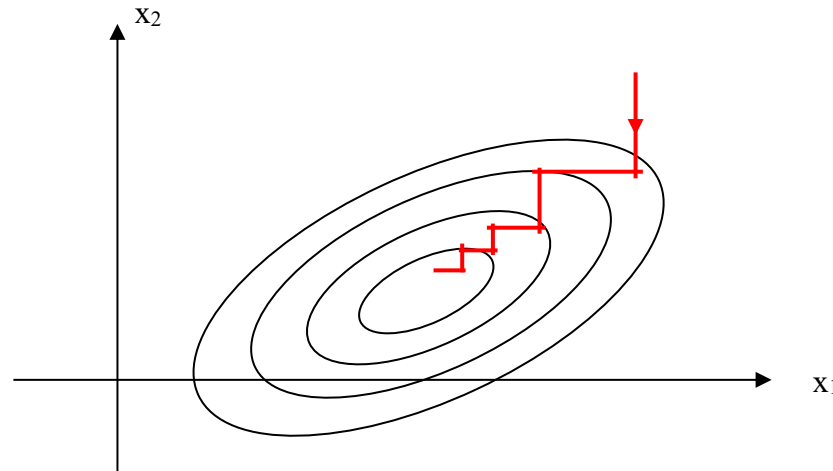
$$\frac{\partial F}{\partial x_1} = 2x_1 = 0 \text{ at } \mathbf{x}^{(2)} \qquad \frac{\partial^2 F}{\partial x_1^2} = 2$$

$$\frac{\partial F}{\partial x_2} = 2x_2 = 0 \text{ at } \mathbf{x}^{(2)} \qquad \frac{\partial^2 F}{\partial x_2^2} = 2$$

$$\frac{\partial^2 F}{\partial x_1 \partial x_2} = 0 = \frac{\partial^2 F}{\partial x_2 \partial x_1}$$

- This gives the matrix $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$, which is positive definite.
- Using the facts that the slope of $F(\mathbf{x})$ along any direction $\mathbf{p}$ is $\mathbf{p}^T \nabla F$ and that ∇F is orthogonal to the contour at every point, we can predict the sequence of points obtained.
- Although we only required 2 iterations in the last example, in general, the univariate search method requires many iterations and, therefore, many evaluations of F .

- For example, consider the following diagram, where the contours do not line up so conveniently with the x_1 and x_2 axes (as in the last example). In this case, it will take several iterations before we converge to the minimum point.



- The problem with univariate search is that it doesn't learn – it always uses the same set of search iterations. The Davies Swann and Campey Algorithm attempts to alleviate this problem.

3.2 The Davies Swann and Campey Algorithm (DSC)

- This begins by carrying out n iterations of univariate search. The search direction on iteration $(n + 1)$ is taken as the direction from $\mathbf{x}^{(0)}$ to $\mathbf{x}^{(n)}$, i.e.:

$$\mathbf{x}^{(n)} - \mathbf{x}^{(0)}$$

- Let **one cycle of the DSC algorithm** be defined as a set of n successive iterations of univariate search.
- Let $\mathbf{p}_j^{(k)}$ be the j^{th} direction of search on the k^{th} cycle.
- Let $\mathbf{x}_j^{(k)}$ be the j^{th} point generated on the k^{th} cycle.
- Then, **on cycle 1** (i.e. the first n iterations of the univariate search), we have n search directions:

$$\begin{array}{l} \mathbf{p}_1^{(1)} = \mathbf{e}_1 \\ \vdots \\ \mathbf{p}_i^{(1)} = \mathbf{e}_i \end{array}$$

$$\begin{matrix} \vdots & \vdots \\ \mathbf{p}_n^{(1)} = \mathbf{e}_n \end{matrix}$$

where $\mathbf{e}_1 = [1 \ 0 \ 0 \ \dots \ 0]^T$, $\mathbf{e}_i = [0 \ \dots \ 1 \ \dots \ 0]^T$, etc...

- The corresponding set of points generated is: $\mathbf{x}_1^{(1)}, \dots, \mathbf{x}_i^{(1)}, \dots, \mathbf{x}_n^{(1)}$
- Starting from $\mathbf{x}_0^{(1)}$ (arbitrary starting value), we use the formula:

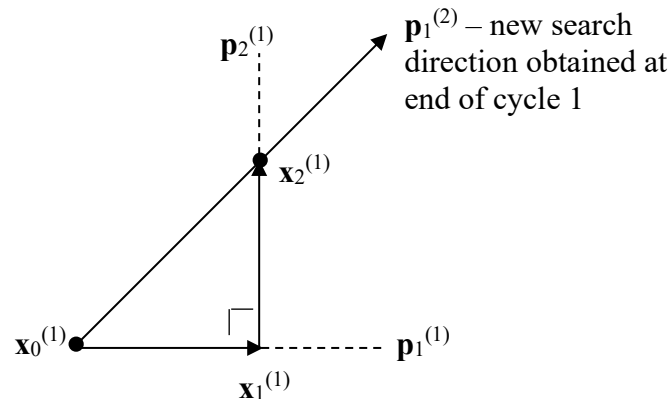
$$\mathbf{x}_i^{(1)} = \mathbf{x}_{i-1}^{(1)} + \alpha_i^{(1)} \mathbf{p}_i^{(1)}$$

- Recall that $\alpha_i^{(1)}$ is chosen to minimise F along the search direction $\mathbf{p}_i^{(1)}$
- On the next cycle (i.e. the second cycle) we start by choosing:

$$\mathbf{p}_1^{(2)} = \frac{\mathbf{x}_n^{(1)} - \mathbf{x}_0^{(1)}}{\|\mathbf{x}_n^{(1)} - \mathbf{x}_0^{(1)}\|}$$

- Note that the denominator simply scales the vector to ensure a vector of unit length. The direction of the search line remains unchanged.

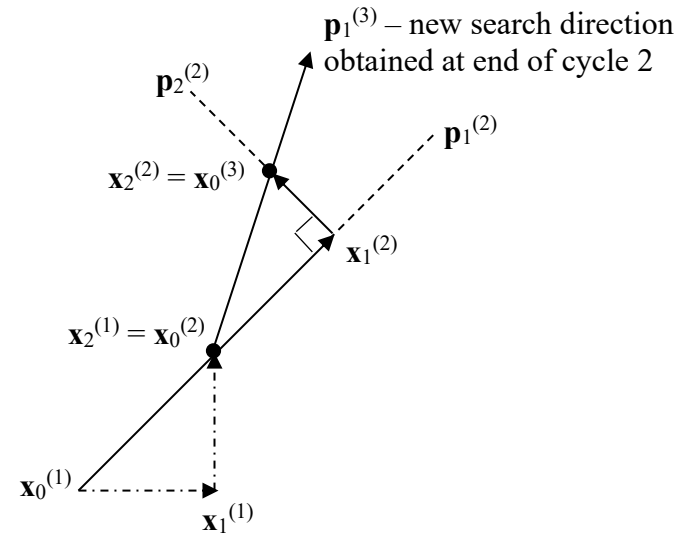
- This leaves us with $n-1$ directions to find! We then need to generate $n-1$ directions orthogonal to $\mathbf{p}_1^{(2)}$ and to each other.
- The method normally used to do this is the **Gram-Schmidt Orthogonalisation method**.
- Before we look at this method, let's illustrate how the DSC algorithm works graphically - we will consider the case of $n = 2$.
- Hence, we have two search directions to consider for each cycle. The first cycle is just the same as carrying out the standard univariate search method:



- At the end of the first cycle the new search direction $\mathbf{p}_1^{(2)}$ is the line joining $\mathbf{x}_0^{(1)}$ and $\mathbf{x}_2^{(1)}$ as shown above.
- This is the fundamental difference between the DSC algorithm and the basic univariate search method.
- We now obtain the minimum along the new search direction and, from this point, generate a new search direction orthogonal (i.e. perpendicular) to the current one.
- Note that the end point of cycle 1 is the start point of cycle 2, i.e.:

$$\mathbf{x}_2^{(1)} = \mathbf{x}_0^{(2)}$$

- The first and second cycles of the DSC algorithm are shown below:



- At the end of the second cycle, we now have the new search direction $\mathbf{p}_1^{(3)}$. And thus, the process continues until we obtain a suitable minimum value – for example, if the difference between two successive estimates of the minimum is sufficiently small.
- Note that the search direction $\mathbf{p}_2^{(2)}$ is orthogonal to $\mathbf{p}_1^{(2)}$. How do we obtain this search direction?

- As mentioned earlier, the method normally used to do this is the **Gram-Schmidt Orthogonalisation method**.

Gram-Schmidt Orthogonalisation Method

- We begin by defining a matrix $\mathbf{P}^{(k)}$, where k represents the current DSC cycle, as:

$$[\mathbf{p}_1^{(k)} \quad \mathbf{p}_2^{(k-1)} \quad \dots \quad \mathbf{p}_n^{(k-1)}]$$

- We know the current search direction $\mathbf{p}_1^{(k)}$. The other search directions are from the previous cycle, but this is fine. All we need are vectors that belong to the plane associated with each of the different function variables.
- It is important that the first column is the current search direction, because, as we will soon see, this is also the first column vector of the $\mathbf{Q}$ matrix, which will store our desired orthogonal search directions.
- We are looking for matrices $\mathbf{Q}$ and $\mathbf{U}$ such that:

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}, \quad \text{the unit identity matrix (nxn) and}$$

$QU = \mathbf{P}^{(k)}$, where U is an upper triangular matrix (i.e. the bottom half off-diagonal terms are all zero)

- Let Q be written as $[\mathbf{q}_1 \ \mathbf{q}_2 \ \dots \ \mathbf{q}_n]$, where $\mathbf{q}_i$ is the i^{th} column vector. Since $Q^T Q = \mathbf{I}$, all the column vectors of Q are orthogonal to each other. This is what we are looking for!
- The matrix U is given by:

$$\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix}$$

- Then $QU = \mathbf{P}^{(k)}$ gives:

$$[\mathbf{q}_1 \ \mathbf{q}_2 \ \dots \ \mathbf{q}_n] \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix} = [\mathbf{p}_1^{(k)} \ \mathbf{p}_2^{(k-1)} \ \dots \ \mathbf{p}_n^{(k-1)}]$$

- Multiply out the LHS and equate columns of matrices on LHS side to those on the RHS.
- For the first column we obtain:

$$u_{11} \mathbf{q}_1 = \mathbf{p}_1^{(k)} \quad (\text{equation X for reference purposes})$$

- Pre-multiplying both sides by its own transpose gives:

$$(u_{11} \mathbf{q}_1)^T (u_{11} \mathbf{q}_1) = (\mathbf{p}_1^{(k)})^T (\mathbf{p}_1^{(k)})$$

~~~~~

### *Aside Notes*

- **Note 1 :** The vector  $\mathbf{p}_1^{(k)}$  is normalised to have a unit length. Hence  $(\mathbf{p}_1^{(k)})^T (\mathbf{p}_1^{(k)}) = 1$ .
- For example, take the vector  $\mathbf{k} = [k_1 \ k_2]^T$ . The normalised vector is given by:

$$\begin{bmatrix} \frac{k_1}{\sqrt{k_1^2 + k_2^2}} & \frac{k_2}{\sqrt{k_1^2 + k_2^2}} \end{bmatrix}^T$$

- And therefore:

$$\mathbf{k}^T \mathbf{k} = \begin{bmatrix} \frac{k_1}{\sqrt{k_1^2 + k_2^2}} & \frac{k_2}{\sqrt{k_1^2 + k_2^2}} \end{bmatrix} \begin{bmatrix} \frac{k_1}{\sqrt{k_1^2 + k_2^2}} \\ \frac{k_2}{\sqrt{k_1^2 + k_2^2}} \end{bmatrix}$$

$$= \frac{k_1^2}{\left(\sqrt{k_1^2 + k_2^2}\right)^2} + \frac{k_2^2}{\left(\sqrt{k_1^2 + k_2^2}\right)^2} = 1$$

- **Note 2:** We know that  $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$ , i.e.:

$$[\mathbf{q}_1 \quad \mathbf{q}_2 \quad \cdots \quad \mathbf{q}_n] \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \vdots \\ \mathbf{q}_n \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

- Multiplying out gives  $\mathbf{q}_1^T \mathbf{q}_1 = 1$ , etc...
- Also note that off diagonal terms are zero, i.e.  $\mathbf{q}_1^T \mathbf{q}_2 = 0$ , etc...

- ~~~~~
- Evaluating gives:

$$u_{11}^2 \mathbf{q}_1^T \mathbf{q}_1 = (\mathbf{p}_1^{(k)})^T (\mathbf{p}_1^{(k)})$$

$$\Rightarrow u_{11}^2 (1) = 1$$

- Therefore, since  $u_{11} = 1$ , equation X now gives:  $\mathbf{q}_1 = \mathbf{p}_1^{(k)}$
- Hence the first column of the matrix  $\mathbf{Q}$  is the same as the first one of matrix  $\mathbf{P}$ .

- Now consider the second column of the expansion of  $\mathbf{QU} = \mathbf{P}^{(k)}$ :

$$u_{12} \mathbf{q}_1 + u_{22} \mathbf{q}_2 = \mathbf{p}_2^{(k-1)} \quad (\text{equation Y for reference purposes})$$

- Pre-multiplying both sides by  $\mathbf{q}_1^T$  gives:

$$u_{12} \mathbf{q}_1^T \mathbf{q}_1 + u_{22} \mathbf{q}_1^T \mathbf{q}_2 = \mathbf{q}_1^T \mathbf{p}_2^{(k-1)}$$

$$\Rightarrow u_{12} (1) + u_{22} (0) = \mathbf{q}_1^T \mathbf{p}_2^{(k-1)}$$

$$\Rightarrow u_{12} = \mathbf{q}_1^T \mathbf{p}_2^{(k-1)}$$

- Since we know both  $\mathbf{q}_1^T$  and  $\mathbf{p}_2^{(k-1)}$  we can calculate  $u_{12}$ .

- Rearranging equation Y gives:

$$u_{22} \mathbf{q}_2 = \mathbf{p}_2^{(k-1)} - u_{12} \mathbf{q}_1$$

- Squaring both sides gives:

$$u_{22}^2 \mathbf{q}_2^T \mathbf{q}_2 = \| \mathbf{p}_2^{(k-1)} - u_{12} \mathbf{q}_1 \|^2 \quad ( \dots \text{note that for a vector } \mathbf{a}, \mathbf{a}^T \mathbf{a} = \|\mathbf{a}\|^2 )$$

$$\Rightarrow u_{22}^2 (1) = \| \mathbf{p}_2^{(k-1)} - u_{12} \mathbf{q}_1 \|^2$$

$$\Rightarrow u_{22} = \| \mathbf{p}_2^{(k-1)} - u_{12} \mathbf{q}_1 \|$$

- All the values on the RHS side are known and therefore  $u_{22}$  can be calculated.
- Hence, we can determine  $\mathbf{q}_2$  as follows:

$$\mathbf{q}_2 = \frac{p_2^{(k-1)} - u_{12} q_1}{\| p_2^{(k-1)} - u_{12} q_1 \|}$$

- Continuing in this way for the other columns, we obtain in general for the  $i^{\text{th}}$  column:

$$u_{ji} = \mathbf{q}_j^T \mathbf{p}_i^{(k-1)}, \quad j = 1, 2, \dots, i-1$$

$$\text{and } \mathbf{q}_i = \frac{p_i^{(k-1)} - \sum_{j=1}^{i-1} u_{ji} q_j}{\left\| p_i^{(k-1)} - \sum_{j=1}^{i-1} u_{ji} q_j \right\|}$$

**General Equation for  
Gram-Schmidt  
Orthogonalisation Method**

- The search directions are then  $\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_n$ .
- The reason for using orthogonal search directions is so that we don't end up searching along the same plane all the time!
- Note – in practice, the DSC algorithm is modified so that a new set of directions are only used on iteration  $k$  if a reasonably large step has been taken on the previous iteration,  $k-1$ .

### **Illustrative Example of DSC algorithm and Gram-Schmidt Orthogonalisation**

- Consider the following function (with  $n = 2$ ):

$$f(\mathbf{x}) = x_1 x_2 + x_1^2 + 2x_2^2$$

- Taking the initial starting point  $\mathbf{x}_0 = [1 \ 1]^T$ , carry out two complete cycles of the Davies, Swann and Campey algorithm.

- The first cycle is the same as the univariate method!
- Calculating the function at  $\mathbf{x}_0$  gives  $f(\mathbf{x}_0) = 4$ .
- Taking the first search direction in the direction parallel to the  $x_1$ -axis, we have:

$$\mathbf{p}_1^{(1)} = \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

- Hence:  $\mathbf{x}_1^{(1)} = \mathbf{x}_0^{(0)} + \alpha \mathbf{p}_1^{(1)}$
- This gives:  $\mathbf{x}_1^{(1)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \alpha + 1 \\ 1 \end{bmatrix}$
- Need to find  $\alpha$ , by minimising  $f(\alpha)$ :

$$f(\alpha) = (\alpha + 1).1 + (\alpha + 1)^2 + 2.(1)^2 = \alpha^2 + 3\alpha + 4$$

$$\frac{df}{d\alpha} = 2\alpha + 3 = 0 \Rightarrow \alpha = -1.5 \text{ and } f(\alpha) = 1.75$$

$$\frac{d^2f}{d\alpha^2} = 2 > 0 \Rightarrow \text{minimum}$$

- Hence:  $\mathbf{x}_1^{(1)} = \begin{bmatrix} -0.5 \\ 1 \end{bmatrix}$

- Our next search direction is:

$$\mathbf{p}_2^{(1)} = \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- Therefore:  $\mathbf{x}_2^{(1)} = \begin{bmatrix} -0.5 \\ 1 \end{bmatrix} + \alpha \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.5 \\ \alpha + 1 \end{bmatrix}$

- Once again, we find  $\alpha$ , by minimising  $f(\alpha)$ .
- This gives  $\alpha = -0.875$  (minimum) and  $f(\alpha) = 0.21875$ . Verify these values for yourself!

- This completes the first cycle. At the start of the second cycle we know that:

$$\mathbf{x}_0^{(2)} = \mathbf{x}_2^{(1)} = \begin{bmatrix} -0.5 \\ 0.125 \end{bmatrix}$$

- The new search direction, according to DSC, is given by:

$$p_1^{(2)} = \frac{x_2^{(1)} - x_0^{(1)}}{\|x_2^{(1)} - x_0^{(1)}\|}$$

- ***Note: the scaling factor is only important for the Gram-Schmidt orthogonalisation part. It makes absolutely no difference to the direction itself. Therefore, for now, it's sufficient to take:***

$$p_1^{(2)} = x_2^{(1)} - x_0^{(1)}$$

- Calculating this gives:

$$\mathbf{p}_1^{(2)} = \begin{bmatrix} -1.5 \\ -0.875 \end{bmatrix}$$

- Now we determine the next point as:

$$\mathbf{x}_1^{(2)} = \mathbf{x}_0^{(2)} + \alpha \mathbf{p}_1^{(2)}$$

- This gives:

$$\mathbf{x}_1^{(2)} = \begin{bmatrix} -0.5 \\ 0.125 \end{bmatrix} + \alpha \begin{bmatrix} -1.5 \\ -0.875 \end{bmatrix} = \begin{bmatrix} -0.5-1.5\alpha \\ 0.125-0.875\alpha \end{bmatrix}$$

- Once again, find  $\alpha$  by minimising  $f(\alpha)$ :

$$f(\alpha) = 5.09375\alpha^2 + 1.3125\alpha + 0.21875$$

$$\frac{df}{d\alpha} = 10.1875\alpha + 1.3125 = 0 \quad \Rightarrow \alpha = -0.1288 \quad \text{and} \quad f(\alpha) = 0.1342$$

$$\frac{d^2f}{d\alpha^2} = 10.1875 > 0 \quad \Rightarrow \text{minimum}$$

- Hence:  $\mathbf{x}_1^{(2)} = \begin{bmatrix} -0.3068 \\ 0.2377 \end{bmatrix}$
- At this stage, we need to generate a search direction orthogonal to  $\mathbf{p}_2^{(2)}$ .
- The Gram-Schmidt orthogonalisation method gives:

$$u_{12} = \mathbf{q}_1^T \mathbf{p}_2^{(k-1)}$$

$$\text{and } \mathbf{q}_2 = \frac{\mathbf{p}_2^{(k-1)} - u_{12}\mathbf{q}_1}{\|\mathbf{p}_2^{(k-1)} - u_{12}\mathbf{q}_1\|}$$

- Here,  $\mathbf{q}_1$  = normalised version of  $\mathbf{p}_1^{(2)}$ , i.e.:

$$\mathbf{q}_1 = \frac{\mathbf{p}_1^{(2)}}{\|\mathbf{p}_1^{(2)}\|} = \frac{\begin{bmatrix} -1.5 \\ -0.875 \end{bmatrix}}{\sqrt{(-1.5)^2 + (-0.875)^2}} = \begin{bmatrix} -0.8638 \\ -0.5039 \end{bmatrix}$$

- Recall that k represents the cycle number, therefore  $\mathbf{p}_2^{(k-1)} = \mathbf{p}_2^{(1)}$  which is:

$$\mathbf{p}_2^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- Hence, we calculate  $u_{12}$  as follows:

$$u_{12} = [-0.8638 \quad -0.5039] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = -0.5039$$

- And  $\mathbf{q}_2$ , the orthogonal vector, is determined as follows:

$$\mathbf{q}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0.5039 \begin{bmatrix} -0.8638 \\ -0.5039 \end{bmatrix} = \begin{bmatrix} -0.4353 \\ 0.7461 \end{bmatrix}$$

- Once again, for convenience, we have not normalised this vector as, for now, we are only using it as a search direction.
- Our new search direction  $\mathbf{p}_2^{(2)}$  is given by the vector  $\mathbf{q}_2$ .
- Now we determine the next point as:

$$\mathbf{x}_2^{(2)} = \mathbf{x}_1^{(2)} + \alpha \mathbf{p}_2^{(2)}$$

- This gives:

$$\mathbf{x}_2^{(2)} = \begin{bmatrix} -0.3068 \\ 0.2377 \end{bmatrix} + \alpha \begin{bmatrix} -0.4353 \\ 0.7461 \end{bmatrix} = \begin{bmatrix} -0.3068 - 0.4353\alpha \\ 0.2377 + 0.7461\alpha \end{bmatrix}$$

- Once again, find  $\alpha$  by minimising  $f(\alpha)$ :

$$f(\alpha) = 0.978\alpha^2 + 0.6441\alpha + 0.1342$$

$$\frac{df}{d\alpha} = 1.956\alpha + 0.6441 = 0 \quad \Rightarrow \alpha = -0.3293 \quad \text{and} \quad f(\alpha) = 0.02815$$

$$\frac{d^2f}{d\alpha^2} = 1.956 > 0 \quad \Rightarrow \text{minimum}$$

- Hence:  $\mathbf{x}_0^{(3)} = \mathbf{x}_2^{(2)} = \begin{bmatrix} -0.1635 \\ -0.008 \end{bmatrix}$

- This is the end of the second cycle.
- The actual minimum is  $\mathbf{x} = [0 \ 0]^T$ , so we are clearly getting close!
- In general, how do we know when to terminate an algorithm, especially if we don't know the minimum?
- There are two criteria that can be used. For our example:

(i) if  $\| \mathbf{x}_2^{(2)} - \mathbf{x}_0^{(2)} \| < \varepsilon_1$       or

(ii) if  $| f(\mathbf{x}_2^{(2)}) - f(\mathbf{x}_0^{(2)}) | < \varepsilon_2$

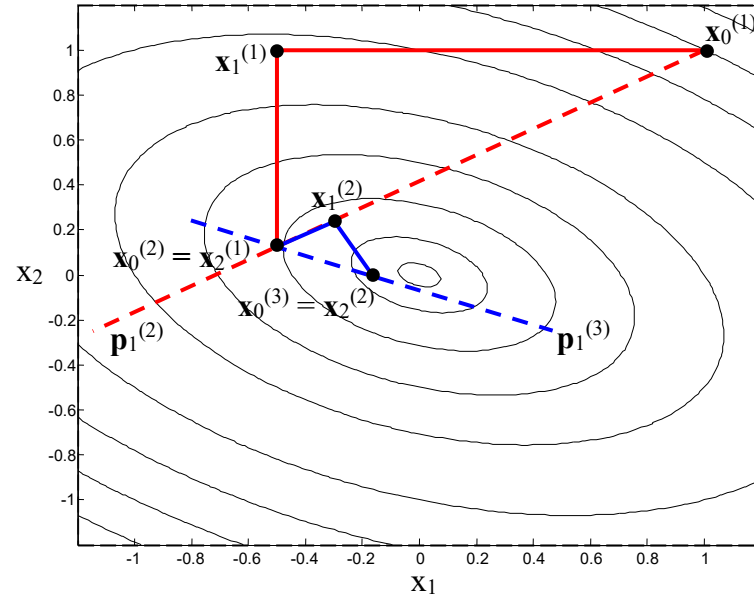
- Here  $\varepsilon_1$  and  $\varepsilon_2$  are pre-defined small positive values.
- In our case we obtain:

$$(i) \ \| \mathbf{x}_2^{(2)} - \mathbf{x}_0^{(2)} \| = \left\| \begin{bmatrix} 0.3365 \\ -0.133 \end{bmatrix} \right\| = 0.3618$$

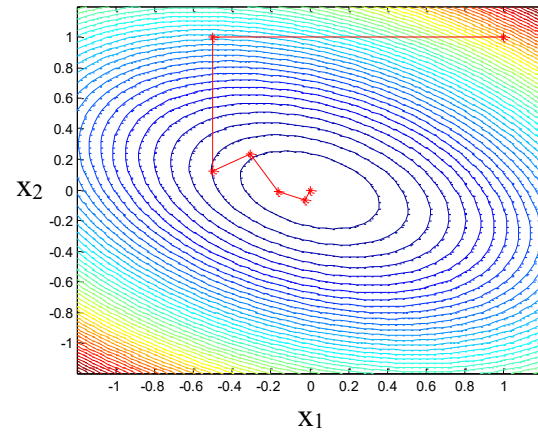
or

$$(ii) |f(\mathbf{x}_2^{(2)}) - f(\mathbf{x}_0^{(2)})| = |0.02815 - 0.21875| = 0.1906$$

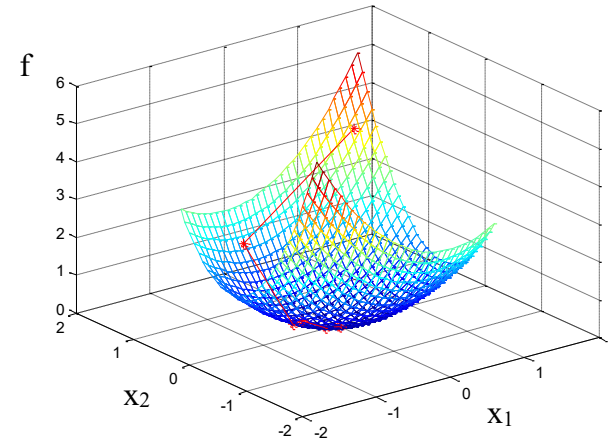
- Therefore, if  $0.3618 < \varepsilon_1$  then we can stop searching.
- Alternatively, if  $0.1906 < \varepsilon_2$ , then we can stop the algorithm.
- Either one or both conditions can be use, with the appropriate tolerance value(s) specified at the start of the algorithm.
- In this example, we were only asked to carry out two iterations, so we don't need to worry about termination!
- The following contour map shows a graphically representation of what has happened thus far:



- Continuing for a few more iterations, we will eventually converge on the minimum as shown below:



*Contour Plot*

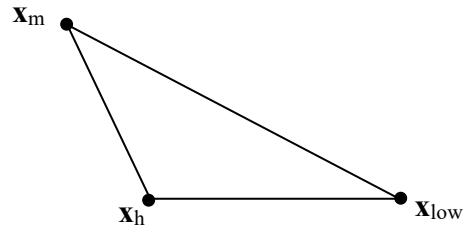


*Mesh Plot of Function*

### 3.3 The Simplex Method (Nelder and Mead)

- The Simplex method is another direct search method but takes a very different approach to the Univariate and the DSC search methods when it comes to determining the sequence of one-dimensional search directions.
- The original Simplex method was derived by Spendley, Hext and Himsworth but was later improved upon by Nelder and Mead.
- First off, a **simplex** in  $n$ -dimensional space is a polygon with  $n+1$  vertices.
- In 2-D space, the simplex is a triangle. In other words, the three points must not lie on a straight line. Similarly, in 3-D space, the 4 points must not lie on the same plane, etc.
- The Simplex method evaluates the function at each of the vertices in the simplex.
- A new point is then generated, and the function is evaluated at this point. The point with the largest function value is rejected and the new point is subsequently introduced into the simplex.

- In this fashion, a sequence of new simplices is created, with the best point in the simplex converging to a local minimum.
- The generation of the new point is obtained by the application of three basic operations – namely *reflection*, *expansion* and *contraction*.
- The Simplex algorithm is now outlined using a 2-D example. In other words we will use a triangle (with  $n = 2$ ) to illustrate the operation of the Simplex method.
- Consider the 3 points,  $\mathbf{x}_{\text{low}}$ ,  $\mathbf{x}_m$ ,  $\mathbf{x}_h$  of a simplex in 2-D space as shown below. The choice of this initial simple is entirely arbitrary.



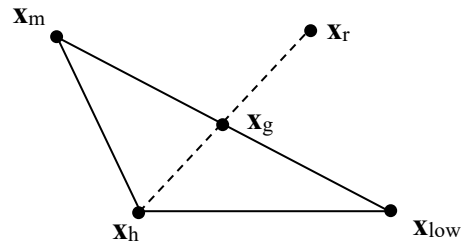
- **Step 1** - We calculate the function at each of these points and rank them from the lowest to the highest. Let's say that:

$$F(\mathbf{x}_{\text{low}}) \leq F(\mathbf{x}_m) \leq F(\mathbf{x}_h)$$

- Find the centroid, say  $\mathbf{x}_g$ , of all the points, except  $\mathbf{x}_h$  ( i.e. the one giving the highest function value):

$$\mathbf{x}_g = \frac{1}{n} \sum_{i \neq h} \mathbf{x}_i$$

- **Step 2 - Reflection** - A new point is generated by reflecting  $\mathbf{x}_h$  about the centroid, as illustrated below:

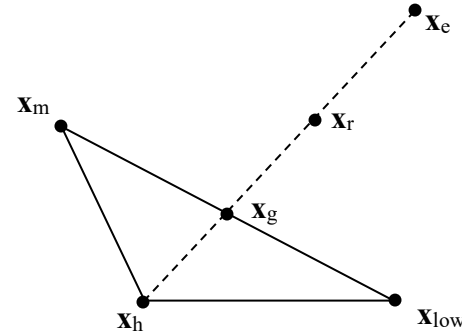


- The reflected point  $\mathbf{x}_r$  satisfies the condition:

$$\mathbf{x}_h - \mathbf{x}_g = \mathbf{x}_g - \mathbf{x}_r$$

$$\text{or } \mathbf{x}_r = 2\mathbf{x}_g - \mathbf{x}_h$$

- We want to find a point that is better than  $\mathbf{x}_m$ .
- If  $F(\mathbf{x}_r) \geq F(\mathbf{x}_m)$  then it's possible that we have moved too far out of the simplex and a better point may be found by making a contraction – go to step 3.
- If, instead,  $F(\mathbf{x}_m) > F(\mathbf{x}_r) > F(\mathbf{x}_l)$  then we have found an improvement on  $\mathbf{x}_m$ . We therefore replace the worst point  $\mathbf{x}_h$  with  $\mathbf{x}_r$  and return to step 1.
- If, finally,  $F(\mathbf{x}_r) < F(\mathbf{x}_{\text{low}})$  then we have found the best point to date. At this stage we make the reasonable assumption that a larger step beyond  $\mathbf{x}_r$  may lead to an even better point:



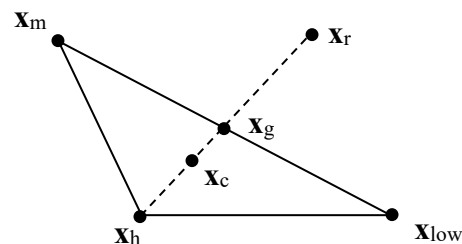
- This *expanded* point  $\mathbf{x}_e$  is obtained so that:

$$\mathbf{x}_e - \mathbf{x}_g = \mu (\mathbf{x}_r - \mathbf{x}_g), \quad \text{with } \mu > 1$$

$$\text{or } \mathbf{x}_e = \mathbf{x}_g (1 - \mu) + \mu \mathbf{x}_r$$

- Now, if  $F(\mathbf{x}_e) < F(\mathbf{x}_r)$  then replace the worst point  $\mathbf{x}_h$  with  $\mathbf{x}_e$  and return to step 1.
- Otherwise  $\mathbf{x}_r$  is the better point and therefore replace  $\mathbf{x}_h$  with  $\mathbf{x}_r$  and return to step 1.

- **Step 3 - Contraction** – There are two possible scenarios:
- **Scenario 1:** If it turns out that  $F(\mathbf{x}_r) \geq F(\mathbf{x}_h)$  then we look for a point  $\mathbf{x}_c$ , located on the line joining  $\mathbf{x}_h$  and  $\mathbf{x}_g$ , that is better than the point  $\mathbf{x}_r$ :



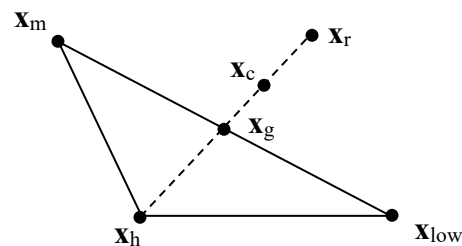
- We make a contraction of the original simplex with the condition:

$$\mathbf{x}_c - \mathbf{x}_g = \eta(\mathbf{x}_h - \mathbf{x}_g), \quad \text{with } 0 < \eta < 1$$

$$\text{or } \mathbf{x}_c = \mathbf{x}_g(1 - \eta) + \eta\mathbf{x}_h$$

- Now, if  $F(\mathbf{x}_c) < F(\mathbf{x}_h)$  we replace  $\mathbf{x}_h$  with  $\mathbf{x}_c$  and go to step 1. Otherwise we go to step 4.

- **Scenario 2:** If  $F(\mathbf{x}_r) < F(\mathbf{x}_h)$ , we look for a point  $\mathbf{x}_c$ , located on the line joining  $\mathbf{x}_g$  and  $\mathbf{x}_r$ :



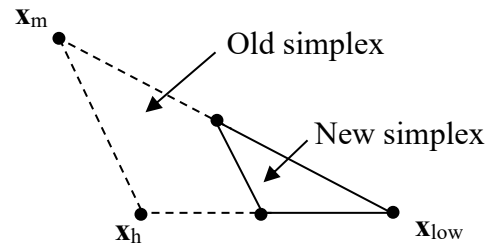
- The point  $\mathbf{x}_c$  now satisfies the condition:

$$\mathbf{x}_c - \mathbf{x}_g = \eta(\mathbf{x}_r - \mathbf{x}_g), \quad \text{with } 0 < \eta < 1$$

$$\text{or } \mathbf{x}_c = \mathbf{x}_g(1 - \eta) + \eta\mathbf{x}_r$$

- Now, if  $F(\mathbf{x}_c) < F(\mathbf{x}_r)$  we replace  $\mathbf{x}_h$  with  $\mathbf{x}_c$  and go to step 1, otherwise go to step 4. (Note, finding  $F(\mathbf{x}_c) < F(\mathbf{x}_h)$  is pointless considering that we know that  $F(\mathbf{x}_r) < F(\mathbf{x}_h)$  at this stage!)

- **Step 4 - Reduction of Simplex** – To reach this step, reflection, expansion and contraction have all failed to return a point better than  $\mathbf{x}_h$ .
- Therefore, it appears that the minimum is enclosed by the current simplex.
- We now contract the simplex around the best point by reducing each side by half.
- In other words, each point in the simplex,  $\mathbf{x}_i$  is replaced by:  $\frac{\mathbf{x}_i + \mathbf{x}_{low}}{2}$
- Graphically:



- We then proceed to step 1 and repeat the process.
- Note the **expansion factor** is typically taken as  $\mu = 2$ , while the **contraction factor** is typically  $\eta = \frac{1}{2}$ .

## Convergence

- For each new simplex we should test for convergence before continuing. One possible criterion is that the simplex satisfies:

$$\sum_{i=1}^{n+1} |F(x_i) - F(x_{\text{low}})| < \varepsilon$$

where  $\varepsilon$  is a positive small value.

- This criterion establishes a simplex having a deviation from the best obtained value less than  $\varepsilon$ .
- If this criterion is met, then  $\mathbf{x}_{\text{low}}$  is taken to be the estimate of the minimum and the simplex algorithm is terminated.

## Illustrative Example of Simplex algorithm

- Once again, let us consider the following function (with  $n = 2$ ):

$$f(\mathbf{x}) = x_1x_2 + x_1^2 + 2x_2^2$$

- Taking the initial starting points  $\mathbf{x}_0 = [1 \ 1]^T$ ,  $\mathbf{x}_1 = [-0.5 \ 1]^T$  and  $\mathbf{x}_2 = [-0.5 \ 0]^T$ , carry out two iterations of the Simplex algorithm. Sketch the simplex after each iteration.

- **First iteration** - The first step is to determine the order of the points in terms of their function output:

$$f(\mathbf{x}_0) = 4, \quad f(\mathbf{x}_1) = 1.75, \quad f(\mathbf{x}_2) = 0.25$$

- Hence  $\mathbf{x}_0$  is the worst point and  $\mathbf{x}_2$  is the best point. For convenience, we'll denote:

$$\mathbf{x}_h = \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{x}_m = \mathbf{x}_1 = \begin{bmatrix} -0.5 \\ 1 \end{bmatrix} \quad \text{and} \quad \mathbf{x}_{\text{low}} = \mathbf{x}_2 = \begin{bmatrix} -0.5 \\ 0 \end{bmatrix}$$

where  $\mathbf{x}_h$ ,  $\mathbf{x}_m$  and  $\mathbf{x}_{\text{low}}$  represent the high (or worse), middle and low (or best) points respectively.

- We now determine the centroid  $\mathbf{x}_g$  using the best two (or  $n$ ) points:

$$\begin{aligned}\mathbf{x}_g &= 0.5 (\mathbf{x}_{\text{low}} + \mathbf{x}_m) \\ &= 0.5 \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix}\end{aligned}$$

- We now carry out reflection of  $\mathbf{x}_h$  (the worse point) about  $\mathbf{x}_g$  to obtain a new point  $\mathbf{x}_r$ :

$$\begin{aligned}\mathbf{x}_r &= 2\mathbf{x}_g - \mathbf{x}_h \\ &= 2 \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \end{bmatrix}\end{aligned}$$

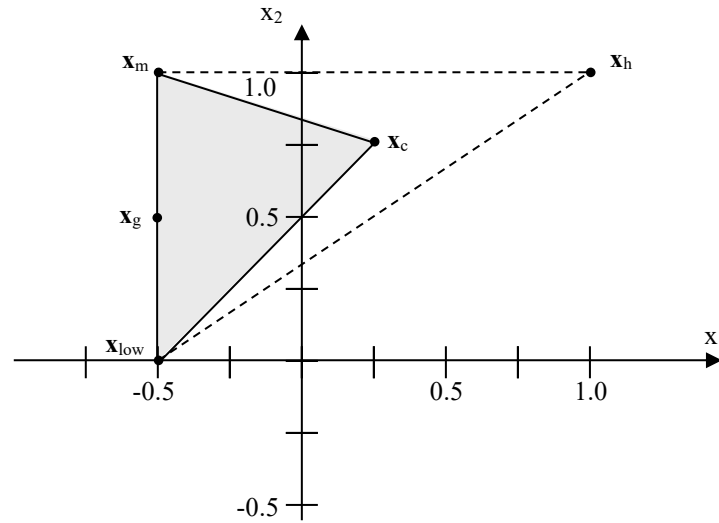
- The function output at this point is  $f(\mathbf{x}_r) = 4$ .
- Since  $f(\mathbf{x}_r) \geq f(\mathbf{x}_h)$ , we need to consider contraction of  $\mathbf{x}_h$  instead. Hence, our new point  $\mathbf{x}_c$  is given by:

$$\mathbf{x}_c = \mathbf{x}_g(1 - \eta) + \eta\mathbf{x}_h$$

- Taking  $\eta = 0.5$  gives:

$$\mathbf{x}_c = \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix} (0.5) + 0.5 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.75 \end{bmatrix}$$

- The function output at this point is  $f(\mathbf{x}_c) = 1.375$ .
- Since this is better than  $f(\mathbf{x}_h)$ , we now replace  $\mathbf{x}_h$  with  $\mathbf{x}_c$  and the first iteration is finished.
- Our new simplex looks like:



- **Second iteration** – We need to re-order the new points:

$$f(\mathbf{x}_{\text{low}}) = 0.25, \quad f(\mathbf{x}_m) = 1.75, \quad f(\mathbf{x}_c) = 1.375$$

- Hence  $\mathbf{x}_m$  is now the worst point, while  $\mathbf{x}_l$  is still the best point. Hence:

$$\mathbf{x}_h = \mathbf{x}_m = \begin{bmatrix} -0.5 \\ 1 \end{bmatrix}, \quad \mathbf{x}_m = \mathbf{x}_c = \begin{bmatrix} 0.25 \\ 0.75 \end{bmatrix} \quad \text{and} \quad \mathbf{x}_{\text{low}} = \begin{bmatrix} -0.5 \\ 0 \end{bmatrix}$$

- The new centroid  $\mathbf{x}_g$  is:

$$\mathbf{x}_g = 0.5 (\mathbf{x}_{\text{low}} + \mathbf{x}_m) = 0.5 \begin{bmatrix} -0.25 \\ 0.75 \end{bmatrix} = \begin{bmatrix} -0.125 \\ 0.375 \end{bmatrix}$$

- We now carry out reflection of  $\mathbf{x}_h$  about  $\mathbf{x}_g$  to obtain a new point  $\mathbf{x}_r$ :

$$\mathbf{x}_r = 2\mathbf{x}_g - \mathbf{x}_h = 2 \begin{bmatrix} -0.125 \\ 0.375 \end{bmatrix} - \begin{bmatrix} -0.5 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.25 \\ -0.25 \end{bmatrix}$$

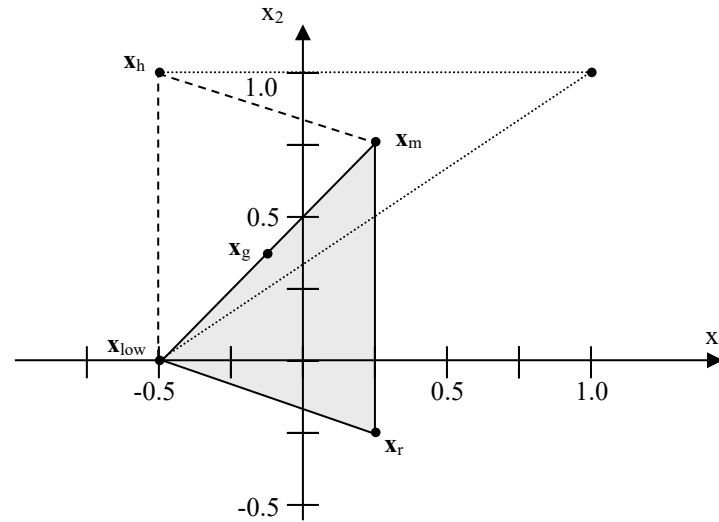
- The function output at this point is  $f(\mathbf{x}_r) = 0.125$ .
- Since  $f(\mathbf{x}_r) < f(\mathbf{x}_{\text{low}})$ , we need to consider expansion by looking at a point beyond  $\mathbf{x}_r$ . This new point  $\mathbf{x}_e$  is determined as follows:

$$\mathbf{x}_e = \mathbf{x}_g (1 - \mu) + \mu \mathbf{x}_r$$

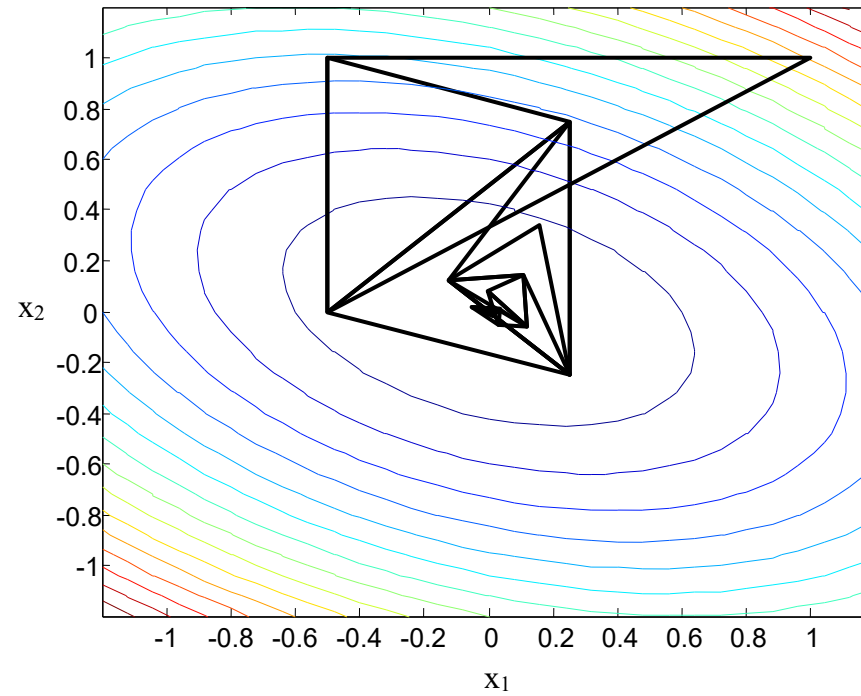
- Taking  $\mu = 2$  gives:

$$\mathbf{x}_e = \begin{bmatrix} -0.125 \\ 0.375 \end{bmatrix} (-1) + 2 \begin{bmatrix} 0.25 \\ -0.25 \end{bmatrix} = \begin{bmatrix} 0.625 \\ -0.875 \end{bmatrix}$$

- The function output at this point is  $f(\mathbf{x}_e) = 1.375$ .
- This is clearly worse than  $f(\mathbf{x}_r)$ , i.e. expansion does not improve the situation in this case! Hence, we replace  $\mathbf{x}_h$  with  $\mathbf{x}_r$  and the second iteration is complete.
- Our new simplex now looks like:



- We were only asked for two iterations. Had we continued on, our simplex would eventually converge on the actual local minimum, which is given by  $\mathbf{x}^* = [0 \ 0]^T$ .
- The following Matlab graph shows 12 iterations of the Simplex method. We can clearly see that it is indeed converging towards the local minimum.



- All the direct methods (Univariate, DSC and Simplex) follow a similar pattern – the minimum is found by a sequence of line searches.
- The main disadvantage of these methods is that they can require numerous function evaluations in order to converge to the minimum point. As noted previously, the fewer function evaluations required the better the algorithm!
- The key advantage of direct methods is that the determination of the search directions is independent of the local properties of the actual function. In particular they do not rely on the derivative of the function. As such, these methods are ideally suited for optimising noisy functions.

## 4. Gradient Methods

- Gradient algorithms not only make use of values of the function  $F$  in determining search directions, but also make use of values of the gradient vector  $\nabla F$ .
- Of course, this is only useful in cases where the first derivatives are easily calculated.
- In this section of the notes, we are going to examine a few different gradient methods.

### 4.1 Steepest Descent

- The most obvious direction in which to search for a minimum is along the direction of steepest descent.
- The gradient of the function defines the steepest ascent, hence the steepest descent is given by:  
$$-\nabla F(\mathbf{x})$$

- This property can be proved as follows. We will consider the case of a 2-variable function, but the principle is easily extended to functions of  $n$  variables.
- Consider a two-variable function  $F(\mathbf{x})$  which has the following first derivatives at the point  $\mathbf{x}$ :

$$\nabla F(\mathbf{x}) = \begin{bmatrix} \partial F / \partial x_1 \\ \partial F / \partial x_2 \end{bmatrix}$$

- Now consider a two-dimensional vector  $\mathbf{d} = [d_1 \ d_2]^T$  and examine the effect of moving a small distance  $\alpha$  in the direction of  $\mathbf{d}$  from the point  $\mathbf{x}$  to the new point  $\mathbf{x} + \alpha\mathbf{d}$ .
- From Taylor's theorem we can state:

$$F(\mathbf{x} + \alpha\mathbf{d}) \approx F(\mathbf{x}) + \alpha\mathbf{d}^T \frac{dF}{dx} \Big|_x$$

$$\Rightarrow F(\mathbf{x} + \alpha \mathbf{d}) \approx F(\mathbf{x}) + \alpha \begin{bmatrix} d_1 & d_2 \end{bmatrix} \begin{bmatrix} \partial F / \partial x_1 \\ \partial F / \partial x_2 \end{bmatrix}$$

$$\Rightarrow F(\mathbf{x} + \alpha \mathbf{d}) \approx F(\mathbf{x}) + \alpha \left( d_1 \frac{\partial F}{\partial x_1} + d_2 \frac{\partial F}{\partial x_2} \right)$$

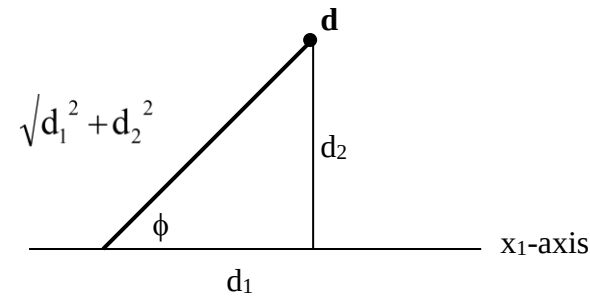
- Clearly, the smaller the value of  $\alpha$  the more accurate the approximation becomes.
- The distance moved,  $\delta d$ , is given by:

$$\delta d = \alpha \sqrt{d_1^2 + d_2^2}$$

- Letting  $\delta F$  represent the change in  $F$ , i.e.  $F(\mathbf{x} + \alpha \mathbf{d}) - F(\mathbf{x})$ , we can state that:

$$\frac{\delta F}{\delta d} \approx \frac{d_1 \frac{\partial F}{\partial x_1} + d_2 \frac{\partial F}{\partial x_2}}{\sqrt{d_1^2 + d_2^2}}$$

- Let the angle between the vector  $\mathbf{d}$  and the  $x_1$ -axis be  $\phi$ , as illustrated below:



- Then:  $\cos \phi = \frac{d_1}{\sqrt{d_1^2 + d_2^2}}$  and  $\sin \phi = \frac{d_2}{\sqrt{d_1^2 + d_2^2}}$

- Hence:

$$\frac{\delta F}{\delta d} = \cos(\phi) \frac{\partial F}{\partial x_1} + \sin(\phi) \frac{\partial F}{\partial x_2}$$

- Okay, so we now have an expression that relates how function  $F$  changes in relation to a small distance travelled in the direction of vector  $\mathbf{d}$ .
- Finding  $\phi$  to maximise this value, i.e. obtaining the direction that results in the largest positive change in  $F$ , gives us the **steepest ascent** direction.
- Conversely, finding  $\phi$  to minimise this value, i.e. obtaining the direction that results in the largest negative change in  $F$ , gives us the **steepest descent** direction.

- Either way, we need to solve the equation:  $\frac{d}{d\phi} \left( \frac{\delta F}{\delta \mathbf{d}} \right) = 0$

- This gives:

$$-\sin(\phi) \frac{\partial F}{\partial x_1} + \cos(\phi) \frac{\partial F}{\partial x_2} = 0$$

- Rearranging gives:

$$\cos(\phi) \frac{\partial F}{\partial x_2} = \sin(\phi) \frac{\partial F}{\partial x_1} \Rightarrow \tan(\phi) = \frac{\partial F}{\partial x_2} / \frac{\partial F}{\partial x_1} = d_2 / d_1$$

- We have one equation to determine two components. The infinite solutions to this equation can be expressed as:

$$\mathbf{d} = \gamma \nabla F$$

- Here  $\gamma$  is a constant value. In other words:

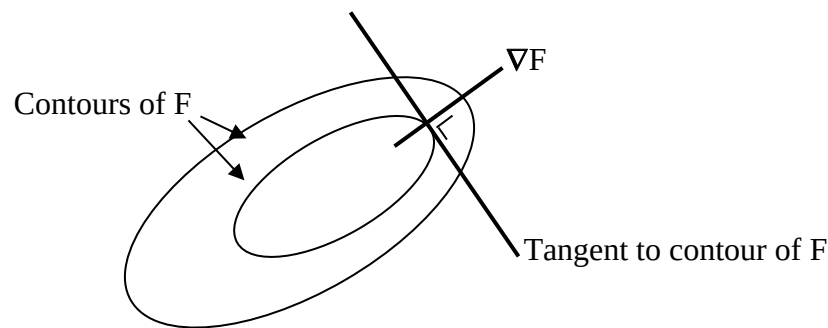
$$\begin{bmatrix} d_1 \\ d_2 \end{bmatrix} = \gamma \begin{bmatrix} \partial F / \partial x_1 \\ \partial F / \partial x_2 \end{bmatrix}$$

- Calculating the second derivative gives:

$$\frac{d^2}{d\phi^2} \left( \frac{\delta F}{\delta d} \right) = -\cos(\phi) \frac{\partial F}{\partial x_1} - \sin(\phi) \frac{\partial F}{\partial x_2} = - \left( \frac{\delta F}{\delta d} \right)$$

- If  $\gamma$  is taken to be positive, then  $\mathbf{d}$  has the same sign as  $\nabla F$  and therefore the second derivative above is negative. Hence positive  $\gamma$  leads to a maximum  $\delta F / \delta d$ .

- If  $\gamma$  is taken to be negative, then  $\mathbf{d}$  has the opposite sign as  $\nabla F$  and therefore the second derivative above is positive. Hence negative  $\gamma$  leads to a minimum  $\delta F / \delta \mathbf{d}$ .
- *In summary, the maximum change in  $F$  occurs when we move in the direction of  $\nabla F$ . Therefore  $\nabla F$  is the direction of the steepest ascent and  $-\nabla F$  is the direction of the steepest descent.*
- So we now know an important property of the gradient vector  $\nabla F$ . Another important property is that  $\nabla F$  is orthogonal to the tangent plane of  $F$ .
- This concept is illustrated below for a two-dimensional example:



- The tangent to the contour of  $F$  can be regarded as the straight line along which a small step produces no change in  $F$ .
- For the two-dimensional vector  $\mathbf{t}$ , along the tangent, we can show (see earlier) that:

$$\frac{\delta F}{\delta t} \approx \frac{t_1 \frac{\partial F}{\partial x_1} + t_2 \frac{\partial F}{\partial x_2}}{\sqrt{t_1^2 + t_2^2}}$$

- Here,  $\mathbf{t} = [t_1 \ t_2]^T$  and  $\delta t$  represents a small step change in the direction of  $\mathbf{t}$ .
- As stated already, we know that a small change along  $\mathbf{t}$ , i.e.  $\delta t$ , shouldn't change  $F$ . So:

$$\frac{\delta F}{\delta t} = 0 \quad \Rightarrow \quad \frac{t_1 \frac{\partial F}{\partial x_1} + t_2 \frac{\partial F}{\partial x_2}}{\sqrt{t_1^2 + t_2^2}} = 0$$

- This gives: 
$$t_1 \frac{\partial F}{\partial x_1} + t_2 \frac{\partial F}{\partial x_2} = 0 \quad \Rightarrow \quad \frac{t_1}{t_2} = - \frac{\partial F / \partial x_2}{\partial F / \partial x_1}$$

- Once again we have one equation to determine the two components  $t_1$  and  $t_2$ . Therefore the condition is satisfied if we let:

$$\mathbf{t} = \begin{bmatrix} -\partial F / \partial x_2 \\ \partial F / \partial x_1 \end{bmatrix}$$

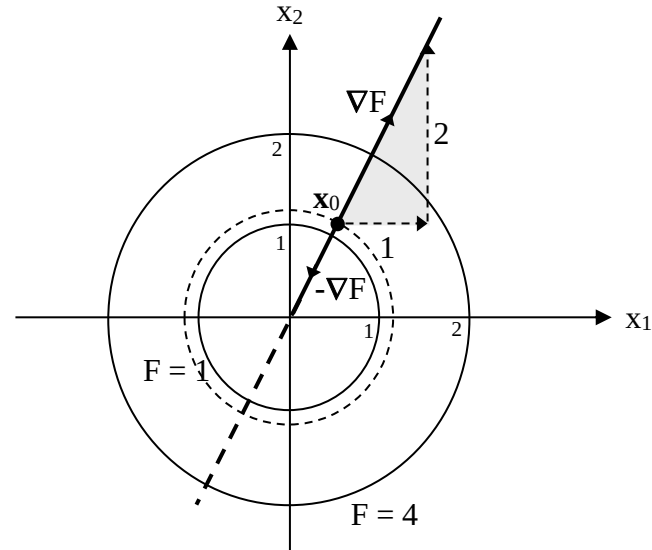
- To confirm that this vector is indeed orthogonal to  $\nabla F$ , we simply show that  $\nabla F^T \mathbf{t} = 0$ , i.e. that the inner product of the two vectors is 0:

$$\nabla F^T \mathbf{t} = \begin{bmatrix} \frac{\partial F}{\partial x_1} & \frac{\partial F}{\partial x_2} \end{bmatrix} \begin{bmatrix} -\frac{\partial F}{\partial x_2} \\ \frac{\partial F}{\partial x_1} \end{bmatrix} = -\frac{\partial F}{\partial x_1} \frac{\partial F}{\partial x_2} + \frac{\partial F}{\partial x_2} \frac{\partial F}{\partial x_1} = 0$$

### Illustrative Example

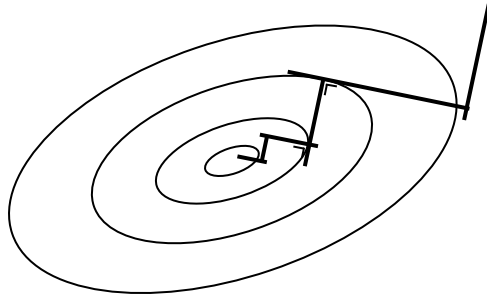
- Consider the function  $F(\mathbf{x}) = x_1^2 + x_2^2 = \mathbf{x}^T \mathbf{x}$ , where  $\mathbf{x} = [x_1 \ x_2]^T$ .

- The gradient of  $F(\mathbf{x})$  is given by:  $\nabla F = \begin{bmatrix} \partial F / \partial x_1 \\ \partial F / \partial x_2 \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 2x_2 \end{bmatrix}$
- Taking the starting point  $\mathbf{x}_0 = \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix}$ , the gradient at this point is:  $\begin{bmatrix} 2(\frac{1}{2}) \\ 2(1) \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ .
- The negative of the gradient at this point is therefore:  $-\nabla F = \begin{bmatrix} -1 \\ -2 \end{bmatrix}$
- Presenting this information of a contour plot gives:



- We see that  $-\nabla F$  passes through the minimum of  $F$  and is a radius of the circular contours.
- Hence, a search along the line  $\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} - \alpha \begin{bmatrix} 1 \\ 2 \end{bmatrix}$  will find the minimum of  $F$ .

- Note that  $\nabla F$  is orthogonal to the contour at  $\mathbf{x}_0$ . This is always true at any point.
- Search directions start orthogonal to one contour and end tangential to another. Hence search directions are orthogonal. Thus, this can lead to slow convergence on non-spherical surfaces, as illustrated below:



- Clearly, the steepest descent approach is not always efficient, although it does eventually converge to a minimum (or a saddle point!).

## Steepest Descent Algorithm

- The steepest descent (or gradient descent) algorithm works in the same manner as the direct search methods but in this case, we search along the line given by:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \nabla F(\mathbf{x}^{(k)})$$

- Once again,  $k$  represents the  $k^{\text{th}}$  iteration of the algorithm. We find the value of  $\alpha$  that minimises the function  $F(\mathbf{x})$ , update our new estimate of the minimum and repeat the process until the solution converges.
- Convergence, in this case, is given by either  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \varepsilon_1$  or  $\|\mathbf{g}^{(k+1)}\| < \varepsilon_2$ . In the latter case, if the gradient is very small, then you are effectively no longer descending!

## Example of Steepest Descent

- Consider the function  $f(\mathbf{x}) = x_1x_2 + x_1^2 + 2x_2^2$ , with the starting point  $\mathbf{x}_0 = [1 \ 1]^T$ . Carry out two iterations of the steepest descent method on this function.

- Firstly, we need to determine the gradient at the starting point.

- The gradient vector is given as:  $\nabla f = \begin{bmatrix} x_2 + 2x_1 \\ x_1 + 4x_2 \end{bmatrix}$

- Evaluated at  $\mathbf{x}_0$ , we get:  $\nabla f(\mathbf{x}_0) = \begin{bmatrix} 3 \\ 5 \end{bmatrix}$

- So now we have our search line:  $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} - \alpha \nabla f(\mathbf{x}^{(0)})$

- Here  $\mathbf{x}^{(0)}$  is the starting value  $\mathbf{x}_0$ . Therefore:  $\mathbf{x}^{(1)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \alpha \begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 1 - 3\alpha \\ 1 - 5\alpha \end{bmatrix}$

- Find the value of  $\alpha$  that minimises  $f(\alpha)$ :

$$f(\alpha) = 74\alpha^2 - 34\alpha + 4$$

$$\frac{df}{d\alpha} = 148\alpha - 34 = 0 \quad \Rightarrow \quad \alpha = 0.2297297 \quad \text{and} \quad f(\alpha) = 0.0946$$

$$\frac{d^2 f}{d\alpha^2} = 148 > 0 \quad \Rightarrow \text{minimum}$$

- This gives the new point:  $\mathbf{x}^{(1)} = \begin{bmatrix} 0.3108 \\ -0.1486 \end{bmatrix}$
- For the second iteration, we need to determine the new search direction. This is simply the gradient at  $\mathbf{x}^{(1)}$ , which is:

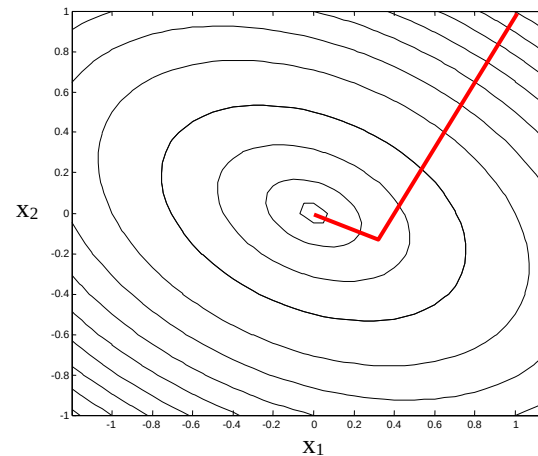
$$\nabla f(\mathbf{x}^{(1)}) = \begin{bmatrix} 0.4730 \\ -0.2836 \end{bmatrix}$$

- So now we have the search line:  $\mathbf{x}^{(2)} = \mathbf{x}^{(1)} - \alpha \nabla f(\mathbf{x}^{(1)})$
- This gives:
$$\mathbf{x}^{(2)} = \begin{bmatrix} 0.3108 - 0.4730\alpha \\ -0.1486 + 0.2836\alpha \end{bmatrix}$$
- The value of  $\alpha$  that minimises  $f(\alpha)$  is

$$\alpha = 0.6072 \quad \text{and} \quad f(\alpha) = 0.0022 \quad (\text{getting close to } 0!)$$

- This gives the new point:  $\mathbf{x}^{(2)} = \begin{bmatrix} 0.02359 \\ 0.02360 \end{bmatrix}$

- Graphically, we can represent our calculations as follows:



- So, in this example, after two iterations we have almost converged on the actual minimum of  $[0 \ 0]^T$ .
- However, this is not always the case. In general, like the univariate search method, steepest descent can be inefficient.
- With the univariate search method, we were able to adapt the algorithm slightly to make it more efficient (hence the DSC method). Likewise, it turns out, we can carry out a slight modification of the steepest descent approach in order to improve its efficiency.

## 4.2 The Conjugate Gradient Algorithm

- First, let us analyse the behaviour of the univariate search algorithm on the general spherical function in n-dimensions, i.e.:

$$F(\mathbf{x}) = a + \mathbf{b}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{x}$$

where  $\mathbf{x}$  is a n-dimensional vector.

- Note, this is a specific case of the more general quadratic function, given by:

$$a + \mathbf{b}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{G} \mathbf{x}$$

with the Hessian matrix  $\mathbf{G} = \mathbf{I}$

- Expanding  $F(\mathbf{x})$  we get:

$$F(\mathbf{x}) = a + b_1 x_1 + b_2 x_2 + \dots + b_n x_n + \frac{1}{2} (x_1^2 + x_2^2 + \dots + x_n^2)$$

$$\Rightarrow F(\mathbf{x}) = a + (b_1 x_1 + \frac{1}{2} x_1^2) + (b_2 x_2 + \frac{1}{2} x_2^2) + \dots + (b_n x_n + \frac{1}{2} x_n^2)$$

- This expression highlights the fact that we can minimise  $F(\mathbf{x})$  by choosing values of  $x_1, x_2, \dots, x_n$  separately so as to minimise each bracketed term independently.
- For example:

$$\frac{\partial}{\partial x_1} (b_1 x_1 + \frac{1}{2} x_1^2) = b_1 + x_1 = 0 \Rightarrow x_1 = -b_1$$

- Similarly,  $x_2 = -b_2$ , etc...
- *In other words, we can search along each axis independently and in any order, finding the minimum in  $n$  steps.*
- *This is because the Hessian matrix, the identity matrix in this case, has no off-diagonal terms, i.e. the off-diagonal terms are all zero.*
- Now consider the general quadratic function, with a positive definite  $\mathbf{G}$ :

$$F(\mathbf{x}) = a + \mathbf{b}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{G} \mathbf{x}$$

- Let the vectors  $\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_n$  be a set of search directions and let us define the matrix  $\mathbf{T}$  as:

$$\mathbf{T} = [\mathbf{t}_1 \ \mathbf{t}_2 \ \dots \ \mathbf{t}_n]$$

- We now define a new vector  $\mathbf{y} = [y_1 \ y_2 \ \dots \ y_n]^T$ , so that:  $\mathbf{x} = \mathbf{T}\mathbf{y}$

- As long as  $\mathbf{T}$  is non-singular (i.e.  $\mathbf{t}_i$  is not linearly dependent), this results in a 1-to-1 correspondence between  $\mathbf{x}$  and  $\mathbf{y}$ .

- Thus:

$$F(\mathbf{y}) = a + \mathbf{b}^T \mathbf{T} \mathbf{y} + \frac{1}{2} \mathbf{y}^T \mathbf{T}^T \mathbf{G} \mathbf{T} \mathbf{y}$$

where:

$$\mathbf{T}^T \mathbf{G} \mathbf{T} = \begin{bmatrix} t_1^T \\ t_2^T \\ \vdots \\ t_n^T \end{bmatrix} G \begin{bmatrix} t_1 & t_2 & \cdots & t_n \end{bmatrix} = \begin{bmatrix} t_1^T G t_1 & t_1^T G t_2 & \cdots & t_1^T G t_n \\ t_2^T G t_1 & t_2^T G t_2 & & \\ \vdots & & \ddots & \vdots \\ t_n^T G t_1 & & \cdots & t_n^T G t_n \end{bmatrix}$$

- If we can obtain  $\mathbf{t}_i$  so that:

$$\mathbf{t}_i^T \mathbf{G} \mathbf{t}_j = 0, i \neq j$$

then  $\mathbf{T}^T \mathbf{G} \mathbf{T}$  will have zero off-diagonal terms, i.e.:

$$\mathbf{T}^T \mathbf{G} \mathbf{T} = \begin{bmatrix} d_{11} & & & 0 \\ & d_{22} & & \\ & & \ddots & \\ 0 & & & d_{nn} \end{bmatrix} = \mathbf{D}$$

- Vectors that have the property  $\mathbf{t}_i^T \mathbf{G} \mathbf{t}_j = 0$ ,  $i \neq j$  are said to be **conjugate** with respect to  $\mathbf{G}$ .
- Note that the vectors  $\mathbf{t}_i$  are not necessarily orthogonal unless  $\mathbf{G}$  happens to be a unit matrix.
- Letting  $\mathbf{b}^T \mathbf{T} = \mathbf{c} = [c_1 \ c_2 \ \dots \ c_n]^T$ , we can express  $F(\mathbf{y})$  as:

$$F(\mathbf{y}) = a + \mathbf{c}^T \mathbf{y} + \frac{1}{2} \mathbf{y}^T \mathbf{D} \mathbf{y}$$

$$\Rightarrow F(\mathbf{y}) = a + (c_1 y_1 + \frac{1}{2} d_{11} y_1^2) + (c_2 y_2 + \frac{1}{2} d_{22} y_2^2) + \dots + (c_n y_n + \frac{1}{2} d_{nn} y_n^2)$$

- The minimum of  $F(\mathbf{y})$  can now be found by minimising with respect to  $y_1, y_2, \dots, y_n$  separately in any order in  $n$  steps. For eg.:

$$\frac{\partial}{\partial y_1} (c_1 y_1 + \frac{1}{2} d_{11} y_1^2) = c_1 + d_{11} y_1 = 0 \Rightarrow y_1 = -\frac{c_1}{d_{11}}$$

- Similarly,  $y_2 = -\frac{c_2}{d_{22}}$ , etc...
- An alternative way to view this is that we can search for a minimum along each direction  $\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_n$  independently because:

$$\mathbf{x} = \mathbf{T}\mathbf{y} = y_1 \mathbf{t}_1 + y_2 \mathbf{t}_2 + \dots y_n \mathbf{t}_n$$

- Hence, changing  $y_i$  is equivalent to moving along the line:  $\mathbf{x} + y_i \mathbf{t}_i$
- Therefore finding the minimum of  $F(\mathbf{y})$  with respect to  $y_i$  is equivalent to finding the minimum along  $\mathbf{t}_i$ . The value  $y_i$  determines how far along the direction  $\mathbf{t}_i$  we travel.
- Thus, any optimisation algorithm which generates search directions which are conjugate with respect to the Hessian matrix of a n-dimensional quadratic function can find the minimum in n steps.

- Conjugate search directions are also useful on non-quadratic functions because, close to a local minimum, most smooth continuous functions can be closely approximated by a quadratic and hence fast final convergence can be achieved.

### Illustrative Example (given conjugate vectors!)

- Consider the following quadratic function:

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{x}^T \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 0.5\mathbf{x}^T \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix} \mathbf{x} \\ &= x_1 + x_2 + x_2^2 + 0.5x_1x_2 + 0.5x_1^2 \end{aligned}$$

- Show that the vectors  $\mathbf{t}_1 = [1 \ 1]^T$  and  $\mathbf{t}_2 = [-\frac{5}{3} \ 1]^T$  are conjugate with respect to the hessian matrix and, by searching in the direction of these vectors, determine the minimum of the function, given the starting point  $[2 \ 2]^T$ .
- In order to show that the vectors are conjugate to the hessian matrix, we need to show that  $\mathbf{t}_1^T \mathbf{G} \mathbf{t}_2 = \mathbf{0}$ , where  $\mathbf{G}$  is the hessian matrix. Therefore:

$$\begin{aligned}\mathbf{t}_1^T \mathbf{G} \mathbf{t}_2 &= [1 \quad 1] \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix} \begin{bmatrix} -5 \\ 3 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 3 & 5 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} -5 \\ 3 \\ 1 \end{bmatrix} = -\frac{5}{2} + \frac{5}{2} = 0\end{aligned}$$

- Hence, the vectors  $\mathbf{t}_1$  and  $\mathbf{t}_2$  are indeed conjugate with respect to the hessian matrix.
- In order to obtain the minimum, we simply search along the direction of each conjugate vector in turn.
- Hence looking along  $\mathbf{t}_1$ , we have:

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{t}_1$$

- This gives:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \alpha \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 + \alpha \\ 2 + \alpha \end{bmatrix}$$

- Find  $\alpha$  to minimise  $f(\alpha)$ :

$$f(\alpha) = 2\alpha^2 + 10\alpha + 12$$

$$\frac{df}{d\alpha} = 4\alpha + 10 = 0 \quad \Rightarrow \alpha = -2.5 \quad \text{and} \quad f(\alpha) = -0.5$$

$$\frac{d^2f}{d\alpha^2} = 4 > 0 \quad \Rightarrow \text{minimum}$$

- Hence:  $\mathbf{x}^{(1)} = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}$

- Now search along  $\mathbf{t}_2$ :

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \alpha \mathbf{t}_2$$

- This gives:

$$\mathbf{x}^{(2)} = \begin{bmatrix} -\frac{1}{2} - \frac{5}{3}\alpha \\ -\frac{1}{2} + \alpha \end{bmatrix}$$

- Find  $\alpha$  to minimise  $f(\alpha)$ :

$$f(\alpha) = \frac{14}{9}\alpha^2 - \frac{2}{3}\alpha - \frac{1}{2}$$

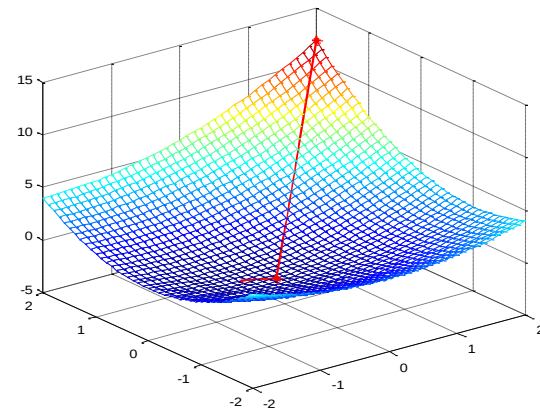
$$\frac{df}{d\alpha} = \frac{28}{9}\alpha - \frac{2}{3} = 0 \Rightarrow \alpha = \frac{3}{14} \quad \text{and} \quad f(\alpha) = -\frac{4}{7} = -0.5714$$

$$\frac{d^2f}{d\alpha^2} = \frac{28}{9} > 0 \Rightarrow \text{minimum}$$

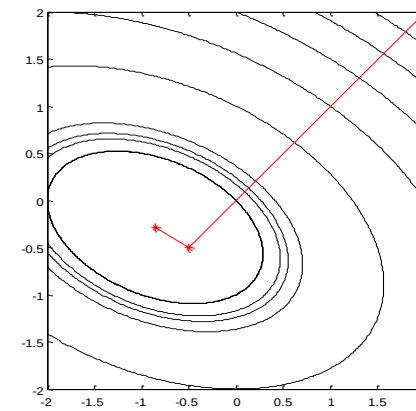
- Hence:

$$\mathbf{x}^{(2)} = \begin{bmatrix} -\frac{6}{7} \\ \frac{2}{7} \end{bmatrix} = \begin{bmatrix} -0.8571 \\ -0.2857 \end{bmatrix}$$

- Thus, after  $n$  iterations ( $n = 2$  in this case), we have reached the exact minimum. We can confirm this by plotting the contours of the function, as shown below:



Surface Plot



Contour Plot

- In the last example, we were given the conjugate directions in which to search. The problem now becomes one of generating actual conjugate search directions!
- There are several different methods that generate such directions one at a time, as they are needed.
- One of these methods requires only a small modification to the steepest descent algorithm and it is known as the ***conjugate gradient method*** of Fletcher & Reeves (or the ***Fletcher-Reeves method***).

#### 4.2.1 Conjugate Gradient or Fletcher-Reeves method

- The Fletcher-Reeves method works as follows.
- As before, we let  $\mathbf{x}^{(0)}$  be the starting point and  $\mathbf{p}^{(k)}$  the search direction from  $\mathbf{x}^{(k)}$ .
- We set the first search direction to be the steepest descent:

$$\mathbf{p}^{(0)} = -\nabla F(\mathbf{x}^{(0)}) = -\mathbf{g}^{(0)}$$

- For convenience, we denote  $\mathbf{g} = \nabla F(\mathbf{x})$ . Hence  $\mathbf{p}^{(0)} = -\mathbf{g}^{(0)}$ .

- We then set:

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(0)}$$

- We choose  $\alpha$  to minimise the function  $F(\mathbf{x})$  along  $\mathbf{p}^{(0)}$  and thus obtain  $\mathbf{x}^{(1)}$ .

- Now we deviate from the steepest descent approach, by setting the next search direction  $\mathbf{p}^{(1)}$  as:

$$\mathbf{p}^{(1)} = -\mathbf{g}^{(1)} + \beta^{(1)} \mathbf{p}^{(0)}$$

where:

$$\beta^{(1)} = \frac{\mathbf{g}^{(1)T} \mathbf{g}^{(1)}}{\mathbf{g}^{(0)T} \mathbf{g}^{(0)}}$$

- In general, for all search directions except for  $\mathbf{p}^{(0)}$ , we have:

$$\mathbf{p}^{(k+1)} = -\mathbf{g}^{(k+1)} + \beta^{(k+1)}\mathbf{p}^{(k)}$$

where:

$$\beta^{(k+1)} = \frac{\mathbf{g}^{(k+1)T}\mathbf{g}^{(k+1)}}{\mathbf{g}^{(k)T}\mathbf{g}^{(k)}}$$

- We then repeat the line search using this new search direction. The process continues until we converge to a suitable value - either  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \varepsilon_1$  or  $\|\mathbf{g}^{(k+1)}\| < \varepsilon_2$ .
- Conjugate gradients is therefore similar to steepest descent, but at each iteration the search direction has a multiple of the previous search direction added to the steepest descent direction.
- As noted already, conjugate gradient has n-step quadratic convergence.
- On non-quadratic functions, we reset the search direction to steepest descent after every n iterations.

- The key points of this particular method is that it is relatively simple and, moreover, the conjugate directions for a quadratic function are determined without actually calculating the Hessian matrix.
- Hence it requires the storage of only one vector,  $\mathbf{p}^{(k-1)}$  as well as  $\mathbf{x}^{(k)}$  and  $\mathbf{p}^{(k)}$ . As a result, this method is often used on very large problems ( $n > 100$ ).
- A drawback of this method is that the function must be accurately minimised along each search direction. This is expensive in terms of function evaluations.

### **Short Proof – Fletcher-Reeves generates conjugate directions**

- Before looking at an example, we will now prove that the Fletcher-Reeves algorithm generates conjugate directions on two-dimensional quadratic functions.
- The 2-d quadratic function is given by:  $F(\mathbf{x}) = a + \mathbf{b}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x}$
- The gradient of this function at point  $\mathbf{x}$  is given by:

$$\mathbf{g} = \nabla F(\mathbf{x}) = \mathbf{b} + \mathbf{H}\mathbf{x}$$

- Hence:

$$\mathbf{g}^{(0)} = \mathbf{b} + \mathbf{H}\mathbf{x}^{(0)} \quad \text{and} \quad \mathbf{g}^{(1)} = \mathbf{b} + \mathbf{H}\mathbf{x}^{(1)} \quad (\text{equations X1 for reference})$$

- The search direction  $\mathbf{p}^{(0)}$  is given by:

$$\mathbf{p}^{(0)} = -\mathbf{g}^{(0)} = -(\mathbf{b} + \mathbf{H}\mathbf{x}^{(0)})$$

- The new point is at:

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(0)} \quad (\text{equation X2 for reference})$$

- The next search direction is then :

$$\mathbf{p}^{(1)} = -\mathbf{g}^{(1)} + \beta^{(1)}\mathbf{p}^{(0)} = -\mathbf{g}^{(1)} - \beta^{(1)}\mathbf{g}^{(0)}, \quad \text{with} \quad \beta^{(1)} = \frac{\mathbf{g}^{(1)T}\mathbf{g}^{(1)}}{\mathbf{g}^{(0)T}\mathbf{g}^{(0)}}$$

- Thus, in order to prove that conjugate search directions are generated, we need to show that:

$$\mathbf{p}^{(1)T} \mathbf{H} \mathbf{p}^{(0)} = 0$$

- We know that:

$$\mathbf{p}^{(1)T} \mathbf{H} \mathbf{p}^{(0)} = (-\mathbf{g}^{(1)} - \beta^{(1)} \mathbf{g}^{(0)})^T \mathbf{H} \mathbf{p}^{(0)}$$

- Rewriting equation X2 gives:

$$\mathbf{p}^{(0)} = \frac{1}{\alpha} (\mathbf{x}^{(1)} - \mathbf{x}^{(0)})$$

- Therefore:

$$\mathbf{p}^{(1)T} \mathbf{H} \mathbf{p}^{(0)} = \frac{1}{\alpha} (-\mathbf{g}^{(1)} - \beta^{(1)} \mathbf{g}^{(0)})^T \mathbf{H} (\mathbf{x}^{(1)} - \mathbf{x}^{(0)})$$

- From equations X1, we can show that:

$$\mathbf{H} (\mathbf{x}^{(1)} - \mathbf{x}^{(0)}) = \mathbf{g}^{(1)} - \mathbf{g}^{(0)}$$

- Therefore:

$$p^{(1)T} \mathbf{H} \mathbf{p}^{(0)} = \frac{1}{\alpha} (-\mathbf{g}^{(1)} - \beta^{(1)} \mathbf{g}^{(0)})^T (\mathbf{g}^{(1)} - \mathbf{g}^{(0)})$$

- Multiplying out the RHS gives:

$$p^{(1)T} \mathbf{H} \mathbf{p}^{(0)} = \frac{1}{\alpha} (-g^{(1)T} \mathbf{g}^{(1)} + g^{(1)T} \mathbf{g}^{(0)} - \beta^{(1)} g^{(0)T} \mathbf{g}^{(1)} + \beta^{(1)} g^{(0)T} \mathbf{g}^{(0)})$$

- Since  $\mathbf{x}^{(1)}$  is the position of the minimum of  $F(\mathbf{x})$  along  $\mathbf{p}^{(0)}$ , and we know that the gradient at this point, i.e.  $\mathbf{g}^{(1)}$ , is orthogonal to the gradient that resulted in  $\mathbf{x}^{(1)}$  in the first place, i.e.  $\mathbf{g}^{(0)}$ , then:

$$g^{(1)T} \mathbf{g}^{(0)} = 0$$

- Hence:

$$p^{(1)T} \mathbf{H} \mathbf{p}^{(0)} = \frac{1}{\alpha} (-g^{(1)T} \mathbf{g}^{(1)} + \beta^{(1)} g^{(0)T} \mathbf{g}^{(0)})$$

- Substituting for  $\beta^{(1)}$  gives:

$$p^{(1)T} \mathbf{H} \mathbf{p}^{(0)} = 0$$

- Thus, the search directions  $\mathbf{p}$  are mutually conjugate and so the Fletcher-Reeves algorithm generates conjugate search directions in two dimensions.
- To prove this for n-dimensions is a little more complicated! – refer to a good textbook for detail!

### **Illustrative Example of Fletcher-Reeves method (not given conjugate vectors!)**

- Once again consider the quadratic function:

$$f(\mathbf{x}) = x_1 + x_2 + x_2^2 + 0.5x_1x_2 + 0.5x_1^2$$

- Starting with the point  $[2 \ 2]^T$ , use the Fletcher-Reeves conjugate gradient method to obtain the minimum of the above function.
- The gradient of the function is given by:

$$\mathbf{g} = \nabla f(\mathbf{x}) = \begin{bmatrix} 1 + 0.5x_2 + x_1 \\ 1 + 2x_2 + 0.5x_1 \end{bmatrix}$$

- The first search direction is simply the steepest descent. Hence:

$$\mathbf{p}^{(0)} = -\mathbf{g}^{(0)} = -\begin{bmatrix} 1 + 0.5(2) + 2 \\ 1 + 2(2) + 0.5(2) \end{bmatrix} = -\begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

- Therefore:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \alpha \begin{bmatrix} -4 \\ -6 \end{bmatrix} = \begin{bmatrix} 2 - 4\alpha \\ 2 - 6\alpha \end{bmatrix}$$

- Now we find  $\alpha$  that minimises  $f(\alpha)$ :

$$f(\alpha) = 56\alpha^2 - 52\alpha + 12$$

- Evaluating  $\frac{df}{d\alpha} = 0$  gives  $\alpha_{\min} = 0.4643$  and  $f(\alpha) = -0.07143$ .

- Hence:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 0.1428 \\ -0.7858 \end{bmatrix}$$

- Now we need to calculate the next search direction which is given by:

$$\mathbf{p}^{(1)} = -\mathbf{g}^{(1)} + \beta^{(1)}\mathbf{p}^{(0)}, \quad \beta^{(1)} = \frac{\mathbf{g}^{(1)T}\mathbf{g}^{(1)}}{\mathbf{g}^{(0)T}\mathbf{g}^{(0)}}$$

- Evaluating each of the necessary terms gives:

$$\mathbf{g}^{(1)} = \begin{bmatrix} 0.75 \\ -0.5 \end{bmatrix}$$

$$\beta^{(1)} = \frac{0.75^2 + (-0.5)^2}{4^2 + 6^2} = 0.015625$$

- Hence, the search direction is:

$$\begin{aligned} \mathbf{p}^{(1)} &= - \begin{bmatrix} 0.75 \\ -0.5 \end{bmatrix} - 0.015625 \begin{bmatrix} 4 \\ 6 \end{bmatrix} \\ &= \begin{bmatrix} -0.8125 \\ 0.40625 \end{bmatrix} \end{aligned}$$

- Our next point is now:

$$\mathbf{x}^{(2)} = \begin{bmatrix} 0.1428 - 0.8125\alpha \\ -0.7858 + 0.40625\alpha \end{bmatrix}$$

- Again we find  $\alpha$  that minimises  $f(\alpha)$ :

$$f(\alpha) = 0.3300779\alpha^2 - 0.8124975\alpha - 0.07143$$

- Evaluating  $\frac{df}{d\alpha} = 0$  gives  $\alpha_{\min} = 1.230766$  and  $f(\alpha) = -0.5714$ .

- Hence:

$$\mathbf{x}^{(2)} = \begin{bmatrix} -0.8571 \\ -0.2857 \end{bmatrix}$$

- This is the actual minimum. See previous example for confirmation.
- Hence, this example shows that the Fletcher-Reeves method successfully identifies suitable conjugate search directions and, therefore, obtains the minimum in two iterations.

## 5. Hessian-based Gradient Methods

- We are now going to study some gradient methods that make use of both the first and second derivatives of  $F(\mathbf{x})$ .

### 5.1 Newton's Method

- Let  $\mathbf{x}^{(k)}$  be the current estimate of a minimum of the function  $F(\mathbf{x})$ .
- Now, using Taylor's Theorem, consider the quadratic approximation to  $F(\mathbf{x})$ , i.e.  $Q(\mathbf{x})$ , about this point:

$$Q(\mathbf{x}) \approx F(\mathbf{x}^{(k)}) + (\mathbf{x} - \mathbf{x}^{(k)})^T \mathbf{g}^{(k)} + \frac{1}{2} (\mathbf{x} - \mathbf{x}^{(k)})^T \mathbf{H}^{(k)} (\mathbf{x} - \mathbf{x}^{(k)})$$

where:

$$\mathbf{g}^{(k)} = \nabla F(\mathbf{x}^{(k)}) \quad \text{and} \quad \mathbf{H}^{(k)} = \nabla^2 F(\mathbf{x}^{(k)})$$

- An improved estimate of the actual minimum can be obtained by finding the minimum of this approximate quadratic approximation.

- Differentiating the quadratic approximation gives:

$$\nabla Q(\mathbf{x}) \approx \mathbf{g}^{(k)} + \mathbf{H}^{(k)}(\mathbf{x} - \mathbf{x}^{(k)})$$

- If we denote the minimum of the quadratic approximation as  $\mathbf{x}^*$ , then

$$\mathbf{g}^{(k)} + \mathbf{H}^{(k)}(\mathbf{x}^* - \mathbf{x}^{(k)}) = 0$$

$$\Rightarrow \mathbf{H}^{(k)}(\mathbf{x}^* - \mathbf{x}^{(k)}) = -\mathbf{g}^{(k)}$$

- Let  $\delta \mathbf{x}^{(k)} = \mathbf{x}^* - \mathbf{x}^{(k)}$ , i.e.  $\delta \mathbf{x}^{(k)}$  is an estimate of the correction to be applied to  $\mathbf{x}^{(k)}$  in order to arrive at the minimum. Hence:

$$\mathbf{H}^{(k)} \delta \mathbf{x}^{(k)} = -\mathbf{g}^{(k)}$$

$$\Rightarrow \delta \mathbf{x}^{(k)} = -\mathbf{H}^{(k)^{-1}} \mathbf{g}^{(k)}$$

- Thus an improved estimate of the actual minimum of  $F(\mathbf{x})$  is given by:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \delta \mathbf{x}^{(k)}$$

$$\text{or } \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}^{(k)^{-1}} \mathbf{g}^{(k)}$$

- Note that the hessian matrix  $\mathbf{H}^{(k)}$  is assumed positive definite. If it wasn't, then the approximating function has no minimum!
- This approach is called **Newton's Method**.
- On a positive definite quadratic function, Newton's method has a 1-step quadratic convergence. In other words, it will always find the minimum in 1 step, regardless of the starting point.
- On non-quadratic functions, it is typically used to generate search directions, with one-dimensional searches along these directions.

## Newton's Algorithm for Quadratic Functions

- Select a starting point  $\mathbf{x}^{(0)}$ .
- Solve the following set of equations to get  $\mathbf{p}^{(0)}$ :

$$\mathbf{H}^{(0)}\mathbf{p}^{(0)} = -\mathbf{g}^{(0)}$$

- The minimum is then:

$$\mathbf{x}_{\min} = \mathbf{x}^{(0)} + \mathbf{p}^{(0)}$$

### **Newton's Algorithm for General Functions**

- Select a starting point  $\mathbf{x}^{(0)}$  and evaluate  $F(\mathbf{x}^{(0)})$ .
- Starting with  $k = 0$ , solve the following set of equations to get  $\mathbf{p}^{(k)}$ :

$$\mathbf{H}^{(k)}\mathbf{p}^{(k)} = -\mathbf{g}^{(k)}$$

- Using a line search procedure, solve:

$$\min_{\alpha} F(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}) \quad \text{to give } \alpha^{(k)}$$

- Check for convergence. If  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|$  is sufficiently small or if  $F(\mathbf{x}^{(k+1)}) - F(\mathbf{x}^{(k)})$  is sufficiently small, then stop. Otherwise, set  $k = k+1$  and repeat the process.

### **Illustrative Example (Quadratic Function)**

- Consider the quadratic function given by:

$$f(\mathbf{x}) = x_1 + x_2 + x_2^2 + 0.5x_1x_2 + 0.5x_1^2$$

- Starting with the point  $\mathbf{x}^{(0)} = [2 \ 2]^T$ , use Newton's method to obtain the minimum of the above function.

- The gradient of the function is given by:

$$\mathbf{g} = \nabla f(\mathbf{x}) = \begin{bmatrix} 1 + 0.5x_2 + x_1 \\ 1 + 2x_2 + 0.5x_1 \end{bmatrix}$$

- Calculate at  $\mathbf{x}^{(0)}$  we get:

$$\mathbf{g}^{(0)} = \nabla f(\mathbf{x}^{(0)}) = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

- The hessian matrix is given by:

$$\mathbf{H} = \nabla^2 f(\mathbf{x}) = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix}$$

- Clearly the hessian matrix is independent of the starting point and therefore is the same for all arbitrary starting points!  
Hence:

$$\mathbf{H}^{(0)} = \nabla^2 f(\mathbf{x}^{(0)}) = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix}$$

- Solving the set of equations  $\mathbf{H}^{(0)}\mathbf{p}^{(0)} = -\mathbf{g}^{(0)}$  gives:

$$\mathbf{p}^{(0)} = \begin{bmatrix} -\frac{20}{7} \\ -\frac{16}{7} \end{bmatrix}$$

- Hence, as in the previous examples, the minimum is:

$$\mathbf{x}_{\min} = \mathbf{x}^{(0)} + \mathbf{p}^{(0)} \Rightarrow \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} -\frac{20}{7} \\ -\frac{16}{7} \end{bmatrix} = \begin{bmatrix} -\frac{6}{7} \\ -\frac{2}{7} \end{bmatrix}$$

- As already stated, Newton's method has a 1-step convergence on positive definite quadratic functions.
- Furthermore, the search directions are downhill if  $\mathbf{H}^{(k)}$  is positive definite. This is because the slope of F along  $\mathbf{p}^{(k)}$  is:

$$\mathbf{p}^{(k)T} \mathbf{g}^{(k)} = -\mathbf{g}^{(k)T} \mathbf{H}^{(k)-1} \mathbf{g}^{(k)}$$

Note:  $(\mathbf{H}\mathbf{g})^T = \mathbf{g}^T \mathbf{H}^T$  and if  $\mathbf{H}$  is symmetric, then  $\mathbf{H}^T = \mathbf{H}$

- Since  $\mathbf{H}^{(k)}$  is taken to be positive definite, so too is  $\mathbf{H}^{(k)-1}$ . Note a positive definite matrix has a positive definite inverse.
- Hence the slope of F is negative, i.e. downhill, unless  $\mathbf{g}^{(k)} = \mathbf{0}$ , in which case, the search direction  $\mathbf{p}^{(k)} = \mathbf{0}$ .
- Unfortunately Newton's method has several disadvantages also:
  - It requires the second derivatives to be computed. This is expensive (especially for large n, as the effort is proportional to  $n^2$ ) and inaccurate unless analytical formulae are available.

- A system of linear equations must be solved on each iteration. This can be costly for problems with large dimensions.
- The Hessian matrix  $\mathbf{H}^{(k)}$  at  $\mathbf{x}^{(k)}$  may be singular (i.e. non-invertible!) even if  $F(\mathbf{x})$  has a local minimum. Hence the method fails to compute a search direction.
- $\mathbf{H}^{(k)}$  may not be positive definite (i.e. uphill search direction produced).
- Therefore Newton's method is not much use, except for small problems ( $n \leq 5$  say), preferably with analytic expressions for the second derivatives! Also, knowledge of a good starting point is useful.
- However, due to its good ultimate convergence, Newton's method does provide the incentive for more efficient algorithms for unconstrained optimisation.

## 5.2 Quasi-Newton or Variable Metric Methods

- Suppose we know  $n+1$  values of the gradient, say  $\mathbf{g}^{(0)}, \mathbf{g}^{(1)}, \dots, \mathbf{g}^{(n)}$ , corresponding to the  $n+1$  points  $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}$  on a positive definite quadratic function  $Q(\mathbf{x})$ .

- Then we can compute the Hessian matrix as follows:
- We know that:  $Q(\mathbf{x}) = a + \mathbf{b}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x}$  and  $\mathbf{g} = \mathbf{b} + \mathbf{H} \mathbf{x}$
- Then:  $\mathbf{g}^{(k)} - \mathbf{g}^{(k-1)} = \mathbf{H}(\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)})$
- Denoting  $\boldsymbol{\lambda}^{(k)} = \mathbf{g}^{(k)} - \mathbf{g}^{(k-1)}$  and  $\boldsymbol{\delta}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$ , we have:  $\boldsymbol{\lambda}^{(k)} = \mathbf{H} \boldsymbol{\delta}^{(k)}$
- Hence:
$$\begin{aligned}\boldsymbol{\lambda}^{(1)} &= \mathbf{H} \boldsymbol{\delta}^{(1)} \\ \boldsymbol{\lambda}^{(2)} &= \mathbf{H} \boldsymbol{\delta}^{(2)} \\ \vdots & \quad \vdots \\ \boldsymbol{\lambda}^{(n)} &= \mathbf{H} \boldsymbol{\delta}^{(n)}\end{aligned}$$
- We can express this set of equations as:

$$\mathbf{K} = \mathbf{H} \Delta$$

where  $\mathbf{K} = \{\lambda^{(1)}, \lambda^{(2)}, \dots, \lambda^{(n)}\}$  and  $\Delta = \{\delta^{(1)}, \delta^{(2)}, \dots, \delta^{(n)}\}$

- Hence, we can calculate  $\mathbf{H}$  as:

$$\mathbf{H} = \mathbf{K} \Delta^{-1} \quad \text{or} \quad \mathbf{H}^{-1} = \Delta \mathbf{K}^{-1}$$

- Thus, we can use Newton's method to obtain the minimum of the quadratic function  $Q(\mathbf{x})$  in one step using:

$$\mathbf{x}^{(n+1)} = \mathbf{x}_{\min} = \mathbf{x}^{(n)} - \Delta \mathbf{K}^{-1} \mathbf{g}^{(n)}$$

- Hence, we have computed a Newton step without having explicitly calculated the Hessian matrix.
- However, this approach still involves solving a set of linear equations. Furthermore,  $n+1$  calculations of the gradient are required before we obtain the second derivative.

- Clearly, this approach is not very practical but it is useful as a basic idea from which to derive better, more efficient methods.
- Better methods have been devised which use the  $\lambda^{(k)}$  and  $\delta^{(k)}$  vectors in a more efficient manner. These methods estimate the inverse of the Hessian, using formulae that improve this estimate with each iteration.
- These methods are called the Quasi-Newton or Variable Metric methods.

### Standard Approach:

- Recall that the Newton's Method produces the search direction:

$$\mathbf{p}^{(k)} = -\mathbf{H}^{(k)^{-1}}\mathbf{g}^{(k)}$$

- The steepest descent method, from earlier, produces the search direction:

$$\mathbf{p}^{(k)} = -\mathbf{g}^{(k)} = -\mathbf{I}\mathbf{g}^{(k)}$$

- Both these approaches produce search directions by multiplying the gradient vector by a matrix (albeit an implicit identity matrix in the latter case!).
- We can view these methods as examples of a general class of methods that generates search directions of the form:

$$\mathbf{p}^{(k)} = -\mathbf{M}^{(k)} \mathbf{g}^{(k)}$$

- The Quasi-Newton methods produce a sequence of matrices  $\mathbf{M}^{(0)}, \mathbf{M}^{(1)}, \dots, \mathbf{M}^{(k)}$ , which are generated by updating  $\mathbf{M}^{(i)}$  to give  $\mathbf{M}^{(i+1)}$ . Each of these are used as a substitute for  $\mathbf{H}^{(k)-1}$ .
- The updating process at  $\mathbf{x}^{(k)}$  involves only the use of  $\mathbf{M}^{(k-1)}$ ,  $\boldsymbol{\lambda}^{(k)}$  and  $\boldsymbol{\delta}^{(k)}$  and is designed to ensure that the following condition (a property of  $\mathbf{H}^{(k)-1}$ ) is satisfied:

$$\mathbf{M}^{(k)} \boldsymbol{\lambda}^{(k)} = \boldsymbol{\delta}^{(k)}$$

- This can be achieved in a number of ways and as such there are various Quasi-Newton methods. The original one is associated with the names **Davidon, Fletcher and Powell**, and we refer to their method as the **DFP algorithm**.

### 5.2.1 The DFP Algorithm

- **Step 1:** Choose an arbitrary starting point  $\mathbf{x}^{(0)}$ .
- Choose a positive definite  $n \times n$  matrix  $\mathbf{M}^{(0)}$ . In practice,  $\mathbf{M}^{(0)}$  is typically chosen as the identity matrix, (hence the first search direction is the steepest descent), i.e.:

$$\mathbf{M}^{(0)} = \mathbf{I}$$

- Compute the gradient vector  $\mathbf{g}^{(0)}$ .
- Stop if  $\|\mathbf{g}^{(0)}\| < \varepsilon_1$ . Otherwise, set  $k = 0$  and continue. Here  $\varepsilon_1$  represents an acceptable tolerance value, specified at the outset of the algorithm.
- **Step 2:** Calculate the search direction:

$$\mathbf{p}^{(k)} = -\mathbf{M}^{(k)} \mathbf{g}^{(k)}$$

- **Step 3:** Compute the new point  $\mathbf{x}^{(k+1)}$  by searching for the minimum along the direction  $\mathbf{p}^{(k)}$ . In other words, determine  $\alpha_{\min}^{(k)}$  to minimise  $F(\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)})$  and set:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_{\min}^{(k)} \mathbf{p}^{(k)}$$

- **Step 4:** Determine the vector:

$$\boldsymbol{\delta}^{(k+1)} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$$

- Stop if  $\|\boldsymbol{\delta}^{(k+1)}\| < \varepsilon_2$ . Otherwise, continue! Here  $\varepsilon_2$  represents an acceptable tolerance value, specified at the outset of the algorithm.
- Calculate the new gradient vector,  $\mathbf{g}^{(k+1)}$ . Stop if  $\|\mathbf{g}^{(k+1)}\| < \varepsilon_1$ . Otherwise determine the vector:  

$$\boldsymbol{\lambda}^{(k+1)} = \mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}$$
- **Step 5:** Compute  $\mathbf{M}^{(k+1)}$  as follows:

$$\mathbf{A}^{(k+1)} = \frac{\delta^{(k+1)} \delta^{(k+1)T}}{\delta^{(k+1)T} \lambda^{(k+1)}}$$

$$\mathbf{B}^{(k+1)} = - \left( \frac{M^{(k)} \lambda^{(k+1)} \lambda^{(k+1)T} M^{(k)}}{\lambda^{(k+1)T} M^{(k)} \lambda^{(k+1)}} \right)$$

$$\mathbf{M}^{(k+1)} = \mathbf{M}^{(k)} + \mathbf{A}^{(k+1)} + \mathbf{B}^{(k+1)}$$

- Set  $k = k+1$  and repeat from step 2.
- Previously, we stated that the Quasi-Newton method satisfies the following condition:

$$\mathbf{M}^{(i)} \lambda^{(i)} = \delta^{(i)}$$

- Let's see if the DFP method has this property.
- From the last equation in step 5 above, we can state that:

$$\mathbf{M}^{(k+1)} \boldsymbol{\lambda}^{(k+1)} = \mathbf{M}^{(k)} \boldsymbol{\lambda}^{(k+1)} + \mathbf{A}^{(k+1)} \boldsymbol{\lambda}^{(k+1)} + \mathbf{B}^{(k+1)} \boldsymbol{\lambda}^{(k+1)}$$

- But:  $\mathbf{A}^{(k+1)} \boldsymbol{\lambda}^{(k+1)} = \frac{\delta^{(k+1)} \delta^{(k+1)T} \boldsymbol{\lambda}^{(k+1)}}{\delta^{(k+1)T} \boldsymbol{\lambda}^{(k+1)}} = \delta^{(k+1)}$

and:  $\mathbf{B}^{(k+1)} \boldsymbol{\lambda}^{(k+1)} = - \left( \frac{M^{(k)} \boldsymbol{\lambda}^{(k+1)} \boldsymbol{\lambda}^{(k+1)T} M^{(k)} \boldsymbol{\lambda}^{(k+1)}}{\boldsymbol{\lambda}^{(k+1)T} M^{(k)} \boldsymbol{\lambda}^{(k+1)}} \right) = - \mathbf{M}^{(k)} \boldsymbol{\lambda}^{(k+1)}$

- Therefore:

$$\mathbf{M}^{(k+1)} \boldsymbol{\lambda}^{(k+1)} = \mathbf{M}^{(k)} \boldsymbol{\lambda}^{(k+1)} + \delta^{(k+1)} - \mathbf{M}^{(k)} \boldsymbol{\lambda}^{(k+1)}$$

$$\Rightarrow \mathbf{M}^{(k+1)} \boldsymbol{\lambda}^{(k+1)} = \delta^{(k+1)}$$

as required!

- The DFP algorithm has the following properties:
  - Only first derivatives are required and there is no need to solve equations.
  - Generates conjugate directions on a positive definite quadratic function and, therefore, can find the minimum of such functions in at most n steps.
  - Converges to a minimum on a well-behaved convex function.

### **Illustrative Example**

- Once again consider the quadratic function:

$$f(\mathbf{x}) = x_1 + x_2 + x_2^2 + 0.5x_1x_2 + 0.5x_1^2$$

- Starting with the point  $[2 \ 2]^T$ , use the DFP Quasi-Newton method to obtain the minimum of the above function.

- The gradient of the function is given by:

$$\mathbf{g} = \nabla f(\mathbf{x}) = \begin{bmatrix} 1 + 0.5x_2 + x_1 \\ 1 + 2x_2 + 0.5x_1 \end{bmatrix}$$

- Choosing  $\mathbf{M}^{(0)} = \mathbf{I}$ , the first search direction is given by:

$$\mathbf{p}^{(0)} = -\mathbf{M}^{(0)}\mathbf{g}^{(0)} = -\begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

- Now,  $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(0)}$ , giving:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 2 - 4\alpha \\ 2 - 6\alpha \end{bmatrix}$$

- Now we find  $\alpha$  that minimises  $f(\alpha)$ :

$$f(\alpha) = 56\alpha^2 - 52\alpha + 12$$

- Evaluating  $\frac{df}{d\alpha} = 0$  gives  $\alpha_{\min} = 0.4643$  and  $f(\alpha) = -0.07143$ .
- Hence:  $\mathbf{x}^{(1)} = \begin{bmatrix} 0.1428 \\ -0.7858 \end{bmatrix}$
- Now we need to calculate  $\mathbf{M}^{(1)}$  :

$$\mathbf{A}^{(1)} = \frac{\delta^{(1)}\delta^{(1)T}}{\delta^{(1)T}\lambda^{(1)}} = \begin{bmatrix} 0.1429 & 0.2143 \\ 0.2143 & 0.3214 \end{bmatrix}$$

$$\mathbf{B}^{(1)} = -\left(\frac{M^{(0)}\lambda^{(1)}\lambda^{(1)T}M^{(0)}}{\lambda^{(1)T}M^{(0)}\lambda^{(1)}}\right) = -\begin{bmatrix} 0.2 & 0.4 \\ 0.4 & 0.8 \end{bmatrix}$$

$$\mathbf{M}^{(1)} = \mathbf{M}^{(0)} + \mathbf{A}^{(1)} + \mathbf{B}^{(1)} = \begin{bmatrix} 0.9429 & -0.1857 \\ -0.1857 & 0.5214 \end{bmatrix}$$

- Hence, our next search direction is given by:

$$\mathbf{p}^{(1)} = -\mathbf{M}^{(1)}\mathbf{g}^{(1)} = -\begin{bmatrix} 0.9429 & -0.1857 \\ -0.1857 & 0.5214 \end{bmatrix} \begin{bmatrix} 0.7499 \\ -0.5002 \end{bmatrix} = \begin{bmatrix} -0.8 \\ 0.4 \end{bmatrix}$$

- Our next point is now  $\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \alpha \mathbf{p}^{(1)}$ , giving:

$$\mathbf{x}^{(2)} = \begin{bmatrix} 0.1428 - 0.8\alpha \\ -0.7858 + 0.4\alpha \end{bmatrix}$$

- Again we find  $\alpha$  that minimises  $f(\alpha)$ :  $f(\alpha) = 0.32\alpha^2 - 0.8\alpha - 0.0714$
- Evaluating  $\frac{df}{d\alpha} = 0$  gives  $\alpha_{\min} = 1.25$  and  $f(\alpha) = -0.5714$ .
- Hence:  $\mathbf{x}^{(2)} = \begin{bmatrix} -0.8572 \\ -0.2858 \end{bmatrix}$
- As we have seen before, this is the actual minimum.

## 5.2.2 The BFGS Algorithm

- The **BFGS (Broyden, Fletcher, Goldfarb, Shanno) algorithm** is very similar to the DFP algorithm, but provides an alternative formula for calculating  $\mathbf{M}^{(k+1)}$ .
- The BFGS algorithm follows exactly the same steps as outlined for the DFP, with the exception of Step 5, where the formula for calculating  $\mathbf{M}^{(k+1)}$  is as follows:

$$\mathbf{A}^{(k+1)} = \left( 1 + \frac{\boldsymbol{\lambda}^{(k+1)T} \mathbf{M}^{(k)} \boldsymbol{\lambda}^{(k+1)}}{\boldsymbol{\delta}^{(k+1)T} \boldsymbol{\lambda}^{(k+1)}} \right) \left( \frac{\boldsymbol{\delta}^{(k+1)} \boldsymbol{\delta}^{(k+1)T}}{\boldsymbol{\delta}^{(k+1)T} \boldsymbol{\lambda}^{(k+1)}} \right)$$

$$\mathbf{B}^{(k+1)} = - \left( \frac{\boldsymbol{\delta}^{(k+1)} \boldsymbol{\lambda}^{(k+1)T} \mathbf{M}^{(k)} + \mathbf{M}^{(k)} \boldsymbol{\lambda}^{(k+1)} \boldsymbol{\delta}^{(k+1)T}}{\boldsymbol{\delta}^{(k+1)T} \boldsymbol{\lambda}^{(k+1)}} \right)$$

$$\mathbf{M}^{(k+1)} = \mathbf{M}^{(k)} + \mathbf{A}^{(k+1)} + \mathbf{B}^{(k+1)}$$

- When both methods use **exact line searches** (i.e. we calculate the **exact minimum** along each search direction) then both BFGS and DFP give exactly the same sequence of points.
- However, in practice, exact line searches are not often used. Instead, it is better to find the search line minimum very approximately and use this as the starting point for the next iteration. This is referred to as an **inexact line search**.
- While this will likely require more iterations of the algorithm, overall it actually requires fewer function evaluations (which, as mentioned several times at this stage, is a good thing!).
- The reason for this is because when we are far away from the actual local minimum, then the quality of the search direction may be quite poor. Therefore an approximate line search minimum can be just as good as the actual minimum and the resulting next search direction can be just as good. The difference here is that this information is achieved with significantly fewer function evaluations!
- On the other hand, when we are close to the actual local minimum, then the final convergence is generally relatively fast, even with an inexact line search.
- When inexact line searches are used, the BFGS and DFP algorithms give different results.

- In practice, it has been found that the BFGS gives better results with inexact line searches. Thus, it is the preferred algorithm and is typically used in most optimisation software packages today.

### 5.3 Wolfe's Conditions & Inexact Line Searches

- Recall that for an exact line search we want to obtain  $\alpha$  to minimise  $F(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)})$ .
- The key issue with an inexact line search is determining a suitable criteria for stopping. In other words, we want to find a suitable value for  $\alpha$  (not necessarily the minimum) that will allow us to continue searching in a new direction.
- In general, and in order to guarantee convergence to a local minimum, the inexact line search terminates once we obtain a value for  $\alpha$  that satisfies **Wolfe's conditions**.
- **Condition I** requires that *the error reduction must not be too small* and is tested by the expression:

$$F(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}) \leq F(\mathbf{x}^{(k)}) + \varepsilon_1 \alpha \mathbf{g}^{(k)T} \mathbf{p}^{(k)}$$

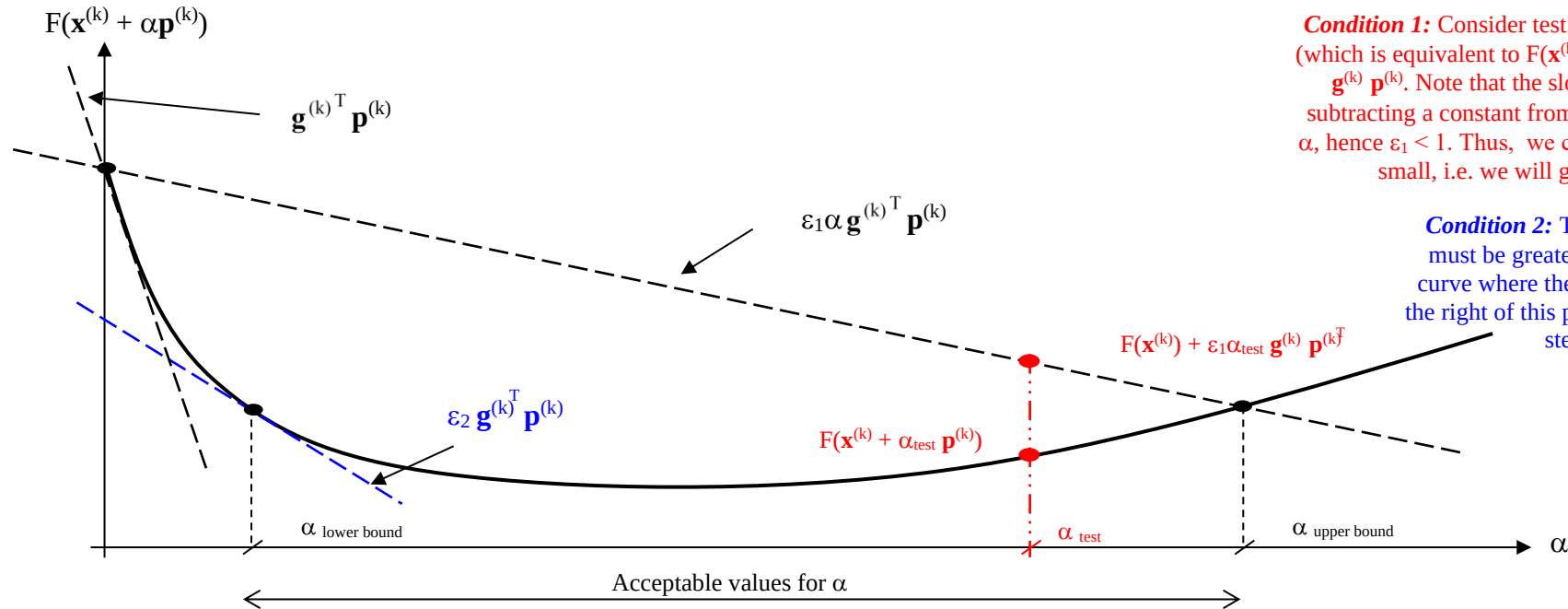
where  $\varepsilon_1$  is a small positive constant,  $\varepsilon_1 < 1$ .

- **Condition II** requires that *the step size must not be too small* and is checked using:

$$\mathbf{g}^{(k+1)T} \mathbf{p}^{(k)} \geq \varepsilon_2 \mathbf{g}^{(k)T} \mathbf{p}^{(k)}$$

where  $\varepsilon_2$  is a small positive constant such that  $\varepsilon_1 < \varepsilon_2 < 1$ .

- These conditions impose an upper and lower bound on the acceptable step size, as illustrated in the following diagram.
- In practice, condition II is not checked as it requires new gradient information at each trial step. Instead, inexact line searches are designed so that excessively small steps are avoided.



**Condition 1:** Consider test point  $\alpha_{\text{test}}$  and  $\epsilon_1 = 0.5$ . Then  $F(\mathbf{x}^{(k)} + \alpha_{\text{test}} \mathbf{p}^{(k)})$  (which is equivalent to  $F(\mathbf{x}^{(k+1)})$ ) must be less or equal to  $\leq F(\mathbf{x}^{(k)}) + 0.5\alpha_{\text{test}} \mathbf{g}^{(k)T} \mathbf{p}^{(k)}$ . Note that the slope is negative, therefore we are effectively subtracting a constant from  $F(\mathbf{x}^{(k)})$ . If  $\epsilon_1 = 1$ , we couldn't find a value for  $\alpha$ , hence  $\epsilon_1 < 1$ . Thus, we can ensure that the error reduction won't be too small, i.e. we will get a decent reduction in the value of  $F$ .

**Condition 2:** The slope of the function at the new point must be greater or equal to  $\epsilon_2 \mathbf{g}^{(k)T} \mathbf{p}^{(k)}$ . The point on the curve where the two are the same is shown. Any value to the right of this point is acceptable. Thus we ensure that the step size taken is not too small.

Bounds on  $\alpha$  imposed by Wolfe's conditions

## 6. The Least Squares Problem

- All the optimisation techniques we have studied to date can be used to minimise (or maximise) fairly general functions.
- However, in certain problems, the efficiency of such techniques can be improved upon by using special features that may exist in the problem.
- Here, we are going to consider one such problem, which is referred to as the **least squares problem**.
- The least squares problem is one that involves the minimisation of a function that is formed by a sum of squares of other functions and is a common problem in nonlinear curve and model fitting as well as nonlinear equation solving and some design problems.
- The least squares minimisation problem takes the form:

$$\min F(\mathbf{x}) = \sum_{i=1}^m f_i^2(x) = f_1^2(x) + f_2^2(x) + \dots + f_m^2(x)$$

where the vector  $\mathbf{x}$  has  $n$  components.

- **Example problem** – Let's say we have developed a model for some real process that depends on two parameters,  $x_1$  and  $x_2$ , and is given by the function:

$$y_m(t) = x_1 \exp(x_2 t)$$

- Let's also say that we have actual real data from the process that is expressed as a set of  $m$  data points  $(t_i, y_i)$ . Then our least squares minimisation problem becomes one of solving:

$$\min F(\mathbf{x}) = \sum_{i=1}^m [y_i - (x_1 \exp(x_2 t_i))]^2$$

- This problem can be solved using any of the methods that we have studied so far. However, when  $m \geq n$ , we can devise better methods to solve the problem.

## 6.1 The Gauss-Newton Method

- Define the vector  $\mathbf{f}$  as:

$$\mathbf{f} = [f_1(\mathbf{x}) \quad f_2(\mathbf{x}) \quad \dots \quad f_m(\mathbf{x})]^T$$

- Hence, the least squares minimisation problem can be expressed as:

$$\min F(\mathbf{x}) = \sum_{i=1}^m f_i^2(\mathbf{x}) = \mathbf{f}^T \mathbf{f} = f_1^2(\mathbf{x}) + f_2^2(\mathbf{x}) + \dots + f_m^2(\mathbf{x})$$

- Differentiating  $F(\mathbf{x})$  gives:

$$\begin{aligned} \frac{\partial F}{\partial x_1} &= 2f_1 \frac{\partial f_1}{\partial x_1} + 2f_2 \frac{\partial f_2}{\partial x_1} + \dots + 2f_m \frac{\partial f_m}{\partial x_1} \\ &\vdots = \vdots \\ \frac{\partial F}{\partial x_n} &= 2f_1 \frac{\partial f_1}{\partial x_n} + 2f_2 \frac{\partial f_2}{\partial x_n} + \dots + 2f_m \frac{\partial f_m}{\partial x_n} \end{aligned}$$

- In matrix form, this is expressed as:

$$\begin{bmatrix} \partial F / \partial x_1 \\ \vdots \\ \partial F / \partial x_n \end{bmatrix} = 2 \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_2}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_1} \\ \vdots & \vdots & \dots & \vdots \\ \frac{\partial f_1}{\partial x_n} & \frac{\partial f_2}{\partial x_n} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \begin{bmatrix} f_1 \\ \vdots \\ f_m \end{bmatrix}$$

or

$$\nabla F = 2 \mathbf{J}^T \mathbf{f}$$

- The  $m \times n$  matrix  $\mathbf{J}$  is referred to as the **Jacobian matrix** of  $F$  and has terms defined by:

$$J_{ij} = \frac{\partial f_i}{\partial x_j} \quad \text{for } i = 1, 2, \dots, m \quad \text{and } j = 1, 2, \dots, n$$

i.e.:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \vdots & \dots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

- Differentiating a second time gives:

$$\frac{\partial^2 F}{\partial x_1^2} = 2 \left( \frac{\partial f_1}{\partial x_1} \cdot \frac{\partial f_1}{\partial x_1} + f_1 \frac{\partial^2 f_1}{\partial x_1^2} \right) + 2 \left( \frac{\partial f_2}{\partial x_1} \cdot \frac{\partial f_2}{\partial x_1} + f_2 \frac{\partial^2 f_2}{\partial x_1^2} \right) + \dots + 2 \left( \frac{\partial f_m}{\partial x_1} \cdot \frac{\partial f_m}{\partial x_1} + f_m \frac{\partial^2 f_m}{\partial x_1^2} \right)$$

$$\frac{\partial^2 F}{\partial x_2 \partial x_1} = 2 \left( \frac{\partial f_1}{\partial x_2} \cdot \frac{\partial f_1}{\partial x_1} + f_1 \frac{\partial^2 f_1}{\partial x_2 \partial x_1} \right) + 2 \left( \frac{\partial f_2}{\partial x_2} \cdot \frac{\partial f_2}{\partial x_1} + f_2 \frac{\partial^2 f_2}{\partial x_2 \partial x_1} \right) + \dots + 2 \left( \frac{\partial f_m}{\partial x_2} \cdot \frac{\partial f_m}{\partial x_1} + f_m \frac{\partial^2 f_m}{\partial x_2 \partial x_1} \right)$$

... and so on for all the other elements of the hessian matrix.

- Dividing the above terms into two groups, one containing the product of first derivative terms and the other containing the second derivatives, we can express the full hessian matrix as follows:

$$\nabla^2 F = 2(J^T J + \sum_{i=1}^m f_i \nabla^2 f_i)$$

- Note that  $\nabla^2 f_i$  is the hessian matrix of the function  $f_i$ .
- At this stage the Gauss-Newton method makes the assumption that the second term in the above expression is negligible in comparison with the first.
- This is often justified if:
  - 1) the absolute values of  $f_i$  are small (as they often are) at or near a minimum of  $F$  and/or
  - 2) the values of  $\nabla^2 f_i$  are small, as is true in many practical cases.
- Hence:  $\nabla^2 F \approx 2J^T J$
- Note that this expression involves only first derivatives!
- Recall that the **Newton's search algorithm** obtains the search direction as:

$$\mathbf{p} = -\mathbf{H}^{-1}\mathbf{g} \quad \text{or} \quad \mathbf{p} = -(\nabla^2 F)^{-1}\nabla F$$

and hence the update is given by:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha ( (\nabla^2 F)^{-1} \nabla F )^{(k)}$$

- Using the Gauss-Newton assumption and expressing  $\nabla F = 2 \mathbf{J}^T \mathbf{f}$ , we obtain the search direction for the **Gauss-Newton algorithm** as:

$$\mathbf{p}^{(k)} = - (J^{(k)T} \mathbf{J}^{(k)})^{-1} J^{(k)T} \mathbf{f}^{(k)}$$

- Note that the constant is left out of the expression for the search direction as it is simply a scaling factor and therefore does not affect the all important direction of the line search.
- Note also that we tend to avoid calculating the inverse of a matrix directly, as it is both very difficult and computationally intensive for large matrices.

- Instead, we employ a technique such as Gauss elimination, Cramer's Rule, etc to solve the set of equations given by:

$$J^{(k)T} \mathbf{J}^{(k)} \mathbf{p}^{(k)} = - J^{(k)T} \mathbf{f}^{(k)}$$

### The Gauss-Newton Algorithm

- Select a starting point  $\mathbf{x}^{(0)}$  and evaluate  $F(\mathbf{x}^{(0)})$ .
- Starting with  $k = 0$ , solve the following set of equations to get  $\mathbf{p}^{(k)}$ :

$$J^{(k)T} \mathbf{J}^{(k)} \mathbf{p}^{(k)} = - J^{(k)T} \mathbf{f}^{(k)}$$

- Using a line search (need not be exact!) procedure, solve:

$$\min_{\alpha} F(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}) \quad \text{to give } \alpha^{(k)}$$

- Check for convergence. If  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|$  is sufficiently small or if  $F(\mathbf{x}^{(k+1)}) - F(\mathbf{x}^{(k)})$  is sufficiently small, then stop. Otherwise, set  $k = k+1$  and repeat the process.

### Illustrative Example

- Consider the simple quadratic function given by:  $y = f(t) = t^2 + 3t - 2$
- Three points, in the form  $(t, y)$ , on this function are  $(2, 8)$ ,  $(-1, -4)$  and  $(0, -2)$ .
- Okay, now consider the following problem. Let's say that we are given the following data points, where  $t$  and  $y$  represents the input and output respectively:

| $t$ | $y$ |
|-----|-----|
| 2   | 8   |
| -1  | -4  |
| 0   | -2  |

- We want to fit a quadratic function of the form  $y_m = t^2 + x_1 t + x_2$  to this data, where  $\mathbf{x} = [x_1 \ x_2]^T$  represents the unknown parameters.
- *Note: firstly we know what the answer to this problem is from above and, secondly, this is a trivial problem that can quickly and easily be solved by other methods. However, here we are simply using this example to illustrate the Gauss-Newton approach.*
- From the data we can write down three equations:

$$\begin{array}{lll} 8 = 4 + 2x_1 + x_2 & \Rightarrow & -4 + 2x_1 + x_2 = 0 \dots f_1(\mathbf{x}) \\ -4 = 1 - x_1 + x_2 & \Rightarrow & 5 - x_1 + x_2 = 0 \dots f_2(\mathbf{x}) \\ -2 = 0 + 0 + x_2 & \Rightarrow & 2 + x_2 = 0 \dots f_3(\mathbf{x}) \end{array}$$

- The least squares problem takes the form:

$$F(\mathbf{x}) = \sum_{i=1}^3 f_i^2(x) = f_1^2(\mathbf{x}) + f_2^2(\mathbf{x}) + f_3^2(\mathbf{x})$$

- Here, we want to minimise  $F(\mathbf{x})$  with respect to  $\mathbf{x}$  and hence we get the best possible fit between our model and the actual data. In this example, we should be able to get the ideal fit and therefore have no error, i.e.  $F(\mathbf{x}) = 0$ .
- Now, given the arbitrary starting value of  $\mathbf{x}^{(0)} = [1 \ 1]^T$ , determine the value of  $\mathbf{x}$  that minimises  $F(\mathbf{x})$ .
- The Jacobian matrix for this problem is:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ -1 & 1 \\ 0 & 1 \end{bmatrix}$$

- In this case, the Jacobian matrix is always the same, i.e. it is independent of the point  $\mathbf{x}$  at every iteration.
- The first search direction, according to the Gauss-Newton method is obtained by solving the set of equations:

$$J^{(0)T} \mathbf{J}^{(0)} \mathbf{p}^{(0)} = -J^{(0)T} \mathbf{f}^{(0)}$$

- Here, the vector  $\mathbf{f}$  is given by  $\mathbf{f} = [f_1(\mathbf{x}) \ f_2(\mathbf{x}) \ f_3(\mathbf{x})]^T$  and  $\mathbf{f}^{(0)}$  simply means we determine the value of this vector for  $\mathbf{x}^{(0)}$ .
- Carrying out the necessary calculations, the above set of equations gives us:

$$\begin{bmatrix} 5 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} 7 \\ -7 \end{bmatrix}$$

- Here  $\mathbf{p}^{(0)} = [p_1 \ p_2]^T$ . Solving (using Gauss Elimination, etc...) gives the search direction:

$$\mathbf{p}^{(0)} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$$

- The next estimate of  $\mathbf{x}$  is given by  $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(0)}$ . Carrying out the usual process, we obtain  $\alpha_{\min} = 1$  and therefore:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$$

- The function output at this point is  $F(\mathbf{x}^{(1)}) = 0$ , i.e. the best possible fit. Hence our quadratic model takes the form  $y_m = t^2 + 3t - 2$ , as expected!
- Compared with Newton's algorithm, the Gauss-Newton method has the advantages:
  - 1) No second derivatives are required to be calculated
  - 2)  $\mathbf{J}^T \mathbf{J}$  is positive semi-definite, which implies that the search directions are never uphill).

### Simple Proof that $\mathbf{J}^T \mathbf{J}$ is positive semi-definite

- $\mathbf{J}^T \mathbf{J}$  is positive semi-definite if, for all non-zero vectors  $\mathbf{z}$ :

$$\mathbf{z}^T \mathbf{J}^T \mathbf{J} \mathbf{z} \geq 0$$

- If we let  $\mathbf{y} = \mathbf{J}\mathbf{z}$ , then the expression  $\mathbf{z}^T \mathbf{J}^T \mathbf{J} \mathbf{z}$  can be written as:

$\mathbf{y}^T \mathbf{y} = \sum y_i^2$ , which is clearly always  $\geq 0$

- Note that although  $\mathbf{z} \neq \mathbf{0}$ , it is possible for  $\mathbf{y} = \mathbf{0}$  if  $\mathbf{J}$  is singular (i.e. non-invertible!).
- The disadvantages of the Gauss-Newton method are:
  - 1) May not converge if the Gauss-Newton assumptions are not valid (but this is rare!)
  - 2) Unfortunately, a set of equations have to be solved on each iteration, i.e.:

$$(\mathbf{J}^T \mathbf{J})^{(k)} \mathbf{p}^{(k)} = - (\mathbf{J}^T \mathbf{f})^{(k)} \quad (\text{solve to find } \mathbf{p}^{(k)})$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}$$

- A key issue with the Gauss-Newton method is that the search direction  $\mathbf{p}^{(k)}$  is not defined if  $(\mathbf{J}^T \mathbf{J})^{(k)}$  is singular or  $\mathbf{p}^{(k)}$  can be inaccurate if  $(\mathbf{J}^T \mathbf{J})^{(k)}$  is ill-conditioned (nearly singular!).

- To avoid this problem, we can add a matrix to  $\mathbf{J}^T \mathbf{J}$  to make it non-singular. This is the motivation behind the **Levenberg-Marquart algorithm**.

## 6.2 The Levenberg-Marquart algorithm

- Consider the following matrix:

$$\mathbf{J}^T \mathbf{J} + \mu \mathbf{I}$$

where  $\mu$  is a scalar parameter such that  $\mu > 0$ .

- Thus, we are simply adding the value  $\mu$  to the main diagonal of  $\mathbf{J}^T \mathbf{J}$ .
- The eigenvalues  $\lambda$  and eigenvectors  $\mathbf{v}$  of this matrix satisfy the property:

$$(\mathbf{J}^T \mathbf{J} + \mu \mathbf{I}) \mathbf{v} = \lambda \mathbf{v}$$

- Therefore:  $\mathbf{J}^T \mathbf{J} \mathbf{v} = (\lambda - \mu) \mathbf{v} = \lambda' \mathbf{v}$
- Now,  $\lambda' (= \lambda - \mu)$  is an eigenvalue of  $\mathbf{J}^T \mathbf{J}$  and  $\mathbf{v}$  is an eigenvector.
- Hence, adding  $\mu \mathbf{I}$  to  $\mathbf{J}^T \mathbf{J}$  leaves the eigenvectors unchanged but adds  $\mu$  to the eigenvalues ( $\lambda = \lambda' + \mu$ ).
- Thus the matrix  $\mathbf{J}^T \mathbf{J} + \mu \mathbf{I}$  has minimum eigenvalues  $= \mu > 0$  and is therefore non-singular.
- Note, a singular matrix is non-invertible, i.e. its determinant  $= 0$ . It also implies that one of its eigenvalues is 0. Thus if the matrix  $(\mathbf{J}^T \mathbf{J} + \mu \mathbf{I})$  has eigenvalues  $> 0$ , then it is non-singular!
- Hence the following set of equations can always be solved:

$$(\mathbf{J}^T \mathbf{J} + \mu \mathbf{I})^{(k)} \mathbf{p}^{(k)} = - (\mathbf{J}^T \mathbf{f})^{(k)}$$

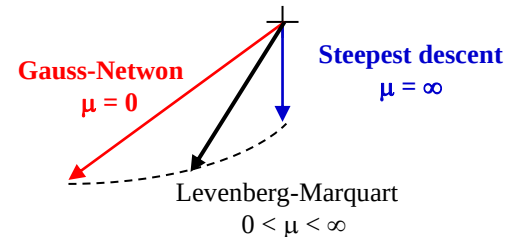
- The Levenberg-Marquart algorithm uses this approach to compute the search directions.

## Choosing a value for $\mu$

- If  $\mu$  is very large then the search direction  $\mathbf{p}$  is:

$$\mathbf{p} \approx -\frac{1}{\mu} J^T \mathbf{f} = -\frac{1}{2\mu} \nabla F \quad (\nabla F = 2 J^T \mathbf{f})$$

- Thus, the search direction  $\mathbf{p}$  tends towards the steepest descent and the  $\|\mathbf{p}\|$  is very small.
- On the other hand, if  $\mu$  is small then  $\mathbf{p}$  tends towards the Gauss-Newton search direction:



- Generally,  $\mu$  is set to zero unless  $\mathbf{J}^T \mathbf{J}$  is singular, in which case it is set to a small value, typically:

$$\mu = 10^{-4} \sum_{i=1}^n (J^T J)_{ii} \quad (\text{i.e. the sum of the diagonal terms})$$

## Condition Number

- The **condition number** of  $\mathbf{J}^T \mathbf{J}$  is defined as:

$$C = \frac{\lambda_{\max}}{\lambda_{\min}}$$

- $C = \infty$  for a singular matrix (i.e. one of the eigenvalues is zero, hence  $\lambda_{\min} = 0$ ).
- Generally, if  $C \geq 10^4$  (for example) then the matrix  $\mathbf{J}^T \mathbf{J}$  is said to be ill-conditioned and the Gauss-Newton search directions may be inaccurate.
- Adding  $\mu \mathbf{I}$  improves the conditioning:

$$\frac{\lambda_{\max} + \mu}{\lambda_{\min} + \mu} < \frac{\lambda_{\max}}{\lambda_{\min}} \quad \text{for } \mu > 0$$

## 6.3 Sensitivity Analysis

- Consider the quadratic function:

$$F(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x},$$

where  $\mathbf{H} = \nabla^2 F$  is the positive-definite hessian matrix.

- If  $F(\mathbf{x})$  is a sum of squares, then using the Gauss-Newton assumption:

$$\mathbf{H} \approx 2\mathbf{J}^T \mathbf{J}$$

- For the  $i^{\text{th}}$  eigenvalue  $\lambda_i$  and eigenvector  $\mathbf{v}_i$  of this matrix, we know that:

$$\mathbf{J}^T \mathbf{J} \mathbf{v}_i = \lambda_i \mathbf{v}_i \quad \text{for } i = 1, 2, \dots, n$$

- Thus, we can write:

$$\mathbf{J}^T \mathbf{J} \mathbf{Q} = \mathbf{Q} \mathbf{D}$$

$$\text{where: } \mathbf{Q} = [\mathbf{v}_1 \ \mathbf{v}_2 \ \dots \ \mathbf{v}_n] \quad \text{and} \quad \mathbf{D} = \begin{bmatrix} \lambda_1 & & & 0 \\ & \lambda_2 & & \\ & & \ddots & \\ 0 & & & \lambda_n \end{bmatrix}$$

- Multiplying both sides by  $\mathbf{Q}^T$  gives:

$$\mathbf{J}^T \mathbf{J} \mathbf{Q} \mathbf{Q}^T = \mathbf{Q} \mathbf{D} \mathbf{Q}^T$$

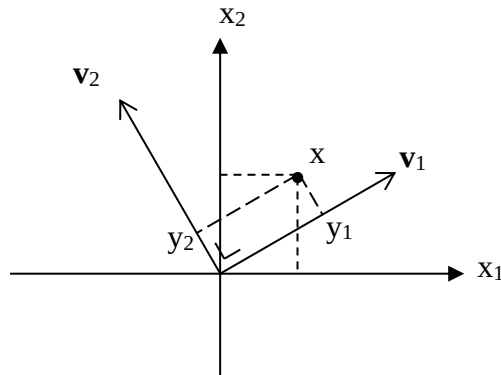
- The eigenvectors of a real symmetric matrix ( $\mathbf{J}^T \mathbf{J}$ ) are mutually orthogonal, and by normalising each eigenvector,  $\mathbf{Q}$  becomes an orthonormal matrix, i.e.:

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I} \quad , \text{ i.e. } \mathbf{Q}^{-1} = \mathbf{Q}^T$$

- Therefore we obtain:

$$\mathbf{J}^T \mathbf{J} = \mathbf{Q} \mathbf{D} \mathbf{Q}^T$$

- Consider the transformation  $\mathbf{x} = \mathbf{Q}\mathbf{y}$ . Here,  $\mathbf{y}$  gives the position of  $\mathbf{x}$  in the co-ordinate system formed by the eigenvectors of  $\mathbf{J}^T \mathbf{J}$ , as illustrated in the following 2-d example:



$$\mathbf{Q} = \begin{bmatrix} v_{11} & v_{12} \\ v_{21} & v_{22} \end{bmatrix}$$

$$\begin{array}{cc} \uparrow & \uparrow \\ \mathbf{v}_1 & \mathbf{v}_2 \end{array}$$

$$\mathbf{x} = y_1 \mathbf{v}_1 + y_2 \mathbf{v}_2$$

- Note that:

$$\| \mathbf{x} \| = \sqrt{\mathbf{x}^T \mathbf{x}} = \sqrt{\mathbf{y}^T \mathbf{Q}^T \mathbf{Q} \mathbf{y}} = \sqrt{\mathbf{y}^T \mathbf{y}} = \| \mathbf{y} \|$$

- Hence, the transformation leaves the lengths unchanged.

- Now:

$$F(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x} = \mathbf{x}^T \mathbf{J}^T \mathbf{J} \mathbf{x} = \mathbf{x}^T \mathbf{Q} \mathbf{D} \mathbf{Q}^T \mathbf{x}$$

- Substituting for  $\mathbf{x}$  gives:

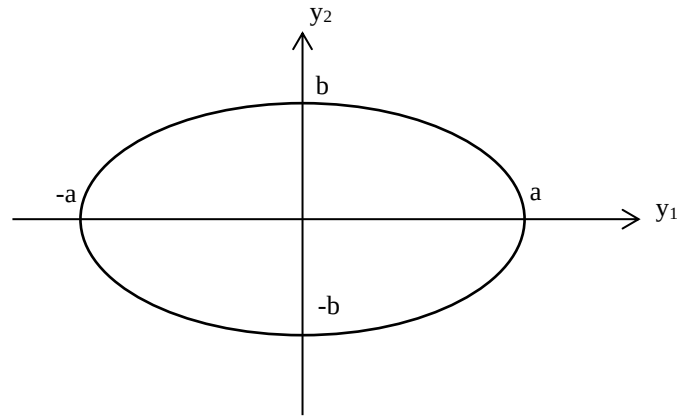
$$\begin{aligned} F(\mathbf{y}) &= \mathbf{y}^T \mathbf{Q}^T \mathbf{Q} \mathbf{D} \mathbf{Q}^T \mathbf{Q} \mathbf{y} \\ &= \mathbf{y}^T \mathbf{D} \mathbf{y} \\ &= \sum_{i=1}^n \lambda_i y_i^2 \end{aligned}$$

- A contour of  $F(\mathbf{y})$  has an equation of the form  $\sum_{i=1}^n \lambda_i y_i^2 = k$ , where  $k$  is a constant.

- This results in an ellipse in n-space, centred at the origin with its axes parallel to the co-ordinate axes (i.e. the eigenvectors of  $\mathbf{J}^T \mathbf{J}$ ) and the length of its axes inversely proportional to  $\sqrt{\lambda_i}$ .
- For example, consider the case for  $n = 2$ . Hence:

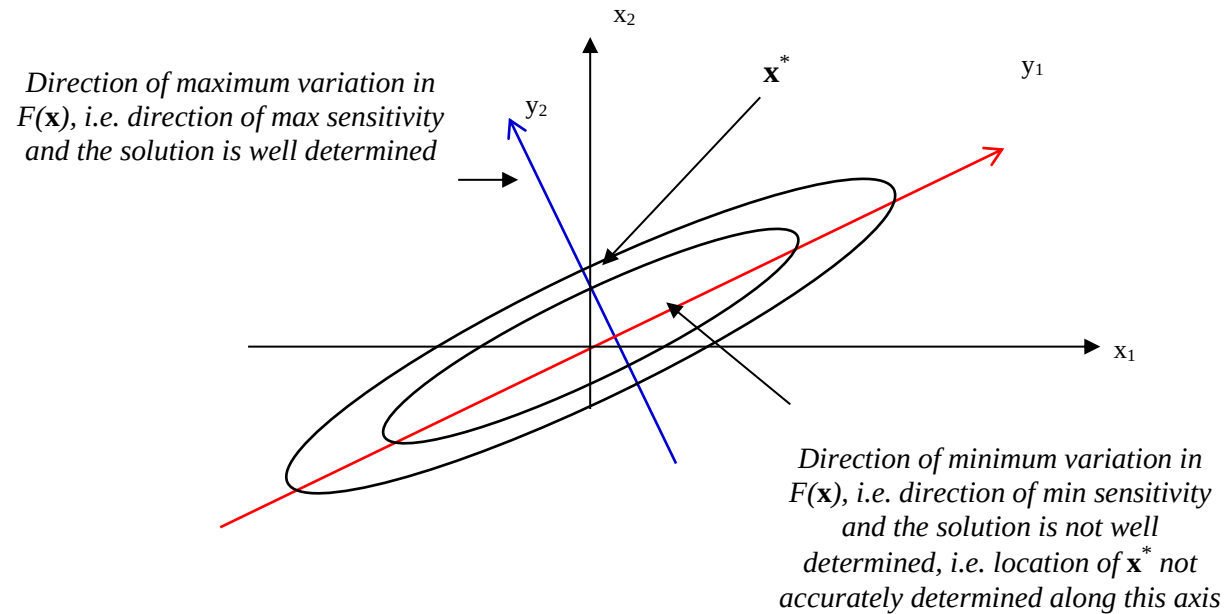
$$\lambda_1 y_1^2 + \lambda_2 y_2^2 = k$$

- Compare this with the standard equation of an ellipse:  $\frac{y_1^2}{a^2} + \frac{y_2^2}{b^2} = 1$
- Thus:  $\frac{1}{a^2} \equiv \frac{\lambda_1}{k}$  and  $\frac{1}{b^2} \equiv \frac{\lambda_2}{k}$
- Hence:  $a = \sqrt{\frac{k}{\lambda_1}}$  and  $b = \sqrt{\frac{k}{\lambda_2}}$



- Therefore the ratio of lengths of axes of the contour ellipses is:  $\frac{a}{b} = \sqrt{\frac{\lambda_2}{\lambda_1}} = \sqrt{C}$

- Thus, if  $C$  is large (for example  $C = 10^4$ ), then the elliptical contours will become very elongated and the problem is ill-conditioned:



- It is important to know whether a function is ill-conditioned or not, in order to know if the resulting local minimum estimate is accurate.
- Therefore, a sensitivity analysis needs to be carried out. This involves computing the eigenvalues and the eigenvector of the matrix  $\mathbf{J}^T \mathbf{J}$ .
- If  $\lambda_{\max} \gg \lambda_{\min}$  then it is possible that the estimated value of  $\mathbf{x}^*$  may be significantly in error, and the correct value may be of the form:

$$\mathbf{x}^* = \mathbf{x}_{\text{estimate}}^* + \varepsilon \mathbf{v}_{\min}$$

where  $\mathbf{v}_{\min}$  is the eigenvector associated with the eigenvalue  $\lambda_{\min}$ .

- The value of  $\varepsilon$  may not be small and, if there are several small eigenvalues (in an n-dimensional problem) then:

$$\mathbf{x}^* = \mathbf{x}_{\text{estimate}}^* + \sum_{i \in \text{small}} \varepsilon_i \mathbf{v}_i$$

## 6.4 Solving Nonlinear Equations

- Consider the problem of finding  $\mathbf{x} \in \mathcal{R}^n$  such that:

$$f_i(\mathbf{x}) = 0 \quad \text{for } i = 1, 2, \dots, n$$

where  $f_i(\mathbf{x})$  are nonlinear functions.

- This is the n-dimensional nonlinear equation problem and can be solved by minimising the sum of squares function  $F(\mathbf{x})$ :

$$F(\mathbf{x}) = \sum_{i=1}^n f_i^2(x)$$

- This function corresponds to a least squares problem with  $m = n$  and leads to a simplification of the Gauss-Newton algorithm. Applying Gauss-Newton, we obtain the search direction:

$$\mathbf{p} = -(\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \mathbf{f}$$

- However, in this case the  $\mathbf{J}$  matrix is now square (i.e.  $n \times n$ ) and hence:

$$\mathbf{p} = - \mathbf{J}^{-1} \mathbf{J}^T \mathbf{f} = - \mathbf{J}^{-1} \mathbf{f}$$

- Hence the update for Gauss-Newton becomes:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha (\mathbf{J}^{-1} \mathbf{f})^{(k)}$$

- In this particular form, the Gauss-Newton is known as the Newton-Raphson method.
- **Illustrative Example:** Consider the following nonlinear functions:

$$f_1(\mathbf{x}) = \sin(10x_1) + \sin(20x_2) = 0.3$$

$$f_2(\mathbf{x}) = e^{2x_1} - e^{-x_2} = 10$$

where  $\mathbf{x} \in \mathbb{R}^2$ .

- We can rewrite these functions as:

$$f_1(\mathbf{x}) = \sin(10x_1) + \sin(20x_2) - 0.3 = 0$$

$$f_2(\mathbf{x}) = e^{2x_1} - e^{-x_2} - 10 = 0$$

- The Jacobian matrix is calculated as:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 10\cos(10x_1) & 20\cos(20x_2) \\ 2e^{2x_1} & e^{-x_2} \end{bmatrix}$$

- Now, given an initial starting point we can apply the Newton-Raphson method to obtain the solution to the above equations.
- For example, if  $\mathbf{x}^{(0)} = [0 \ 0]^T$ , then:  $\mathbf{J}^{(0)} = \begin{bmatrix} 10 & 20 \\ 2 & 1 \end{bmatrix}$ ,  $\mathbf{f}^{(0)} = \begin{bmatrix} -0.3 \\ -10 \end{bmatrix}$

- Solving  $\mathbf{J}^{(0)} \mathbf{p}^{(0)} = -\mathbf{f}^{(0)}$  gives:  $\mathbf{p}^{(0)} = \begin{bmatrix} -6.6567 \\ 3.3133 \end{bmatrix}$
- The remainder of the problem requires us to find:  $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(0)}$
- In order to find  $\alpha$ , we need to minimise  $F(\alpha)$  w.r.t.  $\alpha$  where  $F(\alpha)$  is expressed in least squares form, i.e.:

$$F(\alpha) = \sum_{i=1}^n f_i^2(\alpha)$$

- We can use a line search technique to solve this minimisation problem, if need be.
- The better the solution we obtain, the closer  $F(\alpha)$  gets to zero.

- **Another Illustrative Example:** Let's say that we are given the following data points, where  $t$  and  $y$  represents the input and output respectively:

| $t$ | $y$ |
|-----|-----|
| 2   | 8   |
| -1  | -4  |

- We want to fit a quadratic function of the form  $y_m = t^2 + x_1t + x_2$  to this data, where  $\mathbf{x} = [x_1 \ x_2]^T$  represents the unknown parameters.
- From the data we can write down two equations:

$$\begin{aligned} 8 &= 4 + 2x_1 + x_2 \Rightarrow -4 + 2x_1 + x_2 = 0 \dots f_1(\mathbf{x}) \\ -4 &= 1 - x_1 + x_2 \Rightarrow 5 - x_1 + x_2 = 0 \dots f_2(\mathbf{x}) \end{aligned}$$

- The least squares problem takes the form:

$$F(\mathbf{x}) = \sum_{i=1}^2 f_i^2(x) = f_1^2(\mathbf{x}) + f_2^2(\mathbf{x})$$

- Now, given the arbitrary starting value of  $\mathbf{x}^{(0)} = [1 \ 1]^T$ , determine the value of  $\mathbf{x}$  that minimises  $F(\mathbf{x})$ .
- The Jacobian matrix for this problem is:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix}$$

- Here, the Jacobian matrix is always the same, i.e. it is independent of the point  $\mathbf{x}$  at every iteration.
- Since the number of functions is the same as the number of variables, i.e.  $m = n$ , then the first search direction, according to the Gauss-Newton method is obtained by solving the set of equations given by:

$$\mathbf{J}^{(0)} \mathbf{p}^{(0)} = -\mathbf{f}^{(0)}$$

- Carrying out the necessary calculations, the above set of equations gives us:

$$\begin{bmatrix} 2 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \end{bmatrix} = \begin{bmatrix} 1 \\ -5 \end{bmatrix}$$

- Here  $\mathbf{p}^{(0)} = [p_1 \ p_2]^T$ . Solving (using Gauss Elimination, etc...) gives the search direction:

$$\mathbf{p}^{(0)} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$$

- The next estimate of  $\mathbf{x}$  is given by  $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha \mathbf{p}^{(0)}$ . Carrying out the usual process, we obtain  $\alpha_{\min} = -1$  and therefore:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$$

The function output at this point is  $F(\mathbf{x}^{(1)}) = 0$ , i.e. the best possible fit. Hence our quadratic model takes the form  $y_m = t^2 + 3t - 2$ , as we have already seen!